

Locating the Deployability Wall for Post-Quantum Anonymous Credentials in General-Purpose zkVMs

A Thesis Submitted to the Department of Computer Science and Communications Engineering, the Graduate School of Fundamental Science and Engineering of Waseda University in Partial Fulfillment of the Requirements for the Degree of Master of Engineering

Takumi Otsuka

The Department of Computer Science and Communications Engineering, the
Graduate School of Fundamental Science and Engineering of Waseda University
5124FG15-6

Submission Date : 20 July, 2026

Research Guidance: : Research on Cryptographic Protocols

Advisor: : Prof. Kazue Sako

Abstract

Post-quantum anonymous credentials require a signature whose verification can be proved in zero knowledge. We ask whether the algebraic-hash PLUM signature can be verified inside a general-purpose zero-knowledge virtual machine on a consumer machine (Apple M5 Pro, 24 GB), against a Loquat-instantiated Aurora static circuit as the reference. Without a custom precompile, a single PLUM verification does not complete: it exhausts memory or fails to terminate within three hours. We build a Griffin- $\mathbb{F}_p^{192}$ precompile that absorbs the field-emulation tax; with it the verification proves in about 14.25 minutes (mean of three fixed-config runs) at the 80-bit level, conditional on the constraint-level soundness of the chip. Feasibility for the standalone signature is reached, but the binding constraint moves up the stack: the receipt produced is succinct yet not zero-knowledge, and the only zero-knowledge wraps the toolchain offers are pairing-based, hence not post-quantum, or are blocked by a structural recursion-shape provisioning wall that more memory does not remove. The Aurora reference, with zero knowledge enabled, produces such a proof in about 22 minutes. Deployable post-quantum anonymity is therefore not reached on this stack: we locate this dual obstruction, name the levers that would move it, and report update churn as a secondary deployment-lifecycle cost.

Contents

1	Introduction	9
1.1	Background	9
1.2	The Current State of Post-Quantum Anonymous Credentials	9
1.3	The Hidden Cost: Recompile and Re-audit on Each Change	10
1.4	Zero-Knowledge Virtual Machines as a Structural Alternative	10
1.5	Contributions of This Thesis	11
1.6	Scope and Non-Goals	14
1.7	Thesis Organisation	14
2	Related Work	15
2.1	Post-Quantum Signatures	15
2.2	Anonymous Credentials	16
2.3	Transparent Succinct Arguments and Low-Degree Testing	17
2.4	Zero-Knowledge Virtual Machines	17
2.5	Precompile-Based Cost Reduction	18
2.6	Evaluation Methodology and the Gap This Work Addresses	18
3	Preliminaries	20
3.1	Notation	20
3.2	Cryptographic Primitives	20
3.2.1	Power residue PRF	20
3.2.2	Algebraic hash function	21
3.2.3	Merkle commitment	21
3.3	Digital Signature Scheme	22
3.3.1	Loquat: a Legendre-PRF post-quantum signature with low R1CS verification cost	23
3.3.2	PLUM: a power-residue-PRF post-quantum signature with low R1CS verification cost	25
3.4	zkSNARK	27
3.4.1	Aurora: Transparent Succinct Arguments for R1CS	28
3.5	Anonymous Credential	28
3.5.1	Blockchain-based Digital Education Credentials (BDEC)	30
3.6	Zero-Knowledge Virtual Machines	35
3.6.1	RISC Zero	36
3.6.2	SP1	37
3.6.3	Static-circuit and zkVM proof systems as comparators	37
4	Theoretical Foundations of the Arithmetic Mismatch	38
4.1	Cost Dimensions	38
4.2	The Field-Mismatch Tax	38
4.3	Update Churn	42
4.4	Cost Decomposition of BDEC Relations	43

5	Construction: The Precompile Suite and PLUM-in-BDEC	45
5.1	Scope of the Implementation	45
5.2	The Field-Arithmetic Substrate: $\mathbb{F}_{p^{192}}$ Multiplication via the Host Precompile.....	46
5.3	The Bespoke Griffin- $\mathbb{F}_p^{192}$ Permutation AIR (SP1)	47
5.4	The Sponge and Compression Wrapper	48
5.5	Soundness of the Precompile Suite, and Two Open Questions	48
5.6	Integration into the BDEC Credential-Generation Relation	50
6	Security Analysis	51
6.1	Security Model and Threat Model	51
6.2	Security Preservation	53
6.3	The Zero-Knowledge Barrier as a Finding	56
7	Evaluation	61
7.1	Measurement Methodology	61
7.2	Experimental Setup	61
7.2.1	Settings Justification	61
7.2.2	Reproducibility	63
7.3	PLUM Verification With and Without the Precompile	63
7.4	The Circuit-SNARK Reference Point.....	65
7.5	Cost Attribution	67
7.6	PLUM-in-BDEC and the Consumer-Hardware Frontier.....	68
7.6.1	An internal-verifier rejection rather than a deployability frontier.....	68
7.6.2	The genuine deployability frontier	69
7.7	A Cost Model and the Substrate Crossover.....	70
7.7.1	An additive cost model for the BDEC relations	71
7.7.2	Validation against the measured points	71
7.7.3	The substrate crossover	72
7.8	Update Churn	73
7.9	What the Measurements Establish	74
7.10	Threats to Validity	75
7.11	Summary	77
8	Discussion	78
8.1	The Security–Deployability Tension.....	78
8.2	A Substrate-Selection Rule	78
8.3	What Generalises and What Does Not.....	79
8.4	Comparison to Prior Evaluations.....	80
9	Conclusion	81
9.1	Main Findings	81
9.2	Answer to the Research Question	82
9.3	Limitations	82
9.4	Future Work	83
9.5	Closing	84

Acknowledgement	85
References	86
A Formal Development of the Security Preservation	93
A.1 Building-block properties	93
A.2 Relation preservation	94
A.3 Proof of the preservation theorem.....	94
B Evidence for Assumption 6.1: Specification-Level Soundness of the Griffin-$\mathbb{F}_p^{192}$ AIR	96
B.1 Setup.....	96
B.2 Soundness	97
B.3 Per-family lemmas	97
B.4 Completeness	100
B.5 The argument layer	100
B.6 The open step	100

List of Figures

- 1 One relation, two outer provers. The same BDEC verification relation is proved *either* by a static Aurora circuit (the BDEC baseline) *or* by a general-purpose zkVM (this thesis), and the two are alternatives: the zkVM does not re-prove an Aurora proof. The figure locates the deployability wall: the static Aurora circuit is transparent and zero-knowledge-capable, and the zero-knowledge-enabled reference run is now measured, an accepted post-quantum zero-knowledge proof of the Loquat single-verification building block in about 22 minutes (mean of three runs, 21.4–22.9) on the target machine, whereas the zkVM does not reach that proof on consumer hardware. The agility difference between the two is a footprint distinction that this thesis measures and finds does not translate into a per-proof wall-clock advantage..... 12
- 2 Where the field-mismatch tax is incurred and what the precompile absorbs. Left: each $\mathbb{F}_p^{199}$ operation expands into multi-limb arithmetic that, un-precompiled, does not complete (OOM at 1m45s). Right: a syscall routes it to a native chip, the bespoke Griffin AIR on SP1 or `sys_bigint` on RISC Zero (*not* one artefact), bound to the trace by a cross-table lookup; the chip’s cost is charged host-side, so a guest-cycle saving need not be a wall-clock one. 47
- 3 The dual obstruction. The proof feasible to generate (left, the bare STARK receipt) is not zero-knowledge, hence not anonymous; the proof that would be zero-knowledge (right, a Groth16/PLONK wrap) is blocked, at $\lambda = 80$ on the 24 GB target hardware, by two *independent* prongs: structural (the recursion-shape provisioning wall, with OOM at ≈ 20 GB the default-concurrency symptom) and primitive (pairing-based, not post-quantum, which more memory cannot fix). 57
- 4 Prove time on the configurations that complete ($\lambda = 80$, M5 Pro 24 GB, log scale). The bespoke Griffin precompile (14.25 min) is slower than the SHA-3 control (13.05 min) at matched security, and RISC Zero’s `sys_bigint` route (also SHA-3-hashed) is slower again (6 h 19 m); these are different hash and substrate mechanisms, so the bars are not one artefact. The precompile-free baseline does not complete (DNF, OOM at $\approx 1m45s$) and so has no bar. 64

List of Tables

- 1 Evaluation platform. All prove-mode measurements are local single-machine runs with no network or dev-mode acceleration unless stated. 62
- 2 PLUM single-signature verification, prove mode, $\lambda = 80$, M5 Pro 24 GB. We refer to the three SP1 arms as *Cell 1* (no precompile), *Cell 2* (Griffin- $\mathbb{F}_p^{192}$ precompile), and *Cell 3* (software SHA-3 control) throughout. The precompile-free baseline does not complete; the precompile arm completes in 14.25 min. The SHA-3 arm is a *control* (PLUM with a standard hash computed in software, no hash precompile, only the shared `UINT256_MUL` field-arithmetic precompile active), not a deployable PLUM variant. The Cell 2 and Cell 3 prove times are means of three fixed-config runs on a clean machine (2026-06-30, §7.3); a 2026-06-13 same-session pair gave Cell 2 17.81 min and Cell 3 16.15 min ($n = 2$), and a single 2026-05 run measured Cell 2 32.53 min, the high figure attributed to machine-state contamination and not reproduced under replication. [†]Cell 2 prove time is the mean of three runs, 14.25 min (range 14.05–14.61, ≈ 14.2 GB peak RSS). [‡]RISC Zero receipt-verify wall-clock not separately logged; the receipt verified..... 63
- 3 Aurora over a single Loquat-verification R1CS, M5 Pro 24 GB; mean of three solo runs with the per-run range in parentheses (run records in `docs/measurements/pub_hardening_2026`). This is the *Loquat*-instantiated circuit over $\mathbb{F}_{p^2}$ (127-bit base) [1] at single-verification scope; the full BDEC relation is two such verifications ($\approx$ twice this), not run statically. Prover time includes Aurora parameter setup but is not a recompile-time measurement (§7.8). Achieved security 107.5 bits, below the 128 target. Verification ACCEPT in every run. 65
- 4 Same-scheme substrate comparison: one Loquat verification, $\lambda = 80$, M5 Pro 24 GB, three solo runs per leg, neither zero-knowledge. The Aurora leg is the mean of Table 3; the SP1 leg is the range of three runs (44.4, 47.4, 64.0 min) under the documented reduced-memory configuration (§7.2.1), succinct core STARK, peak RSS 14.1–15.4 GiB, execute-mode cost 397,227,544 cycles (run records in `docs/measurements/pub_ha`). Only the substrate differs (native R1CS versus zkVM emulation); the 12–17 $\times$ prover-time gap is the cost of the zkVM substrate for this scheme. This isolates the substrate for *Loquat*; it does not transfer to PLUM, which has no 127-bit Aurora harness. 66
- 5 Cycle-level cost attribution, PLUM verification, SHA-3 hasher, $\lambda = 128$, *execute mode*. Routing field arithmetic through the host precompile (and removing redundant reductions) cuts 37.9% of SP1 cycles and 70.5% of RISC Zero cycles; RISC Zero’s larger reduction reflects its heavier per-multiplication baseline emulation (6,625 cycles/mul at baseline, falling to 1,826 with the precompile path, against SP1’s baseline of about 2,567 cycles/mul)..... 67

- 6 BDEC credential relations, $\lambda = 80$, M5 Pro 24 GB. On RISC Zero, execute mode validates the honest-input path of CreGen (9.4×10^9 cycles, $\approx 4.69 \times 10^9$ per signature verification); its succinct prove yielded no receipt, from an internal-verifier rejection rather than a resource limit (§7.6.1), not used as a feasibility result. On SP1 both relations complete a *succinct* prove: CreGen in 26.98 and 31.11 min, ShowCre in 40.15 and 44.24 min at $k=1$ and 58.14 min at $k=2$. These receipts are succinct STARKs and are *not* zero-knowledge; the $k=2$ figure agrees with the additive cost model within $\approx 3.3\%$ of the $(k+2)/3$ projection from the $k=1$ mean (run records docs/measurements/pub_hardening_20260611 and audit5_campaign_20260612). 68
- 7 Proof-mode feasibility matrix. Of the modes *produced* on the target hardware by the zkVMs, none is dependably zero-knowledge: the SP1 rows are not zero-knowledge, the RISC Zero receipt carries no formally established zero-knowledge guarantee, and the only zero-knowledge mode the zkVM toolchain exposes (row 3, a pairing wrap) is neither post-quantum nor produced as the toolchain ships, which is the dual obstruction. The zero-knowledge mode that is produced on this hardware is the static-circuit Aurora run (last row; §7.4). [†]The zero-knowledge-enabled Aurora run is now measured (last row); the zk-disabled row is retained for comparison. Both Aurora rows achieve 107.5 bits, below the $\lambda = 128$ target. [‡]RISC Zero’s succinct receipt carries zero-knowledge blinding, but the vendor disclaims a formal zero-knowledge guarantee, so we treat the receipt as not dependably zero-knowledge [2, 3] (Section 6). The SHA-3 row is a control, not a deployable PLUM variant. 102
- 8 Circuit/parameter regeneration footprint ($N_{\text{circ}}, N_{\text{param}}$ of Definition 4.6). The static-circuit regime regenerates and re-audits its R1CS and constraint-dependent parameters on every localised change; the zkVM absorbs a change as a guest-program edit, leaving the universal relation unchanged ($N_{\text{circ}} = N_{\text{param}} = 0$), except where the change crosses the precompile boundary. *The Loquat $\rightarrow$ PLUM swap moves the hash field from $\mathbb{F}_{p^2}$ (127-bit) to the 199-bit $\mathbb{F}_p$, so the field-specific Griffin AIR is regenerated (§4.3 carve-out). The components $N_{\text{src}}, N_{\text{audit}}$ are reported in prose below. 103
- 9 Per-change recompile components for a non-preprocessing (Aurora) versus a preprocessing (Fractal) transparent argument on the same 127-bit-field harness, M5 Pro 24 GB. The 2^{10} – 2^{14} rows are synthetic R1CS instances; the 2^{19} row is the single-Loquat-verification relation at the BDEC building-block size. Aurora has no per-circuit indexer (parameter setup flat at $\sim \mu\text{s}$, so R_{static} stays sub-second); Fractal’s indexer is superlinear and exceeds 24 GB at the 2^{19} size (OOM mid-indexer). The zkVM regenerates no circuit. 103

1 Introduction

1.1 Background

Post-quantum anonymous credentials answer two pressures at once. The credibility of digital credentials and identity attestations is under strain: a single Indian university reportedly sold roughly 36,000 fake degrees [4], forged admission letters have driven student deportations [5], and the 2021 LinkedIn data scrape put approximately 700 million user profiles up for sale on the dark web [6], so a credential system must be at once *verifiable* and *privacy-preserving*. This is now a regulatory requirement and not only an academic one: the European Digital Identity framework (eIDAS 2.0, Regulation (EU) 2024/1183) mandates privacy-preserving attribute attestation with unlinkability for the EU Digital Identity Wallet [7]. A credential meeting it must also be *long-lived*, secure against adversaries equipped with large-scale quantum computers, a requirement that rules out classical anonymous-credential constructions whose security reduces to the discrete-logarithm or RSA assumption and that motivates the migration to standardised post-quantum primitives [8, 9]. Constructions meeting all three requirements at once are only now emerging [10, 1], and the post-quantum signatures on which such systems are built, with verification cheap enough to prove in zero knowledge, are themselves an active line [11, 12, 13]. *Constructing* such a system is the prior question, taken up by those works; this thesis takes up the next one, whether such a system can actually be *deployed*.

Any such credential proves, in zero knowledge, that a post-quantum signature verifies, and that proof must be generated on whatever machine the credential holder owns. The dominant approach compiles the verification predicate into a fixed arithmetic circuit and proves it with a transparent zk-SNARK such as Aurora [14]. This specialisation is efficient, but a deployment is not static: the signature scheme is updated as standards mature, security parameters are retuned, and disclosure policies evolve, and a circuit fixed at deployment time must be regenerated and re-audited on each such change. A general-purpose zero-knowledge virtual machine (zkVM) [15, 16, 17] is an alternative substrate, in which a single universal relation proves the correct execution of an ordinary program, so a change to the predicate becomes a program edit rather than a new circuit. These two substrates trade different costs, and which to choose for deployment on consumer hardware is a live question that the literature has not measured.

The decision belongs to the operator of a real system, a university consortium issuing digital diplomas choosing a proving substrate today, who needs to know whether holders can generate the privacy-protecting proof on their own machines, and what each future scheme or parameter change will cost in regenerated and re-audited artefacts. This thesis answers both by measurement, taking the BDEC anonymous-credential framework [1] instantiated with the PLUM signature [18] as the concrete workload: BDEC-PLUM is the use case, the comparison of proving substrates is the approach, and the dual obstruction it reveals is the finding.

1.2 The Current State of Post-Quantum Anonymous Credentials

Two representative lines of work have attempted to construct post-quantum anonymous-credential systems (the broader lattice-native and PQ-token literature is surveyed in Section 2). The first follows the lattice-based approach exemplified by Jeudy et al. at Crypto 2023 [10], leveraging the hardness of the Short Integer Solution (SIS) and Learning With Errors (LWE) problems. While this approach achieves

quantum resistance, it currently supports only a single message per credential and yields large proof and key sizes that make repeated showings cumbersome.

The second line of work, exemplified by the recent Blockchain-based Digital Education Credential (BDEC) framework of Li, Zhang et al. [1], takes a symmetric-primitive-based approach. BDEC instantiates a generic anonymous-credential construction with the Loquat post-quantum signature scheme [11], whose verification predicate is comparatively cheap to encode in rank-1 constraint systems (R1CS), and the Aurora transparent zkSNARK [14], which proves the R1CS relation in zero-knowledge. The resulting system achieves unforgeability, anonymity, unlinkability, conditional linkability, and revocability, and, per the authors’ analysis, outperforms the lattice-based approach [1].

A more recent post-quantum signature, PLUM [18], is a successor to Loquat that replaces the Legendre PRF with a t -th power residue PRF and the FRI low-degree test with the STIR protocol. At the 128-bit security level, PLUM signatures are approximately $1.5\times$ smaller than Loquat’s, and PLUM verification is expected to be up to $4\times$ faster, while requiring roughly $1.28\times$ fewer R1CS constraints [18]. Because PLUM shares Loquat’s overall structure, a symmetric-primitive PRF assumption verified through a transparent low-degree test, while reporting lower verification cost at matching security, it is the signature primitive we adopt for the zkVM instantiation studied in this thesis. PLUM is a research signature, not yet standardised, chosen for the low R1CS constraint count of its verification predicate. The NIST post-quantum signature standards (ML-DSA, SLH-DSA) and the forthcoming FN-DSA remain the deployment baselines. Their lattice and hash verification predicates are precisely the ones that are expensive to arithmetise, and that is why a symmetric-primitive design is studied here.

1.3 The Hidden Cost: Recompilation and Re-audit on Each Change

Both Loquat and PLUM are designed to be efficiently verifiable inside a zkSNARK; in BDEC the verification predicate is compiled into a fixed R1CS for the Aurora transparent argument at deployment time, and this specialisation is what makes the system efficient. Specialisation has a deployment-lifecycle cost that is rarely measured. Any change to the verification *relation*, whether substituting the signature scheme (Loquat $\rightarrow$ PLUM, or a future Sig’), retuning a security parameter, or changing the presentation *structure*, requires recompilation of the R1CS matrices and re-derivation of any parameters that depend on the constraint count, and a deployment that tracks evolving standards, institutional policies, and threat models incurs this cost repeatedly. (A presentation *policy* that only changes which condition the verifier checks on the disclosed attributes, at fixed structure, is handled verifier-side and forces no recompilation.) We refer to this hidden cost as the *recompilation-and-re-audit cost* of the static-circuit approach, and to its quantitative manifestation as *update churn*. The per-change recompile wall-clock turns out to be small (Section 7.8); what the static regime pays per change is a regeneration-and-re-audit footprint, and characterising that footprint, rather than presupposing a wall-clock gap, is the form in which rigidity enters this thesis’s evaluation.

1.4 Zero-Knowledge Virtual Machines as a Structural Alternative

A zero-knowledge virtual machine (zkVM) is a zkSNARK whose underlying relation is fixed once, at design time of the prover, to the correctness of executing a program on a deterministic instruction-set architecture [15, 16, 19]. Anything that a developer can express in a high-level language becomes a guest program; the proof system is universal. Updating the verification predicate becomes a guest-

program edit under a fixed universal relation. The audit obligation does not vanish, since the guest binary, its image ID, the host I/O binding, and any precompile interface still require review, but it changes in kind: from auditing a *new relation-specific circuit* to auditing *software and its image under the same, unchanged universal relation*.

In exchange for this generality, a zkVM pays a cost on algebraic primitives whose native arithmetic does not match the zkVM’s prover field. PLUM verification involves tens of thousands of explicit multiplications over a large prime field, beside the algebraic hash whose every permutation is itself large-field arithmetic (PLUM’s specification labels this field 192-bit, though the concrete modulus we instantiate is 199 bits; see Remark 3.5), while modern zkVM provers operate over 31-bit native fields (BabyBear, $2^{31} - 2^{27} + 1$, in RISC Zero; KoalaBear, $2^{31} - 2^{24} + 1$, in the SP1 fork evaluated here); Loquat is analogous, over the quadratic extension $\mathbb{F}_{p^2}$ of the 127-bit Mersenne prime field. Each large-field operation must therefore be emulated by multi-limb arithmetic over the small field. We refer to this cost as the *field-mismatch tax*. Without mitigation, the field-mismatch tax can render PQ-anon-credentials-in-zkVM impractical on consumer hardware: in our measurements the precompile-free standalone PLUM verification exhausts memory (OOM) on the SP1 emulation arm on Apple M-series hardware with 24 GB, and the full credential relation’s succinct proof did not complete, in an anomaly examined in Section 7.

Both RISC Zero [16] and SP1 [20] permit *application-defined precompiles*: native sub-circuits for designated primitives that the prover recognises as syscalls. A precompile can absorb the field-mismatch tax for a primitive that would otherwise dominate the trace. The question, then, is whether a precompile suite covering the algebraic primitives of PLUM verification can make PLUM-instantiated BDEC tractable inside a zkVM on consumer hardware, and how the resulting system compares to the static-circuit baseline along the dimensions that actually matter for deployment.

1.5 Contributions of This Thesis

This thesis asks whether a general-purpose zkVM is a viable substrate for post-quantum anonymous credentials on consumer hardware, and answers: *not yet*. On the target hardware the proof one can afford to generate, the standalone PLUM-verification succinct receipt, is not zero-knowledge, while the only zero-knowledge wraps the SP1 toolchain exposes are pairing-based, hence not post-quantum, and the attempted PLONK wrap does not complete on the stack as shipped, blocked by the recursion-shape provisioning wall rather than by a memory bound; on RISC Zero the succinct receipt carries zero-knowledge blinding but no formally established zero-knowledge guarantee, and its pairing wrap does not run on this hardware at all. The provisioning wall is structural rather than a memory bound: it is surmountable only by a one-time regeneration of the toolchain’s static recursion artifacts, not by adding memory; the wrap inventory is a fact about the deployed toolchains that no timing number moves. We call this the *dual obstruction*, and it is the central finding. Around it, two further measurements falsify natural predictions: the bespoke algebraic-hash precompile proves *slower* than a software SHA-3 control at matched security, and the zkVM’s flexibility advantage over the transparent static circuit never appears in wall-clock. The contributions below are the artifact, the measurements, and the analysis that locate exactly where the wall sits and what would move it.

1. **A PLUM-instantiated BDEC artifact and its frontier measurements.** We express the BDEC credential-generation (CreGen) relation as a guest program on RISC Zero with PLUM in place of Loquat, realise the full ShowCre presentation relation as a $k + 2$ -verification guest, and run

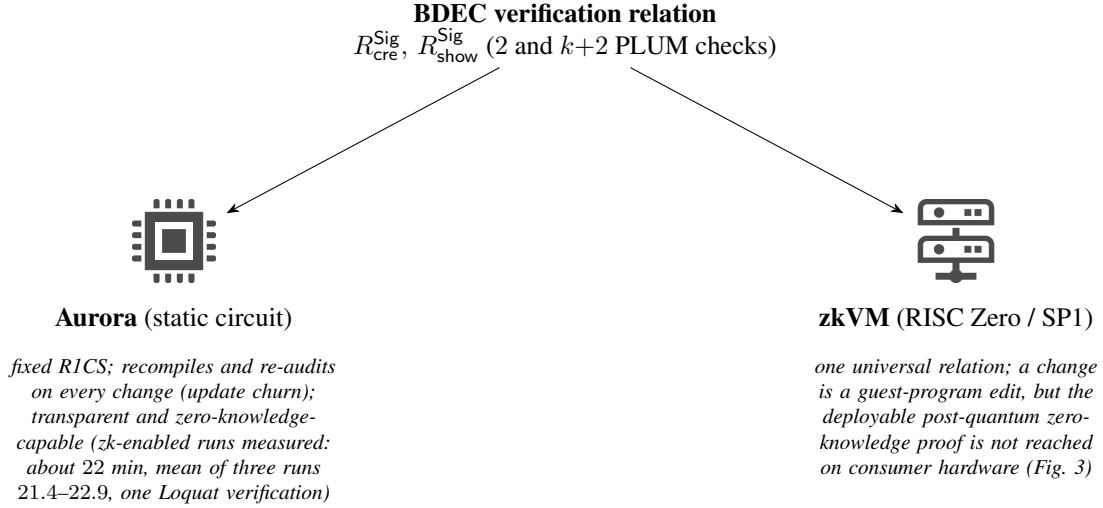


Figure 1: One relation, two outer provers. The same BDEC verification relation is proved *either* by a static Aurora circuit (the BDEC baseline) *or* by a general-purpose zkVM (this thesis), and the two are alternatives: the zkVM does not re-prove an Aurora proof. The figure locates the deployability wall: the static Aurora circuit is transparent and zero-knowledge-capable, and the zero-knowledge-enabled reference run is now measured, an accepted post-quantum zero-knowledge proof of the Loquat single-verification building block in about 22 minutes (mean of three runs, 21.4–22.9) on the target machine, whereas the zkVM does not reach that proof on consumer hardware. The agility difference between the two is a footprint distinction that this thesis measures and finds does not translate into a per-proof wall-clock advantage.

standalone PLUM signature verification on SP1. In execute mode at $\lambda = 80$, CreGen executes both of its signature verifications to acceptance in 9.4×10^9 guest cycles, and ShowCre measures 1.41×10^{10} cycles at $k = 1$ and 1.88×10^{10} at $k = 2$, confirming the additive cost model. In prove mode on the target hardware (Apple M5 Pro, 24 GB), standalone PLUM verification proves in 14.25 minutes (mean of three fixed-config runs, 14.05–14.61; Section 7) on SP1 at $\lambda = 80$, under the documented reduced-memory configuration, default settings exhausting memory; on RISC Zero the CreGen succinct prove terminated in an internal-verifier anomaly distinct from a resource limit (under diagnosis, Section 7, with no claim of this thesis resting on it), whereas on SP1 both BDEC relations complete a succinct, non-zero-knowledge prove, CreGen in 26.98 and 31.11 min and ShowCre in 40.15 and 44.24 min at $k=1$ and 58.14 min at $k=2$. We report cycle counts, wall-clock times, peak resident-set sizes, and prove and verify times where proving completes, with parameters and procedures documented for reproducibility. To our knowledge this is the first realisation of PLUM-based BDEC components in execute mode and the first standalone PLUM-verify prove-mode measurement inside a general-purpose zkVM.

2. **The deployability wall: a dual obstruction to post-quantum anonymity.** Neither BDEC relation yields a zero-knowledge *proof* on the target hardware, and we locate why. The measured frontier datum is the wrap obstruction: at default concurrency the PLONK wrap of the standalone PLUM-verification receipt is killed at roughly 20 GB of the 24 GB budget, but this is a concurrency artifact rather than a memory bound: throttled, the wrap runs well inside the budget and

fails instead at recursive compression, where the recursion shapes are provisioned for smaller relations than PLUM verification; the ShowCre wrap, larger, is a fortiori unreachable. The receipt one can generate is succinct but not zero-knowledge on SP1, whose only zero-knowledge wraps are pairing-based and so not post-quantum; on RISC Zero the succinct receipt is blinded but carries no formally established zero-knowledge guarantee, and the pairing wrap is unavailable on Apple silicon. Deployable post-quantum anonymity is therefore not reached on this stack, by a route that is the recursion-shape provisioning wall rather than a memory limit, a structural fact that more memory does not remove.

3. **The precompile as instrument, a falsified speedup, the wall it builds, and the cost model behind them.** We design and implement a Griffin- $\mathbb{F}_p^{192}$ precompile [21] (a bespoke AIR on SP1; on RISC Zero the same arithmetic is carried by `sys_bigint`) for the algebraic-hash primitive that the cost analysis of Section 4 identifies as the dominant tax-bearing operation, integrating it with the zkVMs’ existing big-integer modular-multiplication precompiles (SP1’s `UINT256_MUL` and RISC Zero’s `sys_bigint`) [20, 16]. Against the precompile-free path, which exhausts memory at $\approx 1\text{m}45\text{s}$ (Table 2), the precompile turns standalone PLUM verification into the finite 14.25-minute measurement above; introducing a custom AIR adds a trusted component to the verification path, so the security statement is *conditional* on the constraint-level soundness of the Griffin AIR, carried as open Assumption 6.1 in Section 6. The precompile does not, however, deliver a speedup where one was expected: at matched $\lambda = 80$ the Griffin arm proves slower than a PLUM arm using a software SHA-3 hasher and the big-integer precompile alone in every run we took, on both the May and the June builds. The sign of this inversion is robust; under fixed-config replication ($n = 3$ per arm) its magnitude is $\approx 1.09\times$ in prove wall-clock, the metric on which deployment cost depends, an earlier $2.45\times$ single run retired as machine-state contamination. We report this inversion as a substantive finding, and the trace-area cost model of Section 4 postdicts its sign together with the feasibility recovery: a precompile pays exactly when the core trace area it removes exceeds the chip area it adds, an inequality these two measurements evaluate at opposite operating points. A third measurement makes the theme structural: the Griffin chip that recovers feasibility is the sole cause of the recursion-shape provisioning wall that blocks the zero-knowledge wrap (Section 7). Removing only the chip restores the pre-precompile recursion shape byte-for-byte; with it, the shape grows by 34.8% and overflows the static provisioning the wrap depends on. The instrument that recovers feasibility at the core layer is the one that bills the recursion layer, and the field-mismatch tax it removes from the guest reappears, transformed, one layer up.
4. **A secondary dimension: *update churn* and a deployment decision framework.** We define update churn quantitatively as the regeneration footprint a localised change to the verification relation forces: the set of compiled artefacts and proof-system parameters that must be regenerated and re-audited. We enumerate this footprint for both regimes and measure the recompile cost it was presumed to impose. Against the transparent, non-preprocessing Aurora baseline the presumed wall-clock advantage of the zkVM is absent (the static recompile is sub-second), surviving only as a regeneration-and-re-audit footprint; against a preprocessing baseline (Fractal) the advantage is real and large, the per-circuit indexing itself exceeding the memory budget at the BDEC size. From these measurements and the frontier above we give a decision framework mapping deployment-scenario parameters (policy-update frequency, hardware bud-

get, anonymity-strength requirement) onto a regime recommendation, separating the dimensions on which the comparison is conclusive from those bounded by the consumer-hardware proving frontier.

1.6 Scope and Non-Goals

This thesis is concerned with the practical deployability of PQ anonymous credentials on consumer hardware. We therefore make the following scope choices.

We focus on the BDEC framework [1] as the anonymous-credential construction, because (i) it is the leading symmetric-primitive-based PQ anonymous-credential system in the recent literature, (ii) its structure of two signature-verification predicates under a hidden user public key inside a zkSNARK is representative of a broad class of PQ-anon-cred designs, and (iii) its reliance on Aurora makes the static-circuit baseline well-defined and reproducible.

We focus on RISC Zero and SP1 as the zkVM substrates, because both expose application-defined precompiles, both target consumer hardware, and both serve as representative examples of the current generation of RISC-V-based zkVMs.

We do *not* propose a new post-quantum signature scheme: we take PLUM as given and our contribution lies in how it is embedded into the broader credential system. We do *not* propose new cryptographic assumptions: the security of the resulting system reduces to those already established for Loquat, PLUM, BDEC, RISC Zero, and SP1, together with a small number of premises that we state explicitly and carry as *open* where they arise (the constraint-level soundness of the Griffin AIR, the existence of a PLUM signature-transcript simulator, and the quantum-random-oracle lifts), discharged in Section 6 only as far as those references allow. We do *not* claim that consumer-hardware PQ anonymous credentials are universally deployable today; rather, we quantify the cost frontier and identify the load-bearing pieces of the cost.

1.7 Thesis Organisation

The remainder of this thesis is structured as follows. Section 2 surveys related work on post-quantum signatures, anonymous credentials, zkSNARKs, zkVMs, and precompile design. Section 3 establishes the cryptographic primitives, signature schemes, zkSNARK and zkVM definitions, the BDEC construction, and the informal notion of update churn used throughout the thesis. Section 4 gives the formal definition of update churn and formalises the question we ask of each system regime and the comparative analysis we apply. Section 5 describes the design and implementation of PLUM-in-BDEC in both the static-circuit and the zkVM regimes, including the precompile suite. Section 6 states the conditional security-preservation theorem, its two open assumptions (Griffin-AIR soundness and PLUM transcript simulatability), and the folded lookup-binding and black-box-transfer hypotheses. Section 7 reports the empirical measurements on consumer hardware. Section 8 interprets the measurements as a deployment-lifecycle decision framework. Section 9 concludes and identifies open problems.

2 Related Work

The Introduction set out the thesis question: whether a general-purpose zkVM can verify an algebraic-hash post-quantum signature feasibly on consumer hardware, and where the deployability wall for post-quantum anonymity then sits. Five strands of prior work bear on it. Post-quantum signatures and anonymous credentials establish what BDEC is assembled from; transparent succinct arguments and zero-knowledge virtual machines are the two substrates this thesis sets against each other; and precompile-based cost reduction is the instrument that makes the zkVM substrate measurable at all. We read each strand with one question in view: what the existing evaluation literature measures and what it leaves unmeasured. The closing subsection turns to the omission this thesis fills. A strand is included when it changes the comparison on one of four axes, namely post-quantum credential security, in-circuit post-quantum signature verification, zkVM proving cost, or update churn, which is why deployed verifiable-credential standards and BBS appear as background rather than as direct baselines.

2.1 Post-Quantum Signatures

The threat that large-scale quantum computers pose to RSA- and discrete-logarithm-based signatures has driven a multi-year standardisation effort. The NIST post-quantum cryptography process has produced two general-purpose signature standards, ML-DSA [8] (a lattice-based scheme derived from Dilithium) and SLH-DSA [9] (a stateless hash-based scheme derived from SPHINCS+), with a third, Falcon (FN-DSA, draft FIPS 206 [22]), also lattice-based. These signatures are designed for general-purpose authentication and are well suited to consumer hardware; they are, however, not well suited as the underlying primitive of an anonymous-credential system because their verification predicates do not embed cheaply inside a zkSNARK. Lattice-based verification involves expensive rejection sampling and norm checks, and hash-based verification involves on the order of 10^4 symmetric-primitive invocations per signature verification (5,000–20,000 for SLH-DSA-128f; signing costs a further one to two orders of magnitude) [9], both of which translate into large in-SNARK constraint counts. This concerns the *general* verification predicate. Narrower-functionality results, such as post-quantum Privacy Pass [23] and hardware or precompile acceleration of the hash-based families, reduce these costs in restricted settings, so the “not well suited” judgement applies to the general anonymous-credential predicate and admits these exceptions in restricted ones.

A complementary line of work develops post-quantum signatures whose verification predicate is deliberately structured for a low constraint count when encoded as an R1CS. Picnic, Banquet, and Limbo build signatures around the MPC-in-the-head paradigm [24, 25, 26]. Loquat [11] departs from this line: it bases unforgeability on the Legendre PRF, a symmetric-primitive assumption distinct from lattice and code-based hardness, and is designed so that its verification circuit fits into a comparatively small R1CS. PLUM [18], the focus of this thesis, replaces Loquat’s Legendre PRF with the t -th power residue PRF and replaces the FRI low-degree test with the STIR protocol. The result is, at 128-bit security, signatures approximately $1.5\times$ smaller, verification approximately $4\times$ faster, and approximately $1.28\times$ fewer R1CS constraints per verification, while preserving Loquat’s security guarantees [18]. This trio of improvements is what motivates evaluating PLUM as a replacement for Loquat in the BDEC framework; whether these static-circuit advantages carry over to a zkVM, where the verification runs as emulated execution rather than as a native circuit, is a question this thesis examines, and our SHA-3 control (Section 7) indicates they do not transfer straightforwardly. The same assumption

family has since been extended by Pegasus and PegaRing [27], newer Σ -protocol (ring) signatures built on the power-residue PRF by a Monash group overlapping Loquat’s authors.

A direct competitor in this low-constraint line is CAPSS [28], a framework for SNARK-friendly post-quantum signatures built from arithmetisation-oriented permutations, Griffin among them. CAPSS reports a $5\text{--}8\times$ reduction in R1CS constraints relative to Loquat, so neither Loquat nor PLUM can be read as the lowest-constraint design in this line. It further includes an anonymous-credential application, and it reports measured Aurora/libiop proving figures obtained with zero-knowledge-enabled executables, so measured Aurora proving of an arithmetisation-oriented-hash post-quantum signature predates this thesis. CAPSS contains no zkVM measurement: it occupies SNARK-friendly signature design and in-Aurora evaluation, while this thesis occupies the cross-substrate comparison, the custom large-prime-field precompile inside a general-purpose zkVM, and the attribution of cost between the two substrates. The schemes also differ as much in *assumption* as in constraint count, since Loquat and PLUM rest on the Legendre and power-residue PRFs, MPC-in-the-head designs on generic symmetric primitives, lattice schemes on SIS/LWE, and every algebraic-hash argument additionally on a random-oracle instantiation of its hash, so a constraint-count comparison alone does not settle the choice.

2.2 Anonymous Credentials

Anonymous credentials originate with Chaum’s pseudonymous-credentials proposal [29], formalised by Camenisch and Lysyanskaya (CL) [30], and have since developed into a substantial line including Pointcheval–Sanders signatures, Coconut, and the BBS family [31, 32, 33]. These remain the basis of deployed selective-disclosure systems (W3C Verifiable Credentials [34], BBS-based showing), which we treat as deployment baselines rather than post-quantum-anonymous-credential baselines, since none is quantum-resistant. These classical constructions rely on pairing-based or discrete-logarithm-based assumptions, and are therefore not quantum-resistant. The post-quantum case has, until recently, been pursued largely through lattice-based primitives: Jeudy et al. at Crypto 2023 [10] provide an efficient lattice-based anonymous-credential scheme, leveraging the SIS and LWE assumptions; subsequent work has refined and applied this template. The known limitations of the lattice approach include large proof and key sizes and, in the Jeudy et al. scheme as published, support for only a single message per credential. A second route instantiates the generic construction of post-quantum anonymous credentials from a general-purpose zero-knowledge proof of signature verification: Policharla et al. [23] build a post-quantum Privacy Pass by proving verification of zkDilithium, their STARK-friendly modification of Dilithium2, inside a hand-written Winterfell STARK (2023). That construction pre-empts the generic idea; it uses no zkVM and modifies the signature scheme to fit the prover, so the novelty of this thesis rests on the substrate comparison, the measurement of an unmodified scheme, and the precompile.

The Blockchain-based Digital Education Credential (BDEC) framework of Li, Zhang et al. [1] takes a different path. BDEC is generic over a signature scheme and a zkSNARK, and the authors instantiate it with Loquat and Aurora to obtain a post-quantum anonymous-credential system whose security reduces to collision-resistance and the Legendre PRF. The system supports unforgeability, anonymity, unlinkability, conditional linkability, and revocability, and the authors report concrete-performance improvements over the lattice baseline of Jeudy et al. [10]. The BDEC construction is the anonymous-credential framework this thesis builds on; we examine it in detail in Section 3. We

distinguish BDEC-as-proposed from the instantiation we measure: BDEC’s anonymity and unlinkability require the underlying argument to be zero-knowledge, which the consumer-hardware zkVM receipt we produce does not yet satisfy (Section 6), so our artifact does not inherit those properties unconditionally.

2.3 Transparent Succinct Arguments and Low-Degree Testing

A zkSNARK proof system consists of an interactive-oracle-proof (IOP) compiled into a non-interactive argument via a hash-based transformation, most commonly the Ben-Sasson–Chiesa–Spooner (BCS) transformation [35]. The BCS transformation requires a low-degree test that confirms a committed function is close to a low-degree polynomial. Two such tests dominate recent transparent-argument constructions: FRI (Fast Reed–Solomon Interactive Oracle Proof of Proximity) [36] and STIR (Reed–Solomon proximity testing with fewer queries) [37]. STIR achieves lower query complexity and a smaller code-rate after folding, at the cost of slightly more involved verifier logic; both rest on Reed–Solomon proximity-gap results [38].

Aurora [14] is the canonical transparent succinct argument for R1CS in the IOP-plus-BCS family. It produces zero-knowledge proofs of R1CS satisfiability with polylogarithmic proof size but *linear* verifier complexity, where the verifier reads the explicit R1CS matrices [14, Thm. 1.2], with no relation-specific trusted setup. (The holographic variant Fractal [39] attains a polylogarithmic verifier through preprocessing; we use plain Aurora as the BDEC baseline.) Aurora is the zkSNARK choice of the BDEC framework as instantiated by Li, Zhang et al. [1], and is therefore the static-circuit baseline against which we compare in this thesis. Other transparent succinct arguments share the property of relation-specific R1CS encoding and therefore the same recompilation-and-re-audit cost discussed in Section 1; the holographic ones (Fractal, Marlin [40]) differ in one respect that matters for the recompile cost we measure, namely that they preprocess each relation into an index, so their per-circuit setup is large rather than negligible, unlike the non-preprocessing Aurora that the BDEC baseline uses.

2.4 Zero-Knowledge Virtual Machines

A zero-knowledge virtual machine (zkVM) is a zkSNARK whose underlying relation is the correctness of executing a guest program on a fixed instruction-set architecture [15, 19]. The term “zkVM” is ecosystem terminology: the *default* receipt these systems produce is succinct but not necessarily zero-knowledge, a gap that becomes this thesis’s central security finding (Section 6). Several zkVM designs are mature enough to be considered for credential-system deployment:

- **RISC Zero** [16] is a RISC-V zkVM whose prover is a recursive STARK over the BabyBear field. Application-defined precompiles are supported via the Zirgen big-integer dialect.
- **SP1** [20] is a RISC-V zkVM whose prover is built on the Plonky3 toolkit. SP1 supports application-defined precompiles via its custom-precompile framework, and offers Groth16 and PLONK proof modes that internally compress the succinct STARK receipt and wrap it to obtain a zero-knowledge, succinctly-verifiable proof.
- **Jolt** [41] is a RISC-V zkVM whose prover uses a sum-check-and-lookup approach instead of segment-based STARK proving. Jolt’s design optimises for guest cycle counts of moderate size and is included in this discussion for completeness.

A separate strand of work, exemplified by zkLLVM [42], takes a compiler approach in which high-level programs are compiled into circuit-friendly intermediate representations rather than RISC-V; this approach inherits the recompilation-and-re-audit cost of static circuits and is therefore positioned closer to the Aurora regime than to the zkVM regime as we use the term in this thesis. The same applies to circuit-frontends such as Circom [43] and Noir, which target a single underlying SNARK system and lock the verification predicate at compile time.

2.5 Precompile-Based Cost Reduction

The general technique of replacing emulated computation in a zkVM with a native AIR-based sub-circuit, recognised by the prover via a syscall, has been applied to several primitives. The R0VM 2.0 release adds precompiles for BN254 and BLS12-381 elliptic-curve operations and reports reducing Ethereum block-proving from approximately 35 minutes to approximately 44 seconds [44]. Each such precompile introduces a primitive-specific arithmetisation that becomes part of the trusted computing base and must be audited, as our Griffin precompile is (Assumption 6.1, Section 6). The speedup therefore comes with an audit obligation. Arguzz [45], a metamorphic-testing fuzzer for soundness and completeness bugs in zkVMs, found eleven such bugs across six zkVMs, including a RISC Zero soundness bug that earned a \$50,000 bounty despite prior audits; results of this kind motivate the written soundness arguments that accompany the precompile in this thesis. A community RSA precompile for zkVMs has likewise been reported to cut RSA signature verification from tens of millions of guest cycles to the order of 10^5 , roughly two orders of magnitude.¹ SP1’s standard library ships SHA-256 and ECDSA precompiles by default [20]; RISC Zero ships SHA-256 and BigInt-256 modular-multiplication precompiles by default [16].

These results share a structural pattern. A fixed primitive whose software emulation over the prover’s native field would dominate the trace is replaced by a native AIR, and the per-call cost from the guest’s perspective collapses to a single syscall. The Griffin- $\mathbb{F}_p^{192}$ precompile proposed in this thesis sits within this pattern, with the additional consideration that the primitive in question is parameterised over a non-trivial 199-bit prime field (labelled $\mathbb{F}_p^{192}$ in the syscall name; Section 5), whose multiplications are carried by the zkVMs’ existing 256-bit modular-multiplication precompiles via zero-padding, so no field-width-specific bigint circuit is required.

2.6 Evaluation Methodology and the Gap This Work Addresses

The recent literature on PQ-anonymous-credential systems reports evaluation along the conventional dimensions: prover wall-clock time, verifier wall-clock time, proof size, and, less commonly, peak memory [1, 18]. The same is true of the zkVM literature, which adds cycle counts and segment counts to the standard list.

The cost incurred when the verification predicate must be changed, in response to a change in the presentation structure, a primitive substitution, or a security-parameter retuning, is not reported alongside these dimensions. The concept that such changes carry a migration cost is itself not new: crypto-agility work motivates this cost as a design concern, and post-quantum verifiable-credential proposals have noted the need to re-derive parameters when the underlying primitive changes. For a

¹Reported in vendor/community engineering write-ups; we cite it only as an order-of-magnitude indication of the precompile pattern, not as a peer-reviewed measurement.

static-circuit deployment that change requires recompilation of the R1CS matrices and re-derivation of any parameters that depend on the constraint count; for a zkVM deployment it requires only an edit to the guest program. Without accounting for this cost, comparisons between the two regimes omit the regeneration-and-re-audit footprint that each relation change imposes on the static side. We measure that footprint in Section 7, and we report there that its wall-clock component is sub-second against a transparent non-preprocessing argument, so the separation between the two regimes is one of re-audit footprint rather than recompile time, and is itself conditional on the static baseline’s preprocessing model. This is a deployment-lifecycle cost, paid once per relation change, and it falls outside the per-proof wall-clock figures that the prior work reports. The prior work that names the concern stops short of measuring it. We introduce *update churn* as a measurement dimension and report it alongside the conventional dimensions for both regimes.

The closest prior work confirms that this measurement dimension is unreported. Post-quantum anonymous credentials have been built lattice-natively (Bootle et al. [46]; Argo et al. [47], the latter with sub-80 KB credentials and sub-second native showing), classically from zkSNARKs composed with existing identity infrastructure (zk-creds [48], which is pairing-based and not post-quantum), and on a hand-rolled relation-specific STARK for a lattice signature (Policharla et al. [23]); none runs its verification predicate inside a general-purpose zkVM, and none reports the per-change deployment cost. Low-constraint post-quantum signature *design* has likewise been generalised into a framework (CAPSS [28], discussed above), which carries an anonymous-credential application and measured Aurora proving figures but contains no zkVM measurement. On the substrate side, individual post-quantum *signatures* have recently been verified inside general-purpose zkVMs as engineering demonstrations: s2morrow [49] batch-verifies Falcon-512 and SPHINCS+-128s in the Cairo zkVM proven with Stwo, published as a repository with no accompanying paper, no published proving benchmarks, no custom precompile, and no credential application; HAPPIER [50] verifies XMSS aggregate signatures in RISC Zero on a laptop using the built-in SHA precompile; and a practitioner implementation [51] verifies full ML-DSA (Dilithium) in SP1 through a guest-code NTT gadget at roughly 5.6 million cycles, with proofs produced on the SP1 prover network. None of these works authors a custom precompile or compares substrates, and we therefore do not claim the first post-quantum signature in a zkVM. Our claim is narrower and measured: to our knowledge, the first reported execution-and-proving study of an algebraic-hash post-quantum *anonymous credential* (BDEC instantiated with PLUM) inside a general-purpose zkVM. The gap this work fills is therefore the deployability question none of the above answers: whether a general-purpose zkVM can carry such a credential system to a zero-knowledge, post-quantum proof on consumer hardware, and where it fails when it cannot. No prior work measures that wall; this thesis locates it, and reports the update-churn dimension alongside it as the secondary axis on which the two substrates were presumed to differ. We position against the recent zkVM systematisation [17], which benchmarks general-purpose zkVMs on classical workloads but measures no post-quantum anonymous credential. The next section develops the primitives the measurement rests on, beginning with PLUM, BDEC, and the two argument substrates.

3 Preliminaries

3.1 Notation

Let λ denote the security parameter. A function $f(\lambda)$ is negligible if for every polynomial p there exists N such that, for all $\lambda > N$, $f(\lambda) < 1/p(\lambda)$. We write $\text{negl}(\lambda)$ for an unspecified negligible function. All algorithms and adversaries considered below are classical probabilistic polynomial-time unless stated otherwise; security statements inherited from the underlying signature schemes hold in the classical random-oracle model, and any quantum (QROM) variant is flagged explicitly where it arises. For a randomized algorithm A , the notation $y \leftarrow A(x)$ means that A is run on input x using fresh randomness and outputs y . For a finite set S , $y \leftarrow S$ means that y is sampled uniformly from S . We write $[n] = \{1, \dots, n\}$. Attribute vectors are denoted by $A = (a_1, \dots, a_m)$, and predicates over attributes are denoted by ϕ . Public keys, secret keys, messages, and signatures are denoted by pk , sk , m , and σ , respectively.

Definition 3.1 (Efficiently decidable relation). A relation $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ is efficiently decidable if there exists a deterministic polynomial-time algorithm D_R such that, for every pair (x, w) ,

$$D_R(x, w) = 1 \iff R(x; w) = 1.$$

We call x the public instance and w the witness. The statement $R(x; w) = 1$ means that w is a valid witness for the public instance x .

3.2 Cryptographic Primitives

This subsection collects the primitives that appear inside both the signature schemes (Loquat, PLUM) and the BDEC construction below. We work with a single odd prime p throughout; the same field is reused by every primitive in any one instantiation, eliminating field-lifting overhead.

3.2.1 Power residue PRF.

Definition 3.2 (t -th power residue PRF). Let p be an odd prime and let $t \geq 2$ be an integer with $t \mid (p-1)$. Let $g \in \mathbb{F}_p^*$ be a generator and let

$$\zeta_t = g^{(p-1)/t} \in \mathbb{F}_p^*$$

be a primitive t -th root of unity, so that the cyclic subgroup

$$\mu_t = \langle \zeta_t \rangle = \{1, \zeta_t, \zeta_t^2, \dots, \zeta_t^{t-1}\}$$

is the set of t -th roots of unity in $\mathbb{F}_p^*$. For a key $K \in \mathbb{F}_p$ and an input $a \in \mathbb{F}_p$ with $K + a \neq 0$, define

$$\chi_t(K, a) = (K + a)^{(p-1)/t} \in \mu_t,$$

and let $L_t(K, a) \in \mathbb{Z}/t\mathbb{Z}$ be the unique exponent for which

$$\chi_t(K, a) = \zeta_t^{L_t(K, a)}.$$

The keyed function $L_t(K, \cdot) : \mathbb{F}_p \setminus \{-K\} \rightarrow \mathbb{Z}/t\mathbb{Z}$ is the t -th power residue PRF.

The special case $t = 2$ makes $\chi_2(K, a) \in \{+1, -1\}$ the Legendre symbol of $K + a$ modulo p ; the corresponding L_2 is the Legendre PRF used by Loquat. Taking $t > 2$ yields more entropy per evaluation. The PLUM construction described later in this section pins $t = 256$. Definition 3.2 only fixes the algebraic map; its use as a PRF rests on a separate hardness assumption, namely that $L_t(K, \cdot)$ is pseudorandom for a uniformly random key K . This assumption generalises the Legendre-PRF assumption underlying Loquat and inherits that assumption’s cryptanalytic status rather than being implied by the map being well-defined.

3.2.2 Algebraic hash function.

We require an algebraic hash function whose internal computation is dominated by low-degree field operations over the same field $\mathbb{F}_p$ used by the signature scheme and the PRF. We use the Griffin family for this purpose. Following the cited literature, a Griffin instance is parameterised by a state width t_{state} and a number of rounds; each round consists of constant-addition, linear mixing by a fixed matrix, and a low-degree S-box substep using small-degree power maps: a forward degree- d power map on one lane, its inverse $x \mapsto x^{1/d}$ (well-defined since $\gcd(d, p-1) = 1$) on a second, and a Horst-type quadratic map on the remaining lanes (§5.3). Each constraint then has degree at most d while the per-round map has high algebraic degree. We write

$$H_{\text{Griffin}} : \mathbb{F}_p^* \rightarrow \mathbb{F}_p^c$$

for a Griffin-based sponge or compression hash producing a c -element digest, following the construction of [21]. The instantiation used throughout has state width $t_{\text{state}} = 4$, capacity 2 (hence rate 2 and digest length $c = 2$), S-box degree $d = 3$ (the smallest integer ≥ 3 with $\gcd(d, p-1) = 1$ over PLUM’s prime; §5.3), and 14 rounds, the value the Griffin round-count argument of [21] yields for this width-4, 199-bit instance (computed and checked in the reference implementation, §5.3). The digest length $c = 2$ over the 199-bit field is intended to provide collision resistance at the targeted security level, conditional on the security analysis of [21] applying to this width-4, $d = 3$, 14-round instance. The implemented linear layer, the circulant $\text{circ}(2, 1, 1, 1) = I + J$ inherited from Loquat’s implementation, is invertible but not MDS, unlike the standard width-4 layer of [21], so the cited analysis must additionally be re-checked against this linear-layer deviation. A second inherited deviation sits in the nonlinear layer: the published definition builds the lane-3 combination L_3 from the *input* lane x_2 , whereas the implementation, which updates lanes in place, feeds it the already-updated lane y_2 ; invertibility is preserved, but the permutation computed throughout this thesis (in both the Fp127 Loquat path and the Fp192 PLUM path, hence consistently across all measurements) is this variant rather than the published Griffin- π , and the cited security analysis must be re-checked against it as well.

When a zkVM exposes Griffin as a precompile we will write the corresponding syscall as $\text{Griffin}_{\mathbb{F}_p^{192}}$, indicating that the precompile operates natively over the same prime field used by PLUM (nominally labelled 192-bit; concretely the 199-bit prime of Remark 3.5). The cost of one Griffin permutation under this precompile is, by definition, one zkVM syscall.

3.2.3 Merkle commitment.

Definition 3.3 (Merkle commitment). Let $H : \mathbb{F}_p^c \times \mathbb{F}_p^c \rightarrow \mathbb{F}_p^c$ be a collision-resistant compression function and let $\ell = (\ell_1, \dots, \ell_n)$ with $\ell_i \in \mathbb{F}_p^c$ be a vector of leaves. The Merkle root

$$\text{rt} = \text{MerkleRoot}_H(\ell) \in \mathbb{F}_p^c$$

is defined by iterated pairwise application of H along a complete binary tree over ℓ , padding to the next power of two as needed. For index $i \in [n]$, the membership proof path_i is the sequence of sibling digests along the root path. The verification predicate

$$\text{MerkleVerify}(\text{rt}, \ell_i, \text{path}_i) \in \{0, 1\}$$

returns 1 iff recomputing the root from (ℓ_i, path_i) yields rt .

We instantiate the Merkle commitments of the BDEC system below with $H = H_{\text{Griffin}}$ over $\mathbb{F}_p$, so that each path step reduces to a small number of algebraic-hash invocations; the ledger trees so committed are checked by the relying party outside the proof, while the in-proof Griffin Merkle cost arises from the BCS commitments internal to PLUM verification. A *sparse* Merkle tree is the same structure indexed over the digest range $\{0, 1\}^\lambda$ rather than $[n]$; non-membership is shown by a membership proof of a domain-separated sentinel leaf at the queried index, and we write $\text{NonMemberVerify}(\text{rt}, x, \text{path})$ for the corresponding predicate.

3.3 Digital Signature Scheme

Definition 3.4 (Digital signature). A digital signature scheme is a tuple of probabilistic polynomial-time algorithms

$$\text{Sig} = (\text{Sig.Setup}, \text{Sig.KeyGen}, \text{Sig.Sign}, \text{Sig.Verify}).$$

On input the security parameter, the setup algorithm outputs public parameters

$$\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda).$$

The parameters pp_{Sig} determine a public-key space $\mathcal{PK}_{\text{pp}_{\text{Sig}}}$, a secret-key space $\mathcal{SK}_{\text{pp}_{\text{Sig}}}$, a message space $\mathcal{M}_{\text{pp}_{\text{Sig}}}$, and a signature space $\Sigma_{\text{pp}_{\text{Sig}}}$. The algorithms have the following syntax:

$$\begin{aligned} (\text{sk}, \text{pk}) &\leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}}), & (\text{sk}, \text{pk}) &\in \mathcal{SK}_{\text{pp}_{\text{Sig}}} \times \mathcal{PK}_{\text{pp}_{\text{Sig}}}, \\ \sigma &\leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}, m), & m &\in \mathcal{M}_{\text{pp}_{\text{Sig}}}, \sigma \in \Sigma_{\text{pp}_{\text{Sig}}}, \\ b &\leftarrow \text{Sig.Verify}(\text{pp}_{\text{Sig}}, \text{pk}, m, \sigma), & b &\in \{0, 1\}. \end{aligned}$$

When a higher-level protocol uses Sig , every value to be signed must first be represented as an element of the message space $\mathcal{M}_{\text{pp}_{\text{Sig}}}$. We write

$$\text{Encode}_{\text{Sig}} : \{0, 1\}^* \rightarrow \mathcal{M}_{\text{pp}_{\text{Sig}}}$$

for this encoding map. Thus, if a protocol value v is signed, the actual message given to Sig.Sign is

$$m = \text{Encode}_{\text{Sig}}(v) \in \mathcal{M}_{\text{pp}_{\text{Sig}}}.$$

Correctness. Correctness requires that for all λ , all $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$, all $(\text{sk}, \text{pk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$, and all messages $m \in \mathcal{M}_{\text{pp}_{\text{Sig}}}$, if $\sigma \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}, m)$, then

$$\Pr[\text{Sig.Verify}(\text{pp}_{\text{Sig}}, \text{pk}, m, \sigma) = 1] \geq 1 - \text{negl}(\lambda),$$

where the probability is over the randomness of the algorithms.

Existential Unforgeability under Chosen Message Attack. For unforgeability, define $\text{Exp}_{\text{Sig}, \mathcal{A}}^{\text{euf-cma}}(\lambda)$ as follows. The challenger samples

$$\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda), \quad (\text{sk}, \text{pk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}}),$$

and gives $(\text{pp}_{\text{Sig}}, \text{pk})$ to $\mathcal{A}$. The adversary has oracle access to

$$\mathcal{O}_{\text{Sig}}(m) : \sigma \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}, m),$$

for messages $m \in \mathcal{M}_{\text{pp}_{\text{Sig}}}$, and the challenger records every queried message in a set Q . Eventually $\mathcal{A}$ outputs (m^*, σ^*) . The experiment outputs 1 iff

$$m^* \in \mathcal{M}_{\text{pp}_{\text{Sig}}} \quad \wedge \quad m^* \notin Q \quad \wedge \quad \text{Sig.Verify}(\text{pp}_{\text{Sig}}, \text{pk}, m^*, \sigma^*) = 1.$$

The signature scheme is EUF-CMA secure if for every adversary $\mathcal{A}$,

$$\Pr[\text{Exp}_{\text{Sig}, \mathcal{A}}^{\text{euf-cma}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

Plain EUF-CMA suffices for the credential reductions of Section 6: BDEC's showing relation binds the signed message inside the proof, and the unforgeability reduction extracts a *witness* (a signature on a statement outside the issued set) rather than relying on signature uniqueness or non-malleability, so neither strong unforgeability nor resistance to signature re-randomisation is required.

Signature-Verification Predicate. Fix public parameters $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$. For $\text{pk} \in \mathcal{PK}_{\text{pp}_{\text{Sig}}}$, $m \in \mathcal{M}_{\text{pp}_{\text{Sig}}}$, and $\sigma \in \Sigma_{\text{pp}_{\text{Sig}}}$, define

$$\text{Pred}_{\text{Verify}}^{\text{Sig}, \text{pp}_{\text{Sig}}}(\text{pk}, m, \sigma) = 1 \iff \text{Sig.Verify}(\text{pp}_{\text{Sig}}, \text{pk}, m, \sigma) = 1.$$

The signature σ is computed by $\sigma \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}, m)$. The predicate does not compute σ ; it only checks whether a given σ is valid for (pk, m) . When this predicate is inserted into a zkSNARK relation, the public instance and private witness must be specified. For example,

$$R_{\text{Sig}}((\text{pk}, m); \sigma) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{pp}_{\text{Sig}}}(\text{pk}, m, \sigma) = 1,$$

where $x = (\text{pk}, m)$ is the public instance and $w = \sigma$ is the witness.

3.3.1 Loquat: a Legendre-PRF post-quantum signature with low R1CS verification cost.

Loquat is a post-quantum digital signature scheme based on the Legendre PRF and collision-resistant hash functions, designed so that its verification algorithm can be implemented by a relatively small rank-1 constraint system (R1CS) circuit (a system of bilinear constraints of the form $(Az) \circ (Bz) = Cz$ over $\mathbb{F}_p$, made precise where Aurora is introduced below). Its post-quantum claim rests on the conjectured hardness of the Legendre PRF, collision resistance of the hash, and the Fiat–Shamir/BCS (Ben-Sasson–Chiesa–Spooner) transform [35], the compiler that turns an interactive oracle proof into a non-interactive argument; it is a research scheme and is not standardised in the manner of ML-DSA or SLH-DSA.

The Loquat literature [11] describes such verification as “SNARK-friendly”. We avoid that term as a load-bearing descriptor, since it conflates several distinct cost properties, and instead refer directly to the structural quantity that matters here: a low constraint count for the verification predicate when encoded as an R1CS.

Formally, a Loquat instance is a tuple of probabilistic polynomial-time algorithms

$$\text{Loquat} = (\text{Loquat.Setup}, \text{Loquat.KeyGen}, \text{Loquat.Sign}, \text{Loquat.Verify}).$$

On input the security parameter, the setup algorithm outputs public parameters

$$\text{pp}_{\text{Loquat}} \leftarrow \text{Loquat.Setup}(1^\lambda).$$

These parameters fix, among other things, a prime field $\mathbb{F}_p$, a public list $I = (I_1, \dots, I_L) \in \mathbb{F}_p^L$ for some integer L , and auxiliary parameters for the underlying Legendre-PRF key-identification protocol and its compilation to a non-interactive proof system.

The key-generation algorithm samples a secret Legendre-PRF key and its corresponding public key as a vector of PRF evaluations:

$$\begin{aligned} \text{sk} &= K \leftarrow \text{Loquat.KeyGen}(\text{pp}_{\text{Loquat}}), \\ \text{pk} &= (L_K(I_1), \dots, L_K(I_L)) \in \{0, 1\}^L, \end{aligned}$$

where $L_K(\cdot)$ denotes the keyed Legendre pseudorandom function and each $L_K(I_\ell) \in \{0, 1\}$ is the Legendre-symbol output bit on input $K + I_\ell$.

The message space $\mathcal{M}_{\text{Loquat}}$ and signature space Σ_{Loquat} are those induced by the non-interactive key-identification protocol obtained from the underlying interactive-oracle proof via the Fiat–Shamir/BCS transformation. On input $(\text{pp}_{\text{Loquat}}, \text{sk}, m)$ with $m \in \mathcal{M}_{\text{Loquat}}$, the signing algorithm runs this protocol using the secret key K as the witness and outputs a signature

$$\sigma \leftarrow \text{Loquat.Sign}(\text{pp}_{\text{Loquat}}, \text{sk}, m) \in \Sigma_{\text{Loquat}}.$$

The verification algorithm is deterministic and checks the hash chain, Merkle-tree commitments, univariate-sumcheck proof, low-degree test, and quadratic-residuosity conditions implied by the Legendre-PRF relation:

$$b \leftarrow \text{Loquat.Verify}(\text{pp}_{\text{Loquat}}, \text{pk}, m, \sigma) \in \{0, 1\}.$$

Loquat-Verification Predicate. Fix $\text{pp}_{\text{Loquat}} \leftarrow \text{Loquat.Setup}(1^\lambda)$. For $\text{pk} \in \mathcal{PK}_{\text{Loquat}}$, $m \in \mathcal{M}_{\text{Loquat}}$, and $\sigma \in \Sigma_{\text{Loquat}}$, define

$$\text{Pred}_{\text{Verify}}^{\text{Loquat}, \text{pp}_{\text{Loquat}}}(\text{pk}, m, \sigma) = 1 \iff \text{Loquat.Verify}(\text{pp}_{\text{Loquat}}, \text{pk}, m, \sigma) = 1.$$

When this predicate is encoded as a zkSNARK relation, we take, for example,

$$R_{\text{Loquat}}((\text{pk}, m); \sigma) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Loquat}, \text{pp}_{\text{Loquat}}}(\text{pk}, m, \sigma) = 1,$$

where $x = (\text{pk}, m)$ is the public instance and $w = \sigma$ is the witness.

3.3.2 PLUM: a power-residue-PRF post-quantum signature with low R1CS verification cost.

PLUM is a post-quantum digital signature scheme that follows the same overall structure as Loquat but replaces the Legendre PRF and FRI-based (Fast Reed–Solomon Interactive Oracle Proof of Proximity [36]) low-degree test with a t -th power residue PRF and the STIR proximity test [37] (Reed–Solomon proximity testing with fewer queries than FRI), and selects a prime field tailored to these primitives. These substitutions reduce the polynomial degrees and query complexity in the underlying proof system, leading to smaller signatures and fewer R1CS constraints in the verification circuit while, per the PLUM authors’ analysis, retaining EUF-CMA security at the claimed level (subject to the prime-instantiation caveat of Remark 3.5).

Formally, a PLUM instance is a tuple of probabilistic polynomial-time algorithms

$$\text{Plum} = (\text{Plum.Setup}, \text{Plum.KeyGen}, \text{Plum.Sign}, \text{Plum.Verify}).$$

On input the security parameter, the setup algorithm outputs public parameters

$$\text{pp}_{\text{Plum}} \leftarrow \text{Plum.Setup}(1^\lambda).$$

The setup chooses an odd prime p and an integer $t \geq 2$ such that $t \mid (p - 1)$, fixes a generator $g \in \mathbb{F}_p^*$, and defines the t -th power residue PRF L_t as in Definition 3.2. It then samples a public list $I = (I_1, \dots, I_L) \in \mathbb{F}_p^L$ and sets the parameters for the univariate-sumcheck and STIR low-degree test (including the code domain U , folding parameter, degree bounds, and query-repetition parameters) as in Algorithm 1 of [18].

Remark 3.5 (Prime instantiation). The implementation evaluated in this thesis fixes a concrete 199-bit prime $p = 2^{64} p_0 + 1$ with p_0 a 135-bit prime,² chosen so that $t = 256 \mid (p - 1)$ and the 2-adicity of $p - 1$ is at least 64, as the STIR low-degree test requires. The value of p_0 used is *not* the decimal printed in the published parameter table [18, §3.3]: that printed value is composite (its smallest prime factor is 97, witnessed by a primality test in the artefact), almost certainly a typographical error in the manuscript, and is therefore unusable as a field modulus. We substitute a nearby 199-bit value satisfying all the structural requirements above. The substitution is benign for the measurements of this thesis, whose cost is a function of the field *bit-width* and smoothness rather than of the exact decimal value; it is recorded explicitly because any concrete security-bit claim that depends on the specific value of p is conditional on this instantiation, and a paper-faithful or interoperable deployment should confirm the intended modulus with the PLUM authors. The security analysis (Section 6) carries this conditionality as an explicit hypothesis.

The key-generation algorithm samples a secret PRF key and derives the public key as a vector of t -th power residue PRF evaluations:

$$\begin{aligned} \text{sk} &= K \leftarrow \text{Plum.KeyGen}(\text{pp}_{\text{Plum}}), \\ \text{pk} &= (L_t(K, I_1), \dots, L_t(K, I_L)) \in (\mathbb{Z}/t\mathbb{Z})^L, \end{aligned}$$

where each $L_t(K, I_\ell) \in \mathbb{Z}/t\mathbb{Z}$ is a symbol of the t -th power residue PRF on input $K + I_\ell$ (for $t = 2$ this degenerates to the Loquat binary Legendre symbol). The public key thus occupies $L \lceil \log_2 t \rceil$ bits.

²The implementation prime is 199-bit (with an approximately 135-bit p_0). The “192-bit” / “ $\mathbb{F}_p^{192}$ ” naming used elsewhere in this thesis for the PLUM field and the Griffin precompile is a loose label rather than the exact bit-width; PLUM’s own source is internally inconsistent on this point, the prose and the printed p_0 disagreeing.

Compared to Loquat, taking $t > 2$ increases the entropy per symbol and allows PLUM to use fewer public-key symbols L and fewer challenged symbols B at a given security level.

The message space $\mathcal{M}_{\text{Plum}}$ and signature space Σ_{Plum} are induced by the non-interactive key-identification protocol obtained from the PLUM-specific interactive-oracle proof via the Fiat–Shamir/BCS transformation, with FRI replaced by STIR in the low-degree test component. On input $(\text{pp}_{\text{Plum}}, \text{sk}, m)$ with $m \in \mathcal{M}_{\text{Plum}}$, the signing algorithm runs this protocol using the secret key K as the witness and outputs a signature

$$\sigma \leftarrow \text{Plum.Sign}(\text{pp}_{\text{Plum}}, \text{sk}, m) \in \Sigma_{\text{Plum}}.$$

The verification algorithm is deterministic and checks the hash chain, Merkle-tree commitments, univariate-sumcheck proof, STIR low-degree test, and t -th-power-residue conditions implied by the PLUM PRF relation:

$$b \leftarrow \text{Plum.Verify}(\text{pp}_{\text{Plum}}, \text{pk}, m, \sigma) \in \{0, 1\}.$$

For the 128-bit parameter set, PLUM uses $t = 256$, a public-key length of $L = 2^{12}$ PRF symbols, and $B = 28$ challenged symbols, and verifying a single signature with Griffin as the hash instantiation requires about 1.16×10^5 RICS constraints, versus $\approx 1.48 \times 10^5$ for Loquat under comparable assumptions [18, §4.2]. The corresponding signature size is about 37 KB (versus 57 KB for Loquat); the signing and verification runtimes are *estimated*, from the Loquat implementation rather than a from-scratch PLUM measurement [18, Table 2], to be up to a factor 4 faster than Loquat at the same security level.

PLUM-Verification Predicate. Fix $\text{pp}_{\text{Plum}} \leftarrow \text{Plum.Setup}(1^\lambda)$. For $\text{pk} \in \mathcal{PK}_{\text{Plum}}$, $m \in \mathcal{M}_{\text{Plum}}$, and $\sigma \in \Sigma_{\text{Plum}}$, define

$$\text{Pred}_{\text{Verify}}^{\text{Plum}, \text{pp}_{\text{Plum}}}(\text{pk}, m, \sigma) = 1 \iff \text{Plum.Verify}(\text{pp}_{\text{Plum}}, \text{pk}, m, \sigma) = 1.$$

When this predicate is encoded as a zkSNARK relation, we can take, for example,

$$R_{\text{Plum}}((\text{pk}, m); \sigma) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Plum}, \text{pp}_{\text{Plum}}}(\text{pk}, m, \sigma) = 1,$$

where $x = (\text{pk}, m)$ is the public instance and $w = \sigma$ is the witness.

Quantitative Comparison to Loquat. At the same 128-bit security level and with both schemes instantiated using Griffin as the algebraic hash, PLUM reduces signature size from 57 KB to 37 KB, reduces the verification circuit from about 148,825 (Loquat [11]) to 116,285 RICS constraints, and is expected to decrease signing and verification time by up to a factor of four, according to the analysis and parameter choices of [18]. These improvements come from (i) using a t -th power residue PRF to halve the polynomial degrees in the sumcheck relation, (ii) replacing FRI with the lower-query-complexity STIR protocol, and (iii) choosing a prime p that simultaneously supports the PRF and the smooth multiplicative subgroups required by STIR, avoiding the field-lifting overhead present in Loquat. These are *static RICS* advantages; Section 7 shows they do not translate cleanly into zkVM proving cost, where the field-mismatch tax on Griffin over a non-native prover field dominates.

3.4 zkSNARK

Let R be an efficiently decidable relation and let

$$L_R = \{x : \exists w \text{ such that } R(x; w) = 1\}$$

be the corresponding language. A zkSNARK for R is a tuple

$$\Pi = (\Pi.\text{Setup}, \Pi.\text{Prove}, \Pi.\text{Verify}).$$

Two orthogonal axes matter here: *transparency* (whether $\Pi.\text{Setup}$ needs a relation-specific trusted setup) and *universality* (whether the proving key/preprocessing is independent of the concrete relation R). Update churn is driven by the latter: a transparent but relation-specific argument such as Aurora still re-derives relation-dependent parameters on every change. We first state the universal/relation-independent case, in which the setup is independent of the concrete relation:

$$\text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda).$$

If a relation-specific proof system is used instead, setup should be written as $\text{crs}_R \leftarrow \Pi.\text{Setup}(1^\lambda, R)$. The proving and verification algorithms are

$$\pi \leftarrow \Pi.\text{Prove}(\text{crs}, x, w), \quad b \leftarrow \Pi.\text{Verify}(\text{crs}, x, \pi) \in \{0, 1\}.$$

Completeness. Completeness requires that for every (x, w) such that $R(x; w) = 1$,

$$\Pr[\Pi.\text{Verify}(\text{crs}, x, \Pi.\text{Prove}(\text{crs}, x, w)) = 1] \geq 1 - \text{negl}(\lambda).$$

Soundness. Soundness requires that for every adversary $\mathcal{A}$,

$$\Pr[x \notin L_R \wedge \Pi.\text{Verify}(\text{crs}, x, \pi) = 1 : \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda), (x, \pi) \leftarrow \mathcal{A}(\text{crs})] \leq \text{negl}(\lambda).$$

Knowledge soundness (argument of knowledge). A property strictly stronger than soundness, and the one a higher-level reduction needs when it must *extract* a witness rather than merely conclude that one exists. The argument is *knowledge-sound* if there is a probabilistic polynomial-time extractor $\mathcal{E}$ such that, for every prover $\mathcal{A}$ producing an accepting proof for a statement x with non-negligible probability, $\mathcal{E}$, given $\mathcal{A}$'s code and randomness (or rewinding/oracle access), outputs a witness w with $R(x; w) = 1$, except with negligible *knowledge error*. BDEC's unforgeability reduction relies on this property of Π to pull a credential and its signatures out of an accepting showing; plain soundness, which only certifies that a witness exists, does not suffice.

Zero-knowledge. Zero-knowledge requires that there exists a simulator Sim such that, for every x and w satisfying $R(x; w) = 1$, the distributions

$$\{(\text{crs}, x, \pi) : \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda), \pi \leftarrow \Pi.\text{Prove}(\text{crs}, x, w)\}$$

and

$$\{(\text{crs}, x, \tilde{\pi}) : \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda), \tilde{\pi} \leftarrow \text{Sim}(\text{crs}, x)\}$$

are computationally indistinguishable. Unless stated otherwise this is computational zero-knowledge with a simulator in the (classical) random-oracle model; whether a given instantiation's simulator extends to the QROM is flagged where the distinction affects the receipt-mode claims of Section 5.

3.4.1 Aurora: Transparent Succinct Arguments for R1CS

Aurora [14] is a transparent zkSNARK for the rank-1 constraint system (R1CS) relation. Given an R1CS instance (A, B, C) over a finite field $\mathbb{F}$ and a candidate assignment $z \in \mathbb{F}^n$ satisfying the Hadamard identity

$$(Az) \odot (Bz) = Cz,$$

Aurora produces a succinct, publicly verifiable, zero-knowledge proof of the existence of a satisfying witness, with polylogarithmic proof size and linear verifier complexity in n . Aurora is transparent in the sense that it requires no relation-specific trusted setup beyond a collision-resistant hash function. The Aurora reference in this thesis is measured in both modes: with zero-knowledge disabled (`zk=false`), where the measurement bounds proving cost only and says nothing about anonymity, and with zero-knowledge enabled, where the same Loquat single-verification circuit completes in about 22 minutes (mean of three runs, 21.4–22.9) (Section 7). All runs achieve 107.5-bit security.

When instantiated for a fixed relation R , the R1CS encoding fixes the matrices (A, B, C) at compile time; any change to R that affects which gates appear in the verification predicate requires recompilation of these matrices and re-derivation of any parameters that depend on the constraint count. This is the property that motivates the zkVM approach: the underlying relation is fixed to the ISA semantics once, and program changes are expressed inside the witness rather than in the matrices.

The BDEC construction of [1] instantiates its zkSNARK Π with Aurora over the R1CS encoding of the credential-issuance and credential-showing relations defined later in this section. The dominant component of proof generation in this regime is Aurora’s low-degree-test (LDT) sub-protocol; verification is instead dominated by the verifier’s linear pass over the explicit R1CS matrices [14, Thm. 1.2].

3.5 Anonymous Credential

Definition 3.6 (Anonymous credential). Let λ be a security parameter, let Attr be an attribute space, and let Φ be a class of predicates over attribute vectors. An anonymous credential scheme is a tuple of algorithms or interactive protocols

$$\text{AC} = (\text{AC.Setup}, \text{AC.UserKeyGen}, \text{AC.IssuerKeyGen}, \text{AC.Issue}, \text{AC.Show}, \text{AC.Verify}).$$

For an attribute vector $A = (a_1, \dots, a_m) \in \text{Attr}^m$, issuance gives a user a credential cred . For a predicate $\phi \in \Phi$, the showing algorithm outputs a public statement and proof

$$(\text{stmt}, \pi) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}, \text{usk}, \text{ipk}, A, \phi).$$

The statement contains intentionally disclosed information. The proof proves membership in a relation

$$R_{\text{AC}}(\text{stmt}; w) = 1,$$

where the witness contains at least $w = (\text{cred}, \text{usk}, A)$ and the relation checks that cred is valid and that $\phi(A) = 1$.

Correctness. Correctness requires that honestly generated credentials and honestly generated showing proofs are accepted by the verifier. More formally, for every λ , every

$$\text{pp} \leftarrow \text{AC.Setup}(1^\lambda),$$

every honestly generated user key pair and issuer key pair

$$(\text{usk}, \text{upk}) \leftarrow \text{AC.UserKeyGen}(\text{pp}), \quad (\text{isk}, \text{ipk}) \leftarrow \text{AC.IssuerKeyGen}(\text{pp}),$$

every honestly issued credential

$$\text{cred} \leftarrow \langle \text{User}(\text{pp}, \text{usk}, \text{upk}, A), \text{Issuer}(\text{pp}, \text{isk}, \text{ipk}, \text{upk}, A) \rangle.$$

and every predicate $\phi \in \Phi$, if $\phi(A) = 1$, then

$$\Pr[\text{AC.Verify}(\text{pp}, \text{ipk}, \text{stmt}, \pi) = 1] \geq 1 - \text{negl}(\lambda),$$

where

$$(\text{stmt}, \pi) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}, \text{usk}, \text{ipk}, A, \phi).$$

The probability is taken over the randomness of the setup, key-generation, issuance, showing, and verification algorithms.

Unforgeability. Unforgeability requires that no adversary can produce an accepting showing proof unless it knows a credential honestly issued by an issuer and a corresponding user secret key satisfying the statement. Let $\mathcal{L}$ be the issuance log containing all tuples

$$(\text{ipk}, \text{upk}, A, \text{cred})$$

honestly issued during the experiment. The adversary may interact with issuing oracles and then outputs $(\text{ipk}^*, \text{stmt}^*, \pi^*)$. The adversary wins if

$$\text{AC.Verify}(\text{pp}, \text{ipk}^*, \text{stmt}^*, \pi^*) = 1$$

and there is no tuple

$$(\text{ipk}^*, \text{upk}^*, A^*, \text{cred}^*) \in \mathcal{L}$$

and no user secret key usk^* such that $(\text{usk}^*, \text{upk}^*)$ is an honestly generated user key pair such that, for

$$w^* = (\text{cred}^*, \text{usk}^*, A^*),$$

we have

$$R_{\text{AC}}(\text{stmt}^*; w^*) = 1.$$

We write $\text{Adv}_{\text{AC}}^{\text{unf}}(\mathcal{A})$ for the probability that $\mathcal{A}$ wins this experiment; the scheme is unforgeable if $\text{Adv}_{\text{AC}}^{\text{unf}}(\mathcal{A}) \leq \text{negl}(\lambda)$ for every probabilistic polynomial-time $\mathcal{A}$.

Anonymity. Anonymity requires that a showing proof does not reveal which honest user generated it, beyond the information intentionally disclosed in the public statement. This is a cryptographic indistinguishability notion: non-cryptographic side channels, such as wall-clock timing, network and ledger-access patterns, revocation-update timing, and issuer or wallet metadata observed out of band, are outside the model and are not covered by the bound below. In the anonymity experiment, the adversary chooses two honestly issued credentials

$$(\text{cred}_0, \text{usk}_0, A_0), \quad (\text{cred}_1, \text{usk}_1, A_1),$$

and a predicate ϕ , such that both credentials satisfy the predicate,

$$\phi(A_0) = \phi(A_1) = 1,$$

and the corresponding public statements reveal the same allowed leakage. More formally, if

$$(\text{stmt}_0, \pi_0) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}_0, \text{usk}_0, \text{ipk}, A_0, \phi)$$

and

$$(\text{stmt}_1, \pi_1) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}_1, \text{usk}_1, \text{ipk}, A_1, \phi),$$

then the challenge is well formed only if

$$\text{Leak}(\text{stmt}_0) = \text{Leak}(\text{stmt}_1),$$

where $\text{Leak}(\text{stmt})$ is the projection of the public statement onto its intentionally-disclosed components, namely the revealed attribute set $A^\downarrow$, the teaching-authority pseudonym multiset $\{\text{ppk}_{U,TA}^{(j)}\}$, and any linking tag carried in the predicate ϕ ; the verifier-facing pseudonym $\text{ppk}_{U,V}$ is freshly sampled at showing time, and the remaining statement components are witness-derived and made independent of the challenge bit by the simulators of the anonymity argument (Appendix A). The challenger samples $b \leftarrow \{0, 1\}$ and returns

$$(\text{stmt}, \pi_b) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}_b, \text{usk}_b, \text{ipk}, A_b, \phi).$$

The adversary outputs b' . We write $\text{Adv}_{\text{AC}}^{\text{anon}}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}|$; the scheme is anonymous if $\text{Adv}_{\text{AC}}^{\text{anon}}(\mathcal{A}) \leq \text{negl}(\lambda)$ for every probabilistic polynomial-time $\mathcal{A}$.

Unlinkability. Unlinkability requires that a shown credential cannot be linked to the pseudonym under which it was issued, unless the public statements intentionally contain a linking tag. In the unlinkability experiment, with the oracle interface of the unforgeability experiment, the challenger creates two pseudonym key pairs $(\text{psk}^0, \text{ppk}^0)$ and $(\text{psk}^1, \text{ppk}^1)$ and an attribute set $A^\downarrow$, samples $b \leftarrow \{0, 1\}$, and issues a single credential $c^{(b)}$ with respect to $\text{ppk}^{(b)}$ on $A^\downarrow$, forwarding it to the adversary. The adversary outputs b' . We write $\text{Adv}_{\text{AC}}^{\text{unlink}}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}|$, and the scheme is unlinkable if it is $\text{negl}(\lambda)$ for every probabilistic polynomial-time $\mathcal{A}$. When the statements intentionally contain a public linking tag, linkability is allowed only as determined by that tag.

3.5.1 Blockchain-based Digital Education Credentials (BDEC)

BDEC is a blockchain-based anonymous credential system for educational credentials. It is parameterized by a digital signature scheme Sig and a zkSNARK Π . The main parties are a user U , teaching authorities TA , and a verifier V .

The blockchain stores credential-record digests, authorized teaching-authority keys, and revocation state. We assume that H and H_{rev} are collision-resistant and domain-separated hash functions. In the privacy-preserving case, the blockchain stores record digests ρ rather than raw credential attributes.

Construction 1. The BDEC construction uses Sig through Sig.Setup , Sig.KeyGen , Sig.Sign , and Sig.Verify . It uses Π to prove relations containing signature-verification predicates; the ledger-membership,

authorized-TA, and revocation checks are performed by the verifier outside the proof, against the published roots (see BDEC.ShowVer below). The BDEC algorithms instantiate the minimal anonymous-credential syntax introduced above as follows: $\text{BDEC.Setup} \leftrightarrow \text{AC.Setup}$; $\text{BDEC.PriGen} \leftrightarrow \text{AC.UserKeyGen}$; $\text{BDEC.TAKeyGen} \leftrightarrow \text{AC.IssuerKeyGen}$; issuance BDEC.IssueCre with credential-proof generation BDEC.CreGen together realising AC.Issue ; $\text{BDEC.ShowCre} \leftrightarrow \text{AC.Show}$; and $\text{BDEC.ShowVer} \leftrightarrow \text{AC.Verify}$. The pseudonym-key, credential-verification, and revocation steps (BDEC.NymKey , BDEC.CreVer , BDEC.RevCre) are BDEC-specific refinements with no counterpart in the minimal syntax.

BDEC.Setup. On input 1^λ , compute

$$\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda), \quad \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda).$$

Output the static parameters $\text{par} = (\text{pp}_{\text{Sig}}, \text{crs})$. At epoch e (the blockchain epoch index, distinct from the power-residue PRF modulus $t = 256$), the public blockchain state is

$$\text{st}_e = (\text{rt}_{\text{cred},e}, \text{rt}_{\text{TA},e}, \text{rt}_{\text{LR},e}),$$

where $\text{rt}_{\text{cred},e}$ is the accepted credential-record root, $\text{rt}_{\text{TA},e}$ is the authorized teaching-authority registry root, and $\text{rt}_{\text{LR},e}$ is the revocation-state root.

BDEC.PriGen. On input par , compute the user's key pair

$$(\text{sk}_U, \text{pk}_U) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$$

and output $(\text{sk}_U, \text{pk}_U)$.

BDEC.TAKeyGen. On input par , a teaching authority computes

$$(\text{sk}_{\text{TA}}, \text{pk}_{\text{TA}}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$$

and publishes pk_{TA} . The public key is accepted only if it is authenticated externally or included in the authorized-TA registry represented by $\text{rt}_{\text{TA},e}$.

BDEC.NymKey. On input $(\text{par}, \text{sk}_U)$, sample a pseudonym $\text{ppk}_{U,\text{TA}} \leftarrow \{0, 1\}^\lambda$ and compute

$$m_{\text{nym},U,\text{TA}} = \text{Encode}_{\text{Sig}}(\text{ppk}_{U,\text{TA}}) \in \mathcal{M}_{\text{pp}_{\text{Sig}}}, \quad \text{psk}_{U,\text{TA}} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_U, m_{\text{nym},U,\text{TA}}).$$

Output $(\text{ppk}_{U,\text{TA}}, \text{psk}_{U,\text{TA}})$. This user-side signature binds the pseudonym to the user's key; it does not certify educational attributes.

BDEC.IssueCre. On input $(\text{par}, \text{sk}_{\text{TA}}, \text{pk}_U, \text{ppk}_{U,\text{TA}}, A, \text{aux})$, the teaching authority first checks externally that the user is entitled to the attributes A . If the check accepts, define

$$m_{\text{cred},U,\text{TA}} = \text{Encode}_{\text{Sig}}(\text{pk}_U, \text{ppk}_{U,\text{TA}}, A, \text{aux}) \in \mathcal{M}_{\text{pp}_{\text{Sig}}}$$

and compute

$$\sigma_{\text{cred},U,\text{TA}} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_{\text{TA}}, m_{\text{cred},U,\text{TA}}).$$

The corresponding blockchain record digest is

$$\rho_{U,\text{TA}} = H(\text{pk}_{\text{TA}}, \text{ppk}_{U,\text{TA}}, A, \text{aux}, \sigma_{\text{cred},U,\text{TA}}).$$

The blockchain stores $\rho_{U,\text{TA}}$ in the accepted credential-record ledger. It need not store A , aux , or $\sigma_{\text{cred},U,\text{TA}}$ in the clear.

BDEC.CreGen. On input $(\text{par}, \text{sk}_U, \text{pk}_U, \text{ppk}_{U,TA}, \text{psk}_{U,TA}, A, \text{aux})$, the algorithm first computes the attribute digest $h_{U,TA} = H(A)$ and the credential $c_{U,TA} \leftarrow \text{Sig.Sign}(\text{ppSig}, \text{sk}_U, h_{U,TA})$. It then generates a proof with respect to the zkSNARK statement and witness; in a zkVM instantiation the public-instance components x_{cre} below must be committed to the receipt journal for the proof to bind to them; this is a statement-binding requirement on which soundness depends, and its status in the prototype is treated in Section 5,

$$\begin{aligned} x_{\text{cre}} &= (c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA}), \\ w_{\text{cre}} &= (\text{pk}_U, \text{psk}_{U,TA}), \end{aligned}$$

where the proved relation is the conjunction of two signature verifications, both under the user's long-term public key pk_U :

$$\begin{aligned} R_{\text{cre}}^{\text{Sig}}(x_{\text{cre}}; w_{\text{cre}}) = 1 &\iff \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}_U, h_{U,TA}, c_{U,TA}) = 1 \\ &\wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}_U, \text{ppk}_{U,TA}, \text{psk}_{U,TA}) = 1. \end{aligned}$$

Equivalently, the circuit proved by the zkSNARK is the AND concatenation of two signature-verification algorithms [1]: the first attests that $c_{U,TA}$ is a valid credential on the attribute digest, the second that the user controls the pseudonym $\text{ppk}_{U,TA}$. Compute

$$\varepsilon_{c_{U,TA}} \leftarrow \Pi.\text{Prove}(\text{crs}, x_{\text{cre}}, w_{\text{cre}})$$

and output $(c_{U,TA}, \varepsilon_{c_{U,TA}})$.

Remark 3.7 (Optional Merkle accumulator; not part of the base relation). The base $R_{\text{cre}}^{\text{Sig}}$ above is exactly the two-verification relation of [1], for which that work's unforgeability and anonymity theorems are proved. To additionally *hide* attributes at presentation time, the same work suggests an optional Merkle-accumulator variant: the user signs the Merkle *root* of the attributes in BDEC.CreGen rather than $H(A)$, and the membership (authentication-path) checks are performed by the verifier in BDEC.ShowVer, not inside $R_{\text{cre}}^{\text{Sig}}$. This thesis implements and measures the base two-verification relation; the accumulator variant, and any consequent re-derivation of the security reductions for an enlarged relation, are out of scope and would constitute a strengthening beyond the cited construction rather than a re-instantiation of it.

BDEC.CreVer. On input $(\text{par}, \text{st}_e, x_{\text{cre}}, \varepsilon_{c_{U,TA}})$, output

$$b \leftarrow \Pi.\text{Verify}(\text{crs}, x_{\text{cre}}, \varepsilon_{c_{U,TA}}) \in \{0, 1\}.$$

BDEC.ShowCre. On input $(\text{par}, \text{st}_e, \text{pk}_U, (\text{ppk}_{U,TA}^{(j)}, \text{psk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V}, \text{psk}_{U,V}, c_{U,V}, A^\downarrow, \phi)$, where $A^\downarrow$ are the selectively disclosed attributes, ϕ the claimed presentation predicate, and k the number of accepted credential records used, the algorithm forms the public instance

$$x_{\text{show}} = (A^\downarrow, (\text{ppk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V}).$$

The private witness is

$$w_{\text{show}} = (\text{pk}_U, (\text{psk}_{U,TA}^{(j)})_{j=1}^k, \text{psk}_{U,V}, c_{U,V}).$$

Following the base relation of [1], the witness is the user’s long-term verification key pk_U , used here as a *private* witness despite being a public key, together with the pseudonym secret keys and the shown credential. Hiding pk_U is what delivers anonymity, since the showing is then not tied to the user’s identity. The pseudonym public keys and the disclosed attributes $A^\downarrow$ are public, so the messages verified inside the proof are formed from public values and the verifier evaluates ϕ on $A^\downarrow$. Membership, non-revocation, and the disclosure check are performed by the verifier in BDEC.ShowVer , which additionally takes the disclosed predicate ϕ , the published roots $\text{rt}_{\text{cred},e}$, $\text{rt}_{TA,e}$, $\text{rt}_{LR,e}$, the credential commitments $(\rho^{(j)})_{j=1}^k$, the membership paths, and the revocation tag r_U . These are verifier-side inputs and are not part of the proof statement or witness. Concealing the attributes outside $A^\downarrow$ is the optional Merkle-accumulator strengthening of Remark 3.7, which this thesis does not implement.

Concrete presentation predicates. The predicate class Φ is the set of Boolean combinations of arithmetic comparisons and equalities over attribute values. Two predicates serve as running examples throughout the thesis and as the workload for the update-churn measurement of Section 7. The first, used at credential count $k = 1$, is a single-attribute threshold; writing $A.\text{gpa}$ for the grade-point attribute,

$$\phi_1(A) = [A.\text{gpa} > 3.5].$$

The second, used at $k = 2$, is a conjunction over two credentials drawn from possibly distinct issuers; writing $A.\text{degree}$, $A.\text{gradYear}$, and $A.\text{major}$ for the relevant attributes,

$$\phi_2(A^{(1)}, A^{(2)}) = [A^{(1)}.\text{degree} = \text{Bachelor}] \wedge [A^{(1)}.\text{gradYear} > 2020] \wedge [A^{(2)}.\text{major} = \text{CS}].$$

Both predicates are evaluated by the verifier on the disclosed attribute set $A^\downarrow$, *outside* the proof relation $R_{\text{show}}^{\text{Sig}}$: following base BDEC (Remark 3.7), the attributes named in ϕ are disclosed in the clear and the verifier checks $\phi(A^\downarrow) = 1$ on the revealed values. For ϕ_1 the GPA is disclosed and the verifier checks the threshold; for ϕ_2 the degree, graduation year, and major are disclosed and the verifier checks the conjunction. The privacy the showing provides is *anonymity*, in that the proof is not linked to the user’s identity pk_U , together with unlinkability across showings; it is *not* concealment of the disclosed attribute *values*. Proving a predicate over an attribute that stays hidden (e.g. “gpa > 3.5” without revealing the GPA) is beyond the base relation and would require the optional Merkle-accumulator strengthening that Remark 3.7 places out of scope. The two predicates differ in arity ($k = 1$ versus $k = 2$), in issuer multiplicity (one issuer versus two), and in Boolean structure (atomic versus a three-clause conjunction). Because ϕ_1 and ϕ_2 require a *different number of credentials*, moving between them changes the presentation *structure*, namely the credential count k , and hence the relation $R_{\text{show}}^{\text{Sig}}$ itself, which is the source of the static-circuit recompilation cost measured in Section 7; a policy change at fixed structure (e.g. retuning the GPA threshold) is absorbed entirely by the verifier-side check and forces no recompilation. For the threat model of Section 6, the timing of ϕ matters: it is chosen by the verifier at presentation time and is not known when the credential is issued.

For each $j \in [k]$, define $m_{\text{nym}}^{(j)} = \text{Encode}_{\text{Sig}}(\text{ppk}_{U,TA}^{(j)})$. Also define $m_{\text{nym},U,V} = \text{Encode}_{\text{Sig}}(\text{ppk}_{U,V})$ and $m_{\text{show}} = \text{Encode}_{\text{Sig}}(A^\downarrow)$, the encoding of the disclosed attributes carried by the shown credential

$c_{U,V}$. The showing relation is

$$\begin{aligned} R_{\text{show}}^{\text{Sig}}(x_{\text{show}}; w_{\text{show}}) = 1 &\iff \bigwedge_{j=1}^k \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}_U, m_{\text{nym}}^{(j)}, \text{psk}_{U,TA}^{(j)}) = 1 \\ &\quad \wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}_U, m_{\text{nym},U,V}, \text{psk}_{U,V}) = 1 \\ &\quad \wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}_U, m_{\text{show}}, c_{U,V}) = 1. \end{aligned}$$

Following BDEC [1], the proof circuit contains $k + 2$ signature-verification predicates: k pseudonym-ownership checks (each binding a teaching-authority pseudonym to pk_U), one verifier-facing pseudonym check, and one check on the shown credential $c_{U,V}$ over the disclosed attributes. The relying party additionally checks, *outside* the proof: accepted-ledger membership and authorised-TA membership against the published roots, non-revocation against the revocation tree, and that the disclosed subset $A^\downarrow$ satisfies the presentation predicate ϕ . Selective disclosure is realised by the chosen subset $A^\downarrow$; no in-circuit predicate enforces it. Compute

$$\varepsilon_{c_{U,V}} \leftarrow \Pi.\text{Prove}(\text{crs}, x_{\text{show}}, w_{\text{show}})$$

and output $\varepsilon_{c_{U,V}}$.

BDEC.ShowVer. On input $(\text{par}, \text{st}_e, x_{\text{show}}, \varepsilon_{c_{U,V}})$, output

$$b \leftarrow \Pi.\text{Verify}(\text{crs}, x_{\text{show}}, \varepsilon_{c_{U,V}}) \in \{0, 1\}.$$

The verifier first checks the zkSNARK proof against the public statement, which attests the $k + 2$ signature-verification predicates of $R_{\text{show}}^{\text{Sig}}$. The relying party then performs the ledger-membership, authorised-TA, non-revocation, and selective-disclosure checks *outside* the proof, against the published roots, the revocation tree, and the disclosed subset $A^\downarrow$, consistent with the relation above.

BDEC.RevCre. On input $(\text{par}, \text{pk}_U)$, compute

$$r_U = H_{\text{rev}}(\text{pk}_U)$$

and update the revocation state so that r_U is included in the revocation list represented by $\text{rt}_{\text{LR},e}$. Future showings are accepted only if the prover supplies a valid non-membership witness for r_U with respect to the current revocation root.

Conditional linkability. Conditional linkability allows controlled linking only when the user intentionally discloses linking information. Let τ be an optional public linking tag contained in the statement. There is a deterministic algorithm Link such that

$$\text{Link}(\text{stmt}_0, \text{stmt}_1) = 1 \iff \tau_0 = \tau_1 \neq \perp.$$

If no tag is disclosed, then linkability reduces to the unlinkability requirement above. The tag τ is chosen by the *user* at showing time, not by the verifier; we assume an honest user does not reuse a tag across contexts they wish to keep unlinkable, and we treat verifier-induced linkability (a verifier that refuses service unless the user supplies a fixed τ) as an out-of-scope deployment risk rather than a property of the scheme.

Revocability. Let LR be a revocation state represented by the sparse-Merkle revocation root $rt_{LR,e}$ (Definition 3.3). A showing is revocation-valid only if

$$r_U = H_{\text{rev}}(\text{pk}_U) \quad \wedge \quad \text{NonMemberVerify}(rt_{LR,e}, r_U, \text{path}_{LR}) = 1.$$

If r_U is added to LR, then any future showing transcript linked to r_U is rejected except with negligible probability. For honest, unrevoked users, correctness continues to hold.

3.6 Zero-Knowledge Virtual Machines

A zero-knowledge virtual machine (zkVM) is a zkSNARK whose relation is fixed once, to the correctness of executing a guest program on a deterministic instruction-set architecture. In contrast to a relation-specific zkSNARK such as Aurora, where the rank-1 constraint system encoding a relation R is compiled together with the proof system, a zkVM commits to a single universal relation R_{vm} describing its instruction-set semantics, and treats the guest program as an input to that relation. Background surveys and step-by-step expositions of this paradigm are given in [15, 19].

Definition 3.8 (Zero-knowledge virtual machine). Let ISA be a deterministic instruction-set architecture with a fixed cycle-level semantics. A zkVM for ISA is a zkSNARK

$$\Pi_{\text{vm}} = (\Pi_{\text{vm}}.\text{Setup}, \Pi_{\text{vm}}.\text{Prove}, \Pi_{\text{vm}}.\text{Verify})$$

for the universal relation

$$R_{\text{vm}}((\text{img}, \text{in}_{\text{pub}}, \text{out}); (P, \text{in}_{\text{priv}})) = 1$$

iff $\text{img} = \text{digest}(P)$ and executing the guest program P on ISA with inputs $(\text{in}_{\text{pub}}, \text{in}_{\text{priv}})$ terminates and emits public output out . The public instance is $x_{\text{vm}} = (\text{img}, \text{in}_{\text{pub}}, \text{out})$ and the witness is $w_{\text{vm}} = (P, \text{in}_{\text{priv}})$.

Host-guest model. A zkVM partitions execution into a *guest* program that runs on the virtual processor under the proof system, and a *host* program that runs natively on the prover machine. The host orchestrates proof generation, supplies inputs to the guest via a read channel, and receives the guest's public output via a commit channel. Values placed on the commit channel become part of the public statement out ; values placed on the read channel that the guest does not commit remain in the witness in_{priv} . In the BDEC instantiation considered in this thesis, the user's long-term secret key sk_U and the credential signatures appear only via the read channel and are never committed, so they remain in w_{vm} across the entire zkSNARK relation.

Receipt structure and the zero-knowledge property. The output of $\Pi_{\text{vm}}.\text{Prove}$ is a *receipt* consisting of a journal of committed outputs together with a succinct proof. Concrete instantiations distinguish two regimes:

- A *succinct STARK receipt* is transparent and large, and verifies in time logarithmic in the number of executed cycles. Whether such a receipt satisfies the formal zero-knowledge property of Section 3 depends on the specific instantiation; in the systems we evaluate the default succinct STARK receipt is not formally zero-knowledge.

- A *SNARK-wrapped receipt* is obtained by composing the succinct STARK with an outer zero-knowledge SNARK (e.g. Groth16 or PLONK). The wrapped receipt is small and zero-knowledge, but the wrap step itself has additional prover memory and time cost.

Any privacy claim about an anonymous-credential scheme instantiated on a zkVM that relies on the underlying zkSNARK being zero-knowledge therefore holds only for the wrapped receipt. We make this dependency explicit when stating anonymity and unlinkability of the BDEC instantiation in Section 5.

Precompiles. A *precompile* is a fixed sub-circuit recognised by the zkVM’s instruction decoder as a syscall and proved natively by the prover rather than via emulated instruction-level execution. From the guest’s perspective a precompile call costs a single syscall instruction; from the prover’s perspective the cost is the proof of one execution of the precompile’s underlying AIR (algebraic intermediate representation). Precompiles trade proof-system generality for performance on algebraic primitives whose software emulation over the prover’s native field would otherwise dominate the trace. A precompile is sound only if (i) its AIR enforces exactly the intended function, accepting no more inputs than the reference specification, and (ii) its memory inputs and outputs are bound to the surrounding VM execution trace via the cross-table lookup argument; otherwise the syscall could be satisfied by values unrelated to the guest’s computation. This functional-equivalence-plus-trace-binding condition is the assumption addressed for the Griffin precompile in Section 6 as Assumption 6.1, where its status is examined.

We refer to the cost of emulating n -bit field arithmetic in software when the underlying prover field is small as the *field-mismatch tax*. A precompile that implements the same arithmetic natively absorbs this tax. The Griffin- $\mathbb{F}_p^{192}$ precompile introduced and measured in this thesis is the load-bearing instance of this pattern.

Update churn. *Update churn* denotes the engineering and verification cost incurred when a deployed proof system must be changed in response to evolving policy, security parameters, or primitive substitutions. In a relation-specific zkSNARK every such change requires regenerating and re-auditing the relation-specific circuit and its parameters; this regeneration-and-re-audit footprint, rather than the recompile wall-clock (which can be small for a transparent argument), is the dominant cost. In a zkVM the same change is a guest-program edit; the universal relation R_{vm} and the trusted setup (if any) are unchanged. Update churn is the evaluation dimension we add to runtime cost in Section 7.

3.6.1 RISC Zero

RISC Zero [16] is a zkVM instantiation whose ISA is RV32IM. The prover is a recursive STARK over the BabyBear field $\mathbb{F}_p$ with $p = 2^{31} - 2^{27} + 1$. Per [16], the prover decomposes guest execution into segments of bounded cycle count, produces a STARK proof per segment, and recursively folds segment proofs into a single succinct receipt. Application-defined precompiles are supported via the Zirgen big-integer dialect.

A RISC Zero receipt is verified against the public statement (img, out), where img is the digest of the guest ELF and out is the contents of the commit journal. The image digest pins the receipt to a specific guest binary: a receipt for one guest does not verify under the image digest of any other guest.

3.6.2 SP1

SP1 [20] is a second zkVM instantiation whose ISA is RV64IM (the `riscv64im-succinct-zkvm-elf` guest target of the SP1 v6.2.1 fork evaluated here) and whose prover is built on the Plonky3 toolkit. Per the SP1 documentation, the default prove path produces a succinct STARK receipt that is not formally zero-knowledge; a zero-knowledge receipt is obtained by wrapping the STARK in a SNARK (the `groth16()` or `plonk()` proof modes in the SP1 SDK, which internally compress the STARK before wrapping). SP1 supports application-defined precompiles via its custom-precompile framework.

3.6.3 Static-circuit and zkVM proof systems as comparators

We will instantiate BDEC in two regimes that this thesis compares directly:

1. A *static-circuit* regime, in which the BDEC CreGen and ShowCre relations are encoded into a fixed R1CS and proved via Aurora (Section 3 above). Each change to the relation requires recompilation of the R1CS.
2. A *zkVM* regime, in which the same relations are expressed as guest programs on RISC Zero (and SP1, where indicated). Changes to the relation become guest-program edits; the universal relation R_{vm} is unchanged.

The runtime cost, memory footprint, and update-churn properties of these two regimes are the empirical subject of Section 7.

4 Theoretical Foundations of the Arithmetic Mismatch

This section sets up the formal cost framework against which the rest of the thesis is measured. We name the dimensions along which the static-circuit and zkVM regimes will be compared (Section 4.1), formalise the *field-mismatch tax* as the quantitative manifestation of the arithmetic mismatch (Section 4.2), formalise *update churn* as a deployment-lifecycle dimension (Section 4.3), and decompose the cost of the BDEC CreGen and ShowCre relations into the components that dominate either regime (Section 4.4).

4.1 Cost Dimensions

We measure each system regime along four dimensions.

Time. Prover and verifier wall-clock time, measured in seconds on the target hardware. For the prover this is the time from invocation of $\Pi.\text{Prove}$ (or $\Pi_{\text{vm}}.\text{Prove}$) until the receipt is produced; for the verifier it is the time from invocation of the verification predicate until the binary outcome is returned.

Memory. Peak resident-set size (peak RSS) of the prover process on the target hardware, measured in gigabytes. Verifier memory is reported when it materially differs from prover memory; otherwise it is dominated by the receipt size.

Proof size. The size of the produced receipt, measured in kilobytes for the relevant receipt class. Of the two receipt regimes of Section 3, the succinct STARK receipt and the SNARK-wrapped receipt, we report the size of whichever is the deployment-relevant figure for the comparison at hand; where the wrap is the deployable artefact but is not produced on the target hardware, its constant size is cited from the toolchain documentation and the workload-dependent core receipt is not serialised (Section 7).

Update churn. The footprint of artefacts that must be regenerated and re-audited when the verification predicate changes by one localised modification. We formalise this dimension in Section 4.3 below.

The first three dimensions are standard in the zkSNARK and zkVM literature [11, 18, 16, 20, 41]. The fourth dimension is introduced in this thesis as a deployment-lifecycle accounting frame. Unlike the first three it is a footprint characterisation rather than a performance metric, and Section 7 shows that it does not translate into a wall-clock advantage against the non-preprocessing static baseline.

4.2 The Field-Mismatch Tax

This section targets one of two distinct field mismatches. The *protocol-level* mismatch, Loquat’s extension-field $\mathbb{F}_{p^2}$ arithmetic, is already eliminated by PLUM’s single-prime design; what remains, and what this section formalises, is the *substrate-level* mismatch, PLUM’s large prime emulated over a zkVM’s small native field. The verification predicate of a post-quantum signature such as Loquat or PLUM is dominated by algebraic-hash (Griffin) permutations together with arithmetic over a prime field $\mathbb{F}_p$ of large characteristic (in PLUM, a nominal 192-bit / concrete 199-bit prime; see Section 3).

Concretely, PLUM [18, §4.2] reports that hash evaluations account for 105,952 of the 116,285 R1CS constraints ($\approx 91\%$) of a PLUM-128 verification, the field-arithmetic constraints making up the remaining $\approx 9\%$; the tax below is carried by this hash-dominated, large-field workload as a whole, and not by field arithmetic on its own. This 91% is a constraint count in a relation-specific zkSNARK and is not a measured zkVM-cycle share: it does not, on its own, fix an Amdahl ceiling on the zkVM cost, and the measured zkVM-cycle attribution is reported separately in Section 7. This constraint figure is moreover reported at the PLUM-128 parameter set, whereas every zkVM measurement in this thesis is taken at $\lambda = 80$ (Section 7); the 91% is therefore a published $\lambda = 128$ constraint share, not a measured hash share at the measured parameter set. When such a predicate is proved by a relation-specific zkSNARK over $\mathbb{F}_p$, every field operation in the predicate is a single constraint over the same field. When it is proved by a zkVM whose prover field $\mathbb{F}_{p_{\text{vm}}}$ is small (BabyBear, $p_{\text{vm}} = 2^{31} - 2^{27} + 1$, in RISC Zero; KoalaBear, $p_{\text{vm}} = 2^{31} - 2^{24} + 1$, in the evaluated SP1 fork), each $\mathbb{F}_p$ operation must be emulated by software running on the zkVM’s RISC-V instruction set, which in turn is encoded in $\mathbb{F}_{p_{\text{vm}}}$. The number of native RISC-V instructions required to emulate one $\mathbb{F}_p$ operation by multi-limb arithmetic is a fixed function of the bit-widths of p and the limb size of the underlying integer encoding.

The emulation tax this definition names is characterised qualitatively in the recent zkVM systematisation [17]. The definition below is a cost-accounting frame that names, in one symbol, the per-operation overhead the precompile suite is built to remove.

Definition 4.1 (Field-mismatch tax). Let op be a field operation over $\mathbb{F}_p$ used inside a verification predicate Pred , and let $\text{cyc}_{\text{em}}(\text{op})$ denote the number of zkVM guest cycles required to emulate op by software on a zkVM whose native field is $\mathbb{F}_{p_{\text{vm}}}$. Let $\text{cyc}_{\text{nat}}(\text{op})$ denote the number of zkVM guest cycles that the same operation would consume if the prover field were $\mathbb{F}_p$ itself, so that no multi-limb emulation is needed; under this idealisation $\text{cyc}_{\text{nat}}(\text{op})$ is the cycle count of a single native field instruction. Because the target zkVMs expose no native $\mathbb{F}_p$ instruction, cyc_{nat} is a conceptual normaliser, not a directly measurable quantity; the empirically measured quantity is the emulated cost Cyc_{em} reported in Section 7. Both quantities are in the same unit, so their ratio is dimensionless. The *field-mismatch tax* of op is

$$\text{FMT}(\text{op}) = \frac{\text{cyc}_{\text{em}}(\text{op})}{\text{cyc}_{\text{nat}}(\text{op})}.$$

Writing $\text{ops}(\text{Pred})$ for the multiset of field operations executed by an honest evaluation of Pred , the cumulative *emulated cost* of Pred is the distinct quantity

$$\text{Cyc}_{\text{em}}(\text{Pred}) = \sum_{\text{op} \in \text{ops}(\text{Pred})} \text{cyc}_{\text{em}}(\text{op}).$$

The tax is not merely a name for a measured overhead; it admits a lower bound that depends only on the two field sizes. Write $\ell = \lceil \log_2 p / \lfloor \log_2 p_{\text{vm}} \rfloor \rceil$ for the number of $\mathbb{F}_{p_{\text{vm}}}$ limbs needed to represent one element of $\mathbb{F}_p$.

Proposition 4.2 (Field-mismatch lower bound). *On any zkVM whose native field is $\mathbb{F}_{p_{\text{vm}}}$ and which exposes no native $\mathbb{F}_p$ multiplication, an emulated $\mathbb{F}_p$ element occupies at least ℓ trace cells, and emulating one $\mathbb{F}_p$ multiplication by a limb-aligned multi-limb routine requires at least ℓ native $\mathbb{F}_{p_{\text{vm}}}$ multiplications. Hence*

$$\text{FMT}(\text{mult}) \geq \ell = \Omega\left(\frac{\log p}{\log p_{\text{vm}}}\right).$$

Schoolbook multi-limb multiplication, the method the evaluated zkVMs use, attains $\Theta(\ell^2)$, so the realised tax per multiplication lies between ℓ and ℓ^2 .

Proof. Representation. An injective encoding of $\mathbb{F}_p$ into tuples of $\mathbb{F}_{p_{\text{vm}}}$ elements needs k components with $p_{\text{vm}}^k \geq p$, so any encoding uses at least $\lceil \log_2 p / \log_2 p_{\text{vm}} \rceil$ cells. Under the base- B limb encoding each cell carries at most $\lfloor \log_2 p_{\text{vm}} \rfloor$ bits, so ℓ limbs, hence ℓ trace cells, are required. *Multiplication.* Decompose the operands into limbs $a = \sum_{i < \ell} a_i B^i$ and $b = \sum_{j < \ell} b_j B^j$, with $B = 2^{\lfloor \log_2 p_{\text{vm}} \rfloor}$. For any fixed nonzero b , multiplication by b is a bijection of $\mathbb{F}_p$, so $a \cdot b \bmod p$ takes distinct values on inputs differing only in the limb a_i , and the routine’s output depends on each a_i . Restrict to a *limb-aligned* routine, in which each native $\mathbb{F}_{p_{\text{vm}}}$ multiplication has at most one limb of a among its two factors and no affine combination of distinct a -limbs is formed before a multiplication; schoolbook multi-limb routines are of this form, while Karatsuba-style routines that pre-combine distinct limbs into a single factor are not. The product is a degree-two function of the inputs, so a_i enters no purely additive term of the output and can influence it only through native multiplications that carry a_i as a factor. Were no such multiplication present, a_i could reach the output only through a b -independent additive coefficient, which cannot reproduce the product’s coefficient $b B^i$ of a_i ; so at least one native multiplication carries each a_i . A limb-aligned multiplication carries at most one of the ℓ distinct limbs $a_0, \dots, a_{\ell-1}$, so these are ℓ distinct multiplications. Normalising by the single native-instruction cost gives $\text{FMT}(\text{mult}) \geq \ell$; the schoolbook method forms all ℓ^2 limb products, attaining the upper end. $\square$

Remark 4.3. For PLUM ($\log_2 p \approx 199$) on a 31-bit host field ($\lfloor \log_2 p_{\text{vm}} \rfloor = 30$, for both RISC Zero’s BabyBear and the SP1 fork’s KoalaBear), $\ell = 7$, so Proposition 4.2 places the per-multiplication tax between 7 and 49. The measured precompile-free-to-precompiled cycle ratio of $56.4\times$ at $\lambda = 80$ (Cell 1 / Cell 2, Section 7) is a whole-workload ratio, dominated by the Griffin permutation’s collapse to a syscall rather than by bare field-multiplication emulation; it lands in the same order as the schoolbook $\ell^2 \approx 49$ bound of Proposition 4.2, which is suggestive of the field-mismatch tax at work rather than a direct instantiation of the per-multiplication bound (that $56.4 > 49$ already precludes reading it literally).

A precompile, as defined in Section 3, replaces $\text{cyc}_{\text{em}}(\text{op})$ for a designated operation by a single zkVM syscall, whose cost from the guest’s perspective is one cycle plus the constant overhead of the syscall ABI. Writing $\text{Pre} \subseteq \text{ops}(\text{Pred})$ for the set of operations covered by precompiles and $\text{cyc}_{\text{syscall}}$ for the per-syscall overhead, the cumulative emulated cost of Pred under a precompile-enabled zkVM is therefore

$$\text{Cyc}_{\text{em}}^{\text{pre}}(\text{Pred}) = \sum_{\text{op} \in \text{ops}(\text{Pred}) \setminus \text{Pre}} \text{cyc}_{\text{em}}(\text{op}) + |\{\text{op} \in \text{ops}(\text{Pred}) \cap \text{Pre}\}| \cdot \text{cyc}_{\text{syscall}}.$$

The performance benefit of adding a primitive to the precompile set is, by this accounting, the difference $\text{cyc}_{\text{em}}(\text{op}) - \text{cyc}_{\text{syscall}}$ summed over its occurrences. This is a *guest-trace* reduction: it counts cycles removed from the emulated execution, not the prover’s wall-clock cost. The precompile shifts those cycles into a relation-specific AIR whose proving cost is charged on the host side, so a positive guest-cycle benefit need not translate into a faster prover, and Section 7 reports a primitive (Griffin) for which it does not.

The host-side charge admits the same style of accounting. A STARK-based zkVM prover commits, per execution shard, one trace matrix per chip, and its work scales quasi-linearly with the total number of committed trace cells; the deployed SP1 toolchain prices its own proving this way (shards are split on

a trace-area threshold, and the executor’s gas formula weights trace area three-to-one over instruction complexity, in `vm/gas.rs`). Write $\bar{a}_{\text{core}}$ for the average number of committed core-chip cells (CPU, memory, and byte-range bookkeeping) per guest cycle, so that the core trace area of Pred under the precompile-enabled zkVM is $A_{\text{core}}(\text{Pred}) = \bar{a}_{\text{core}} \cdot \text{Cyc}_{\text{em}}^{\text{pre}}(\text{Pred})$. Each precompile P additionally contributes its own chip area

$$A_P(\text{Pred}) = E_P (r_P w_P + w_{\text{ctrl},P}),$$

where E_P is the number of syscall events Pred issues to P , r_P the number of trace rows the chip emits per event, w_P its main-trace width in columns, and $w_{\text{ctrl},P}$ the width of the controller row that binds each event’s memory traffic. The per-shard core-proving component of prover wall-clock is then modelled as

$$T_{\text{prove}}(\text{Pred}) \approx \kappa_{\text{core}} A_{\text{core}}(\text{Pred}) + \sum_P \kappa_P A_P(\text{Pred}), \quad (1)$$

where each κ is an effective per-cell cost for the chip family in question, absorbing commitment, constraint evaluation, lookup-argument columns, and padding; the absorbed terms only enlarge κ_P , so the accounting errs conservatively against precompiles. The guest-trace benefit computed above is the reduction in A_{core} ; the term it omits is A_P . Both terms price the per-shard *core* prove; the recursive-compression and wrap stage is a separate cost regime, measured empirically in Section 7 and not represented in this model.

Proposition 4.4 (Precompile break-even). *Under the trace-area model of Equation (1), with the per-cycle core area $\bar{a}_{\text{core}}$ common to the precompiled and unprecompiled executions, adding an operation op to the precompile set lowers the core-proving wall-clock if and only if, per covered occurrence,*

$$(\text{cyc}_{\text{em}}(\text{op}) - \text{cyc}_{\text{syscall}}) \cdot \bar{a}_{\text{core}} \kappa_{\text{core}} > (r_P w_P + w_{\text{ctrl},P}) \cdot \kappa_P, \quad (2)$$

that is, when the core area removed outweighs, at the chips’ per-cell rates, the AIR area added.

Proof. Immediate from Equation (1): replacing the emulated occurrences of op by syscalls lowers A_{core} by $(\text{cyc}_{\text{em}}(\text{op}) - \text{cyc}_{\text{syscall}}) \bar{a}_{\text{core}}$ per occurrence while adding the chip area A_P ; the net change in T_{prove} is negative exactly under the stated inequality. $\square$

The two prove-mode outcomes of Section 7 are this inequality evaluated at two operating points. Against its own emulated baseline, the Griffin permutation removes millions of guest cycles per event at a per-event chip area on the order of 10^4 cells, so the inequality holds by a wide margin and the precompile recovers feasibility. Against a control predicate whose hash runs as cheap native RISC-V software, the covered operation does not occur on the control side at all: the precompiled arm executes more guest cycles and carries the full chip area besides, so both terms of Equation (1) are larger on that arm and the model places it behind the control for every non-negative choice of the κ ’s, provided $\bar{a}_{\text{core}}$ is common to the two arms. Section 7 instantiates the dimensions and calibrates the constants. The same accounting explains why vendor precompiles in production zkVMs report cycle-count collapses far larger than their prove wall-clock gains: the cycle ratio bounds the wall-clock ratio from above, and the bound is approached only when the precompile’s per-event chip area is negligible against the remaining core area, which for permutation-sized chips it is not (SP1’s own cost table prices its Keccak chip at 2,640 columns (`rv64im_costs.json`) with 24 rows per permutation, a per-event area comparable to the bespoke Griffin chip’s).

4.3 Update Churn

Definition 4.5 (Localised verification-predicate change). A *localised verification-predicate change* is a modification to the verification predicate Pred that can be described as one of the following: substitution of one signature scheme for another at matching security level; retuning of a single numerical security parameter (e.g. λ, t, η); a change to the presentation *structure* of a showing relation (the number of credentials shown, or the set of disclosed attributes); or a one-line change to the encoding of a public input. A change that only alters which condition the verifier checks on the *disclosed* attributes, a presentation *policy* at fixed structure, is *not* a verification-predicate change: following base BDEC it is evaluated outside the proof relation (Section 3) and touches neither Pred nor its compiled artefacts.

The deployment-lifecycle cost of a localised change is dominated by the artefacts that must be regenerated and re-audited as a consequence, rather than by the change itself. We formalise this cost as follows.

Definition 4.6 (Update churn). Fix a deployment $\mathcal{D}$ that proves Pred in either the static-circuit or the zkVM regime, and let Δ denote a localised verification-predicate change. The *update-churn footprint* of Δ in $\mathcal{D}$ is the tuple

$$\text{Churn}(\Delta, \mathcal{D}) = (N_{\text{src}}(\Delta, \mathcal{D}), N_{\text{circ}}(\Delta, \mathcal{D}), N_{\text{param}}(\Delta, \mathcal{D}), N_{\text{audit}}(\Delta, \mathcal{D})),$$

where

N_{src} is the number of source-code files that must be modified in response to Δ .

N_{circ} is the number of arithmetic circuits or AIR programs that must be regenerated, counted as the distinct circuit or AIR artefacts whose serialised form changes between the pre- and post- Δ builds.

N_{param} is the number of proof-system parameters (e.g. commitment parameters, R1CS-shape-dependent constants) that must be re-derived.

N_{audit} is the number of distinct components requiring re-audit before redeployment, counted at the granularity of the deployment’s existing security-review boundary. This is an *audit-footprint* count, not a weighted effort model: it reports how many components re-enter the review boundary, not the person-time or risk class of reviewing them.

For the static-circuit regime, a localised change to Pred typically yields a non-trivial N_{circ} (the R1CS must be recompiled), a non-trivial N_{param} (constants that depend on the constraint count must be re-derived), and a non-trivial N_{audit} (the new circuit shape must be audited). For the zkVM regime, the same change always yields a non-trivial N_{src} (the guest program is edited) and zero regenerations of the *universal* relation R_{vm} , which is unchanged. The remaining components depend on whether the change touches a precompiled primitive. The bespoke Griffin _{$\mathbb{F}_p^{192}$} AIR is itself a relation-specific circuit; a change that alters the hash family or the field width therefore regenerates and re-audits that AIR, so $N_{\text{circ}} \geq 1$, $N_{\text{param}} \geq 1$, and $N_{\text{audit}} \geq 1$. The zero-circuit, zero-parameter case applies only to changes that leave every precompiled primitive intact (e.g. raising the presentation count k from 1 to 2), for which $N_{\text{circ}} = 0$, $N_{\text{param}} = 0$, and N_{audit} is confined to the changed guest source. The configuration that would need zero new circuits for *any* change is the pure-emulation zkVM without

precompiles, which Section 7 measures as infeasible (does not finish within the resource bound on the target hardware).

The empirical evaluation of Section 7 tabulates the circuit and parameter components N_{circ} and N_{param} for both regimes and discusses the source-edit and audit components N_{src} and N_{audit} in prose, under a fixed set of localised changes that we exhibit as benchmarks: signature substitution (Loquat $\rightarrow$ PLUM), security-parameter retuning ($\lambda = 80 \rightarrow 128$), and a presentation-structure change (raising the credential count k from 1 to 2).

4.4 Cost Decomposition of BDEC Relations

We now apply the cost framework above to the two BDEC relations of interest. We treat each relation as a multiset of cost-bearing components and compute the cumulative cost under each regime.

CreGen. The BDEC CreGen relation $R_{\text{cre}}^{\text{Sig}}$ defined in Section 3 is the base two-verification relation: two signature-verification predicates $\text{Pred}_{\text{Verify}}^{\text{Sig,PPSig}}$ under a hidden user public key. The credential digest $\rho_{U,TA} = H(\cdot)$ is computed at issuance, and ledger membership is checked outside the proof, as in ShowCre; the full acceptance condition is therefore the conjunction of the in-proof signature-verification predicates with the verifier-side membership check performed by BDEC.CreVer/BDEC.ShowVer (Section 3), so a proof for valid signatures whose digest is absent from the accepted ledger is rejected outside the proof. Of these, the two $\text{Pred}_{\text{Verify}}^{\text{Sig,PPSig}}$ predicates dominate the cost. Inside PLUM verification (Section 3), the load-bearing components are (i) the t -th power-residue PRF symbol checks, (ii) the BCS Merkle-commitment recomputations, and (iii) the low-degree-test (LDT) sub-protocol invocations. The cost of (ii) and (iii) is dominated by the algebraic hash function H_{Griffin} over $\mathbb{F}_p$, which is precisely the primitive absorbed by the $\text{Griffin}_{\mathbb{F}_p^{192}}$ precompile.

ShowCre. The ShowCre proof circuit contains $k + 2$ signature-verification predicates (where k is the number of credentials shown): k pseudonym-ownership checks, one verifier-facing pseudonym check, and one shown-credential check (Section 3). Ledger membership, non-revocation, and the disclosure predicate are checked by the relying party outside the proof. For modest k (the design target is $k \leq 4$), the cost is dominated by these signature-verification predicates and the algebraic-hash operations inside their Merkle and low-degree-test components.

In both relations, the field-mismatch tax of Definition 4.1 is concentrated in the signature-verification predicates: every Griffin permutation involved in a Merkle path or in PLUM’s internal hashing is a tax-bearing $\mathbb{F}_p^{192}$ operation under a zkVM with a small native field.

The decomposition locates the dominant term of both relations in the Griffin permutation count and the field-mismatch tax it bears. It also separates two costs that the framework keeps apart: the per-proof cost, which the precompile suite attacks, and the regeneration cost incurred when a relation changes, which, for changes leaving the precompiled primitive suite intact, only the static-circuit regime pays (a hash-family or field-width change regenerates the Griffin AIR on the zkVM side too; Section 4.3). Section 7 measures the per-proof term directly and characterises the regeneration term as a footprint: where the zkVM cost falls and how much of it the precompile removes, the frontier at which the unmitigated and the zero-knowledge-wrapped configurations cease to complete on the target hardware (the former on memory and time, the latter on the recursion-shape provisioning wall; §7.6.2), and the recompile/setup time together with the enumerated regeneration footprint that separates the two

regimes under a presentation, signature, or security-parameter change. The re-audit burden is reported as a component-count footprint and is not quantified as effort; the static baseline is Loquat/Aurora over $\mathbb{F}_{p_{127}}$ rather than same-scheme PLUM. The next chapter constructs the precompile suite as the instrument for measuring the dominant term this decomposition has located. Equation (2) governs the per-shard *core* prove (whether the chip’s core-trace saving outweighs its own chip area), and Section 7 answers it in both directions: feasibility recovery, and a Griffin arm that proves slower than a SHA-3 control. It does not price the recursive-compression and wrap layer, a separate cost regime in which the same Griffin chips enlarge the recursion verifier’s core machine, the recursion-shape provisioning wall that blocks the zero-knowledge wrap (§7.6.2), measured empirically in Section 7 rather than by this model.

5 Construction: The Precompile Suite and PLUM-in-BDEC

The theoretical analysis of Section 4 identified the algebraic-hash and large-prime-field operations of PLUM verification as the operations that carry the field-mismatch tax inside a small-field zkVM. This section presents the instrument we built to absorb that tax so that the remaining proving cost can be measured: a precompile suite, developed in two stages, and its integration into the BDEC credential-generation relation. The suite serves the measurement reported in Section 7. Its purpose is to move the dominant cost out of emulated guest code so that what remains can be measured, attributed, and bounded. Throughout, we grade the evidence supporting each component, distinguishing what is proved from what is empirically tested and what is trusted from upstream, since the suite is not uniformly sound. Two soundness questions are left explicitly open (§5.5); a measurement whose correctness rested on unstated assumptions would be worth little, so stating the open questions is part of the work. The same Griffin chip that lowers the core-layer cost enlarges the recursion verifier’s core machine, and that enlargement, quantified in Section 7, is what the zero-knowledge wrap obstruction inherits.

5.1 Scope of the Implementation

We state plainly what is and is not built, since the honesty of every measurement in Section 7 depends on this scope being explicit.

- **Fp192 modular multiplication via the host precompile** (§5.2), routed through SP1’s `UINT256_MUL` and RISC Zero’s `sys_bigint`, implemented for both targets. *Built; reduction argued; empirically tested (14/14 adversarial cases).*
- **A bespoke Griffin- $\mathbb{F}_p^{192}$ permutation AIR for SP1** (§5.3), the load-bearing precompile, since PLUM’s constraint budget is dominated by Griffin. All ten constraint families implemented ($\approx$ 3,900 lines of Rust across the chip and executor-compute modules, over 24 commits); a single-permutation smoke proof completes (execute+prove+verify) in 13.0 s on the target hardware. *Built; constraint-level chip-equivalence not yet verified (§5.5).*
- **PLUM signature verification on SP1**, suite active. *Built; proves end-to-end (14.25 min at $\lambda = 80$, mean of three fixed-config runs, tuned memory configuration; Section 7).*
- **PLUM signature verification on RISC Zero** (standalone guest, Griffin hasher emulated in guest code, field arithmetic via `sys_bigint`). *Built; proves end-to-end (6 h 19 m at $\lambda = 80$; Section 7).*
- **PLUM-in-BDEC credential generation (CreGen) on RISC Zero**, with field arithmetic via `sys_bigint`. *Built; validated in execute mode (9.4×10^9 guest cycles, both signature verifications accepting); its succinct proof on RISC Zero did not yield a receipt, ending in an internal-verifier rejection under diagnosis rather than a resource limit; on SP1 the same relation completes a succinct prove (26.98 and 31.11 min, $n = 2$; receipt verified, not zero-knowledge) (Section 7).*
- **PLUM-in-BDEC presentation (ShowCre) on RISC Zero**, the $k + 2$ -verification generalisation of CreGen. *Built; validated in execute mode at $k = 1$ (1.41×10^{10} guest cycles) and $k = 2$*

(1.88×10^{10} cycles), all $k + 2$ verifications accepting; its succinct proof on RISC Zero was not attempted, sharing CreGen’s prover path, whereas on SP1 it completes a succinct prove (40.15 and 44.24 min at $k = 1, n = 2$; 58.14 min at $k = 2, n = 1$; receipt verified, not zero-knowledge) (Section 7).

One component is deliberately *not* built, and we treat it as future work: a port of the bespoke Griffin AIR to RISC Zero’s syscall ABI. For reading Section 7 this matters, because “the precompile” does not denote one artefact uniform across both zkVMs. On SP1 the suite includes the bespoke Griffin AIR. On RISC Zero the same arithmetic is carried by `sys_bigint`. We state which mechanism is active wherever a measurement is reported.

5.2 The Field-Arithmetic Substrate: $\mathbb{F}_{p192}$ Multiplication via the Host Precompile

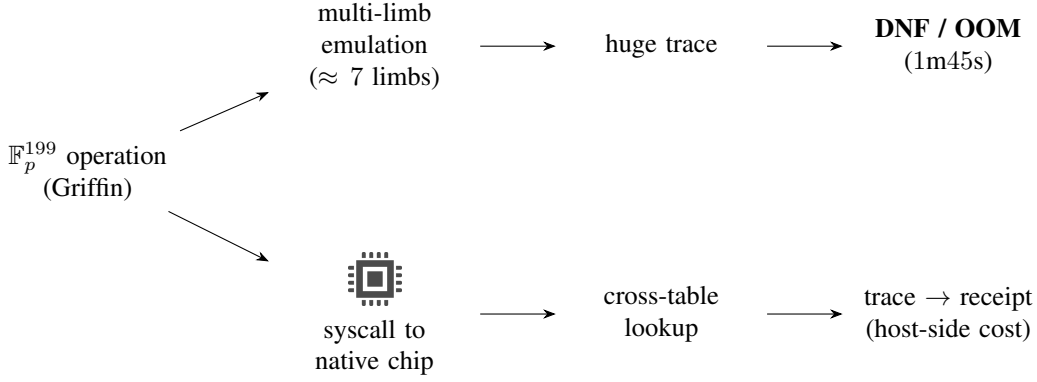
Field and naming. PLUM operates over the prime field $\mathbb{F}_p$ with $p = 2^{64} \cdot p_0 + 1$, p_0 a 135-bit prime, so p is a 199-bit prime. We retain the label “ $\mathbb{F}_p^{192}$ ” for parity with the PLUM paper, whose §3.3 titles the parameter “192-bit smooth prime field”; all soundness arguments use the true 199-bit modulus, which is fixed in the implementation and audited against the paper.³

Construction. We do not write a dedicated 199-bit multiplication circuit. Both target zkVMs already ship a 256-bit modular-multiplication precompile modulo a runtime-supplied modulus: SP1’s `UINT256_MUL` and RISC Zero’s `sys_bigint (OP_MULTIPLY)` [20, 16]. Each $\mathbb{F}_{p192}$ multiplication $a \cdot b \bmod p$ is computed by zero-padding a , b , and p into the precompile’s 256-bit slots and invoking the syscall with modulus $m = p$. The wiring is selected at compile time per target.

Soundness (reduction).

Upstream assumption. For every row of the host modular-multiplication table in a verifying proof, the host AIR enforces $x' \equiv x \cdot y \pmod{m}$ and $x' \in [0, m)$ when $m \neq 0$. For SP1 we point to the two specific AIR constraints that enforce these (the modular product and the output range check); for RISC Zero the same guarantee is enforced by auto-generated constraint code that we *cite* as production-audited rather than re-derive. This RISC Zero citation is the weakest link and is graded accordingly. The precompiles are those of SP1 v6.2.1 (forked) and RISC Zero 3.0.5 (`risc0-circuit-rv32im 4.0.4`), the versions the repository build resolves; the CreGen anomaly run of Section 7 ran on the same build (the compiled library embeds the 3.0.5 sources, built thirteen days before that run). The assumption is thus pinned to these specific versions and not to an unversioned “upstream release process.”

³Two distinct points must not be conflated. (i) The 192-vs-199 *label* is a naming convention: PLUM’s §3.3 titles the field “192-bit” while the concrete modulus is 199 bits; we keep the paper’s label but compute with the true value. (ii) Separately, the specific decimal prime is a *substitution*: as recorded in Remark 3.5 and the open prime-instantiation hypothesis of Section 6, the value used is a nearby prime satisfying PLUM’s structural requirements, and any bit-exact security claim is conditional on it matching the intended parameters. This conditional dependence is an open item rather than a naming convention.



Edge cases. The argument addresses a malicious modulus $m = 0$ (prevented by loading the modulus from a compile-time constant the zkVM’s memory checking binds), non-canonical operand truncation (guarded by a debug assertion on operand width), and regression of the upstream range check (under which the omitted reduction would become load-bearing, detected by the same assertion).

Empirical corroboration. An adversarial probe exercises an honest control plus six single-field tamper cases (corrupted public key, message, and four signature fields: the σ_1 Merkle root, the PRF tag, a virtual-oracle response, and a final STIR coefficient) on each zkVM; all 14 cases across both platforms behave as expected (honest accepts, every tamper rejects). This is regression evidence; it does not characterise malicious-prover behaviour completely, and we report it as such.

5.3 The Bespoke Griffin- $\mathbb{F}_p^{192}$ Permutation AIR (SP1)

No existing SP1 chip computes a Griffin permutation over a 199-bit prime field; the closest upstream pattern is SP1’s Poseidon2 chip [20], implementing the Poseidon2 permutation [52], which has the same shape (a permutation precompile with single-pointer state I/O, per-round columns, and memory binding) but operates over SP1’s 31-bit base field (KoalaBear, $2^{31} - 2^{24} + 1$, in the evaluated fork). We therefore implement a custom AIR for one Griffin- π permutation over $\mathbb{F}_p^4$, exposed to the guest as a single syscall on a 16-word (4-lane) state.

Per-round constraints. Each of the 14 rounds is encoded by five per-round constraint families (the round-function families of Appendix B): a degree-3 forward S-box on one lane; an *inverse* S-box on another lane, constrained in the backward direction ($y^d = x$ with the root witnessed) because the forward inverse exponent is a 199-bit number that would blow past the constraint-degree budget, valid because $\gcd(d, p-1) = 1$ makes the cube map a bijection, so the witnessed root is unique; this side condition is *not* automatic and we verify it for the implemented modulus: with $d = 3$ and $p = 2^{64}p_0 + 1$ we have $p - 1 = 2^{64}p_0 \equiv p_0 \equiv 1 \pmod{3}$ (since $2^{64} \equiv 1 \pmod{3}$, and $p_0 \equiv 1 \pmod{3}$ for the implemented 199-bit modulus, verified by computation), hence $3 \nmid (p-1)$ and $\gcd(3, p-1) = 1$ (were $3 \mid (p-1)$ the cube would be three-to-one and a malicious prover could witness a wrong root); a quadratic layer on the remaining two lanes; the linear layer (the circulant $[2, 1, 1, 1]$, which is invertible but not MDS, unlike Griffin’s standard width-4 matrix); and the SHAKE256-derived round constants. The chip’s remaining five families (round count, memory binding, output canonicity, row scheduling, parameter immutability) constrain the trace as a whole, as described next. The modulus, round count, linear-layer entries, round constants, and quadratic coefficients are bound in preprocessing rather than supplied per row, so a prover cannot choose them. A separate controller chip binds the syscall’s memory reads and writes, and a per-lane output range check enforces output canonicity (§5.2’s cross-compile contract). Two audit-driven hardening passes fixed a polynomial-degree blow-up and an all-zero-padding-row identity failure.

What works. A single-permutation smoke proof passes execute, prove, and verify in 13.0 s on the target hardware, establishing the per-syscall proving cost in isolation.

5.4 The Sponge and Compression Wrapper

The permutation is wrapped in the sponge/compression construction of the Loquat instantiation to provide the 2-to-1 compression used by the BDEC Merkle trees. The state width is $t_{\text{state}} = 4$, the capacity $c_{\text{cap}} = 2$, and the rate $r = t_{\text{state}} - c_{\text{cap}} = 2$, so a digest is the first two lanes of the output state. The 2-to-1 compression of two child digests $x = (x_0, x_1)$ and $y = (y_0, y_1) \in \mathbb{F}_p^2$ is $H_2(x, y) = (\pi(x_0, x_1, y_0, y_1))_{0,1}$: one application of the width-4 permutation π , truncated to the rate lanes. The SHAKE256 domain-separation seed and the linear-layer matrix are matched to the reference hasher, and cross-codebase tests pin the modulus, round count, linear-layer matrix, and seed against the reference implementation.

5.5 Soundness of the Precompile Suite, and Two Open Questions

This subsection states the soundness claim and the two questions it leaves open. The per-constraint audit is deferred to Appendix B.

Theorem 5.1 (Precompile soundness, conditional). *This is a functional (input–output) equivalence statement, and, conditional on the black-box-transfer hypothesis, a transfer of PLUM’s EU-CMA reduction; it is not a standalone cryptographic-security claim. Assume the host zkVM’s cross-table lookup argument is sound; Assumption 6.1 (the Griffin AIR’s constraints are satisfied only by a witnessed transition equal to the reference Griffin- π permutation); and that PLUM’s EU-CMA reduction treats Griffin and field multiplication as black-box input–output maps, inspecting no intermediate representation (such as a limb of a field product); together with the upstream modular-multiplication*

guarantee of §5.2. Then substituting the precompile suite for the emulated operations preserves the input–output value of every PLUM and BDEC verification predicate. Consequently, under the black-box-transfer hypothesis, PLUM’s EU-CMA reduction transfers unchanged: that reduction is in the random-oracle model and depends only on the input–output behaviour of the hash function instantiating the oracle, which, granting Assumption 6.1, the precompile reproduces exactly: under that assumption it computes the same function the emulated code did, so no random-oracle programming step in the reduction is affected. (The reduction’s reliance on (Q)ROM-replaceability of this Griffin instantiation is a separate, upstream assumption, recorded as an open upstream assumption in Section 6 and not discharged here.)

What is established. The field-substrate half rests on the reduction of §5.2. The suite is platform-specific: on SP1 it includes the bespoke Griffin AIR, to which Assumption 6.1 applies; on RISC Zero the Griffin arithmetic is carried by `sys_bigint` emulation, so Assumption 6.1 is not invoked there and is replaced by the analogous input–output equality of the emulated permutation. For the Griffin AIR, the per-family analysis (Appendix B) exhibits, family by family, the constraint *intended* to catch any deviation from reference Griffin, the forgery that deviation would enable, and the implementation status of each; memory binding, output canonicity, parameter immutability, and row scheduling are implemented and constrained. What that analysis does *not* establish, and says so explicitly, is that the implemented constraint system is satisfied *only* by reference-equal witnesses: this is Assumption 6.1, the *chip–constraint equality*. It is empirically supported by the 256-vector cross-codebase agreement and the partial fault-injection coverage discussed below (per-vector false-pass probability $\approx 2^{-199}$ for the vector suite), but the vector agreement pins the *executor* against the reference, a weaker *chip–executor* equality, not the *constraints*; Assumption 6.1 is not verified at the constraint level.

Open question 1: Assumption 6.1 is empirically supported but unproved. The 256-vector agreement establishes that the executor’s native compute equals the reference permutation; it does *not* by itself establish that the *AIR constraints* admit only that computation (that the chip witness equals the executor). For the per-row layer (the per-row families) that stronger statement now carries a written proof: Appendix B bridges the limb identities the chip constrains over its small base field to congruences modulo p and composes them into the reference round, conditional only on the byte-range lookups that the lookup-binding hypothesis already covers. A separate fault-injection suite corroborates this layer in the rejection direction via `debug_constraints`: single-cell corruptions of a satisfying trace are rejected for each of the seven families. The lookup-borne families (round count, memory binding, and the cross-row threading of row scheduling) lie outside both the proof and that harness and require a full-prover harness; parameter immutability is argued by construction. What Assumption 6.1 still carries is the fidelity of the code-to-constraint transcription together with the lookup-borne families; closing it requires deriving the AIR witness from the same reference code and checking constraint satisfaction directly, ideally by machine.

Open question 2: the QROM status of this Griffin instantiation is upstream of the AIR. PLUM’s EU-CMA argument under Fiat–Shamir requires Griffin to be replaceable by a (quantum) random oracle. The Griffin analyses cover the original instantiations; whether they cover this $d = 3$, width-4, 199-bit-field, 14-round instantiation is not something the AIR can settle. If this instantiation were QROM-distinguishable, no amount of AIR engineering would recover security. We flag this as a

thesis-level assumption that still has to be checked against the Griffin literature; we do not close it here.

Further limitations not hidden. The RISC Zero `sys_bigint` soundness is cited from the upstream release process rather than re-derived; the EU-CMA reduction is argued to transfer on the grounds that the reference reduction treats Griffin and field multiplication as black-box input–output maps, but we have not audited PLUM’s reduction line by line for a step that inspects an *intermediate* representation (such as a limb of a field product), which input–output equality alone would not cover; and the zero-knowledge property required by BDEC’s anonymity guarantee is a property of the proof wrap, analysed as the security–deployability tension of Section 6 and measured in Section 7, not closed here.

5.6 Integration into the BDEC Credential-Generation Relation

The credential-generation relation $R_{\text{cre}}^{\text{PLUM}}$ of Section 3 is the AND concatenation of two PLUM signature verifications, both under the user’s public key pk_U , with public statement $x_{\text{cre}} = (c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA})$ and witness $w_{\text{cre}} = (\text{pk}_U, \text{psk}_{U,TA})$. We implement this relation *in full* as a RISC Zero guest, with PLUM in place of Loquat and field multiplications routed through `sys_bigint`; the guest computes exactly the two $\text{Pred}_{\text{Verify}}$ conjuncts that define base CreGen, with no omitted predicate. (The optional Merkle-accumulator variant of Remark 3.7 is not part of this relation and performs its membership checks in `BDEC.ShowVer`; it is out of scope here.) One *statement-binding* requirement remains open at the receipt level, and it bears on security, so we do not treat it as a cosmetic completeness gap: the prototype commits the verification outcome to the journal but not yet the public statement x_{cre} itself, so a receipt currently attests “two signatures verify under some witness” without binding the proof to the specific public $(c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA})$. Until x_{cre} is committed to the journal, the artifact serves as a *functional benchmark* and does not yet act as a statement-bound credential verifier: an accepting receipt does not certify *which* statement was proved. Committing x_{cre} is a small, cost-negligible change (a few field elements) and is noted as immediate next work (Section 9); it does not alter the proved relation. In execute mode at $\lambda = 80$ the guest runs both verifications to acceptance in 9.4×10^9 guest cycles, the two costing essentially the same ($\approx 4.69 \times 10^9$ each), confirming that both pass through the same precompile-backed path. The transition from this execute-mode validation to a completed proof is the subject of Section 7. On SP1 both relations complete a succinct (non-zero-knowledge) prove: CreGen in 26.98 and 31.11 min, ShowCre in 40.15 and 44.24 min at $k = 1$ and 58.14 min at $k = 2$ (Section 7), a receipt distinct both from RISC Zero’s internal-verifier rejection above and from the still-open zero-knowledge wrap. The full presentation relation (`ShowCre`), the $k + 2$ -verification generalisation of this relation, is likewise implemented as a RISC Zero guest and validated in execute mode at $k = 1, 2$ (Section 7); its zero-knowledge proof, like CreGen’s, has not been produced on the target hardware (Section 7).

6 Security Analysis

Section 5 described an instrument: a precompile suite that moves PLUM’s dominant field operations out of emulated guest code, together with a RISC Zero realisation of the BDEC credential-generation relation (on SP1, PLUM verification runs in isolation). A performance instrument is meaningful for a credential system only if it does not silently weaken the security the system provides. This section makes that requirement precise. We fix the security model and threat model in Section 6.1, prove in Section 6.2 a preservation theorem that bounds how far the move to a zkVM substrate and the precompile suite can shift the adversary’s advantage in each of BDEC’s security experiments, and then isolate in Section 6.3 the one property the analysis does not deliver on consumer hardware, namely zero-knowledge, and hence anonymity, of the proof actually produced. This last point follows from how the toolchain produces proofs and not from any weakness in the construction itself, so we treat it as a finding about the substrate.

6.1 Security Model and Threat Model

BDEC is proved secure in three experiments, formalised in Section 3. Unforgeability requires that an adversary with issuance and showing oracles cannot produce an accepting showing for a statement outside its honestly issued set, beyond negligible advantage. Anonymity and unlinkability are challenge-bit experiments in which the adversary cannot distinguish, beyond advantage $|\Pr[b' = b] - \frac{1}{2}|$, which of two honestly issued credentials produced a challenge showing (anonymity) or which of two pseudonym keys issued a single challenge credential (unlinkability). We write $\text{Adv}_{\text{AC}}^{\text{unf}}$, $\text{Adv}_{\text{AC}}^{\text{anon}}$, and $\text{Adv}_{\text{AC}}^{\text{unlink}}$ for the respective advantages, all defined in Section 3.

Definition 6.1 (zkVM-BDEC advantage). For $\bullet \in \{\text{unf}, \text{anon}, \text{unlink}\}$, write $\text{Adv}_{\text{zkVM-BDEC}}^{\bullet}(\mathcal{A})$ for the advantage $\text{Adv}_{\text{AC}}^{\bullet}(\mathcal{A})$ of Section 3 evaluated on the BDEC scheme instantiated with PLUM as the signature scheme Sig and the zkVM as the zkSNARK Π .

Assumption 6.1 (Griffin-AIR constraint soundness). The Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation: its constraint system is satisfied solely by a witnessed transition equal to reference Griffin- π , equivalently the arithmetisation soundness error ϵ_{AIR} is negligible. This assumption is *open*. It is argued at the specification level in Appendix B and empirically supported by the 256-vector cross-codebase equivalence suite together with rejection-direction fault injection on seven of the ten constraint families; it is not established at the constraint level for the lookup-borne families and is not machine-checked.

Assumption 6.2 (PLUM signature-transcript simulatability). PLUM admits a signature-transcript simulator with negligible error ϵ_{sZK} , in the sense of Definition A.3. This assumption is *open*: PLUM’s analysis establishes EU-CMA security, not simulatability. It is required only by the anonymity and unlinkability bounds, whose BDEC reductions simulate the credential and pseudonym transcripts with the signature’s own simulator.

Assumption 6.3 (Griffin (Q)ROM instantiation). PLUM’s EU-CMA argument under Fiat–Shamir models Griffin as a (quantum) random oracle. The published Griffin analyses cover the original instantiations; whether they extend to the $d = 3$, width-4, 199-bit-field, 14-round instantiation used here is *open*, and the question is upstream of the AIR: were this instantiation (Q)ROM-distinguishable, no amount of AIR engineering would recover security.

BDEC’s Theorems 1 through 3 [1] already prove these properties, so this section does not re-prove them. It bounds how far the move to a zkVM substrate shifts each advantage.

The parties are the teaching authorities and the issuer, who hold signing keys, together with the holder and the relying party. The ledger roots $rt_{cred,e}$, $rt_{TA,e}$, and $rt_{LR,e}$ are public and authenticated. The prover is the holder generating a showing, and the verifier is the relying party, which chooses the presentation predicate ϕ at presentation time. The adversary is the standard anonymous-credential adversary of Section 3, lifted to the zkVM execution setting: it may request issuance of credentials on attribute vectors of its choice, observe showings produced by honest holders, act as a malicious verifier choosing ϕ adaptively, and act as a malicious prover attempting to produce an accepting showing it should not be able to prove. Following PLUM, the unforgeability we inherit is established against classical adversaries in the classical random-oracle model. Its post-quantum reading rests on the underlying power-residue PRF and collision-resistance assumptions being quantum-hard, whereas the lift of the EU-CMA reduction itself to the quantum random-oracle model is a separate question that PLUM does not settle and that we do not re-establish here. That such a lift is *attainable* for this PRF family is shown by Pegasus [27], which proves (Q)ROM security for a power-residue-PRF signature; Pegasus is a different construction (a commit-and-open Σ -protocol, not PLUM’s STIR-based design), so it does not close PLUM’s own lift but it removes any presumption that the PRF family itself is the obstacle.

Side-channel and timing attacks on the prover host, denial of service, and the physical security of signing keys are out of scope, since they are orthogonal to the substrate question this thesis studies. The trusted computing base of the zkVM regime is the RISC-V execution circuit, the precompile chips, and the lookup argument that binds them to the main execution trace.

The preservation theorem of Section 6.2 and the precompile-soundness theorem of Section 5 rest on two open assumptions, Assumptions 6.1 and 6.2, together with two further hypotheses carried inline: the cross-table lookup-argument binding (cited [53]) and the black-box-transfer property (established by PLUM’s reduction structure, below). The lookup-binding hypothesis, that the cross-table lookup argument binds the precompile chips to the execution trace, is assumed from the SP1 and RISC Zero implementations (their logarithmic-derivative lookup arguments [53]), not independently verified here. Assumption 6.1, that the Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation, equivalently that ϵ_{AIR} is negligible, is partially discharged and partially open; its constraint-family breakdown and the test coverage it rests on are given with the discussion after Theorem 6.3, where the assumption is load-bearing. The black-box-transfer hypothesis, that PLUM’s EU-CMA reduction treats Griffin and field multiplication as black-box input–output maps, is established by PLUM’s own analysis: that reduction programs the hash as a random oracle and computes field operations at the value level, never opening their internal structure (PLUM §3.2 [18], whose building blocks are treated as black boxes so the proofs carry over from Loquat [11]). It is distinct from Assumption 6.1, which separately asserts that the precompile computes the reference permutation. Assumption 6.2, that PLUM admits a signature-transcript simulator, equivalently that ϵ_{sZK} is negligible, is open and is required only by the anonymity and unlinkability bounds, PLUM’s analysis establishing EU-CMA security but not simulatability. The zkVM is additionally assumed knowledge-sound and the modular-multiplication precompile sound, the latter established for SP1 (§5.2) and cited for RISC Zero; the quantum-random-oracle lift of PLUM’s EU-CMA reduction and of the nested Fiat–Shamir transform is open, the analysis being in the classical random-oracle model.

6.2 Security Preservation

The analysis rests on one structural observation. The zkVM regime changes how the BDEC relation $R_{\text{show}}^{\text{Sig}}$ (and the structurally analogous credential-generation relation $R_{\text{cre}}^{\text{Sig}}$) is proved; the relation itself, what it states, is unchanged. The relation is fixed in Section 3, and both the static-circuit and the zkVM regimes are proof systems for that same relation. Preservation therefore reduces to two claims: that the substrate computes the same relation, and that its proof system supplies the soundness and zero-knowledge that BDEC’s reductions consume. We treat the two in turn.

Lemma 6.2 (Relation preservation). *Suppose the host modular-multiplication precompile is sound, the cross-table lookup argument binds the precompile tables to the main execution trace, the zkVM is a knowledge-sound argument of knowledge for RISC-V execution, and the Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation. Then the zkVM guest program with the precompile suite active accepts a pair (x, w) if and only if $R^{\text{Sig}}(x; w) = 1$, where R^{Sig} denotes either of the BDEC relations $R_{\text{cre}}^{\text{Sig}}, R_{\text{show}}^{\text{Sig}}$ of Section 3. The lemma is stated at the level of the intended guest specification; the code-level claim additionally requires that the zkVM journal be bound to the public statement x , which the current RISC Zero prototype does not yet implement (Section 5). Absent that binding, an accepting receipt does not certify which statement was evaluated, so the code-level reading of the lemma is conditional on the journal fix.*

Proof sketch. A guest evaluation of the BDEC relations invokes only large-field multiplications and Griffin permutations, which the precompiles return identically under the lookup binding, together with native control flow covered by the zkVM’s knowledge-soundness; composing these per-operation equalities along the finite, input-fixed execution yields equality of the accept-or-reject decision at the program level. The Griffin step’s constraint-level soundness rests on the AIR argument of Appendix B, so the equality holds modulo Assumption 6.1; the 9.4×10^9 -cycle execute-mode run confirms the guest program’s I/O but does not exercise the AIR constraints. The step-by-step argument, including the $k + 2$ presentation predicates, is given in Appendix A. $\square$

Theorem 6.3 (Security preservation, conditional; classical ROM). *Suppose PLUM is EU-CMA secure in the classical random-oracle model and, for the anonymity and unlinkability bounds only, additionally admits a signature-transcript simulator with error ϵ_{sZK} (Assumption 6.2, an open assumption parallel to Assumption 6.1: PLUM’s analysis establishes only EU-CMA, whereas BDEC’s Theorems 2–3 simulate the credential and pseudonym transcripts with the signature’s own simulator). Suppose the zkVM is a knowledge-sound argument of knowledge for RISC-V execution, the host modular-multiplication precompile is sound, and the cross-table lookup argument binds the precompile tables to the trace. Suppose further, as the black-box-transfer hypothesis, that PLUM’s EU-CMA reduction treats Griffin and field multiplication as black-box input–output maps, so that the precompile substitution leaves that reduction unchanged. Suppose further, as the unverified Assumption 6.1, that the Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation; this assumption is argued in Appendix B but not established at the constraint level (see the discussion after the proof). Then for every classical adversary $\mathcal{A}$ against the PLUM-instantiated, zkVM-executed BDEC system, in the random-oracle model, the advantage is bounded by five leaf errors: ϵ_{EUF} (PLUM’s EU-CMA error), ϵ_{KS} (the zkVM argument’s knowledge-soundness error), ϵ_{AIR} (the Griffin-AIR constraint-level soundness error), ϵ_{ZK} (the produced proof’s zero-knowledge error), and ϵ_{sZK} (PLUM’s signature-transcript-simulation error).*

ror):

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{unf}}(\mathcal{A}) \leq \epsilon_{\text{EUF}} + \epsilon_{\text{KS}} + \epsilon_{\text{AIR}}, \quad (3)$$

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{anon}}(\mathcal{A}) \leq \epsilon_{\text{ZK}} + \epsilon_{\text{sZK}}, \quad (4)$$

and the same bound $\epsilon_{\text{ZK}} + \epsilon_{\text{sZK}}$ holds for unlinkability, whose game (BDEC Theorem 3) forwards a single challenge credential, so the one-transcript simulation applies once. Equation (4) bounds anonymity by the zero-knowledge error ϵ_{ZK} of the produced proof. On the target hardware that error is not made small: the succinct receipt has no established zero-knowledge simulator, and the wrap that would supply one is not produced on the stack as shipped, blocked by the recursion-shape provisioning wall (Section 6.3). The anonymity and unlinkability bounds are guarantees the construction inherits from BDEC under a zero-knowledge argument, and whether this substrate supplies that argument is settled negatively in Section 6.3. These three bounds re-instantiate BDEC’s Theorems 1–3 [1] with PLUM as the signature scheme and the zkVM as the zkSNARK, expressed directly in the leaf errors below. The term ϵ_{AIR} is the AIR’s constraint-level soundness error: the probability, over the prover-chosen trace and the argument’s verifier randomness, that the argument accepts a trace whose Griffin transition differs from the reference permutation. Assumption 6.1 is the open, empirically-supported claim (Appendix B) that ϵ_{AIR} is negligible. ϵ_{KS} is the knowledge-soundness error of the zkVM argument, ϵ_{EUF} is the EU-CMA error of PLUM ([18, Theorem 1], carried over from Loquat’s reduction [11]), ϵ_{ZK} is the zero-knowledge error of the produced proof, and ϵ_{sZK} is the signature-transcript simulation error of PLUM (Assumption 6.2), required because BDEC’s anonymity and unlinkability reductions simulate the credential and pseudonym transcripts with the signature’s own simulator (Theorems 2–3 [1]), not only the proof simulator.

Proof sketch. By Lemma 6.2 the zkVM proves the same relation as BDEC’s static circuit, so we re-instantiate BDEC’s three reductions with PLUM as the signature scheme and the zkVM as the zkSNARK; all bounds are for classical $\mathcal{A}$ in the random-oracle model. For *unforgeability*, the zkVM knowledge extractor recovers a witness from an accepting showing (cost ϵ_{KS}); a malformed Griffin trace that passes the precompile costs ϵ_{AIR} (Assumption 6.1); the recovered signature is then a PLUM EU-CMA forgery (cost ϵ_{EUF}), giving (3). For *anonymity* and *unlinkability*, a simulator replaces the credential and pseudonym transcripts by PLUM’s signature-transcript simulator (cost ϵ_{sZK} , Assumption 6.2) and the proof by the zkVM’s zero-knowledge (cost ϵ_{ZK}); no witness is extracted, so ϵ_{KS} and ϵ_{AIR} do not enter, giving (4) and, for the single-credential unlinkability game, the same bound. Because BDEC proves its three theorems for a generic (signature, zkSNARK) pair [1], substituting PLUM and the zkVM changes nothing beyond these leaf errors. These guarantees are inherited in principle; Section 6.3 shows that the produced receipt mode has no established zero-knowledge simulator, and that the ZK-wrapped mode is not produced on the target hardware as the toolchain ships, blocked by the recursion-shape provisioning wall, so anonymity and unlinkability are not delivered there. The step-by-step reductions, with the explicit adversary constructions and hybrids, are given in Appendix A. $\square$

The hypotheses of Theorem 6.3 are not all equally settled, and the honest reading of the theorem turns on which are established and which remain open. PLUM’s EU-CMA security is taken from its specification and the Loquat analysis it builds on [11], which establish it in the classical random-oracle model against classical adversaries; the lift of that reduction to the quantum random-oracle model is open, so the unforgeability bound holds against classical adversaries and its quantum case

inherits that open lift. The soundness of the modular-multiplication precompile is established for SP1 by the modular-multiplication reduction of §5.2 and is cited for RISC Zero from its production release, while the knowledge-soundness hypothesis is obtained by applying the STARK and BCS soundness analyses [35] to the two systems’ concrete arguments; the cross-table lookup-argument binding (the lookup-binding hypothesis [53]) is instead assumed from the SP1 and RISC Zero implementations of their logarithmic-derivative lookup arguments, whose published soundness analyses we do not re-establish here. These last are themselves obtained through the Fiat–Shamir compilation [54], whose soundness is a random-oracle property invoked twice and nested, PLUM inside the zkVM, and whose quantum case is again open.

The one hypothesis below the level of an established or cited result is Assumption 6.1, the Griffin AIR’s constraint-level soundness. The chip’s compute matches the reference permutation on the 256-vector equivalence suite, with per-vector false-pass at most $1/p \approx 2^{-199}$ and aggregate at most 2^{-191} under a uniform-wrong-output model (Appendix B; a deterministic bug agreeing on the sampled inputs is not bounded by these figures). This figure is the test *coverage* of a positive-direction equivalence suite, run on valid inputs, and is not a soundness bound: it does not show that the constraint system *rejects* a malformed Griffin trace, which is the direction Assumption 6.1 actually asserts. The equality of the constraint system with the reference is argued rather than machine-checked in Appendix B. Two further points require disclosure. The implementation fixes a concrete 199-bit prime $p = 2^{64}p_0 + 1$, so any security-bit claim that depends on the exact value of p is conditional on this instantiation matching PLUM’s intended parameters. And whether Griffin at the instantiation used here, a degree-3 S-box over a width-4 state of the 199-bit field with 14 rounds, may be replaced by a random oracle in PLUM’s EU-CMA argument is a question the AIR cannot settle and that we carry as open. The reduction of Theorem 6.3 is therefore sound *modulo its hypotheses*, but three of its inputs are not yet closed: the constraint-level AIR equality (Assumption 6.1), PLUM’s signature-transcript simulatability (Assumption 6.2), and the quantum lifts of PLUM and of Fiat–Shamir. The black-box-transfer property is established separately by PLUM’s reduction structure, which programs the hash as a random oracle and never opens its internals (PLUM §3.2). We state the theorem conditionally on them rather than assert guarantees the cited proofs do not establish. The conservative reading is explicit: absent a PLUM transcript simulator (Assumption 6.2), only the EU-CMA-derived *unforgeability* of the credential transfers, and anonymity and unlinkability are conditional on Assumption 6.2; absent the quantum lift, every guarantee holds against classical adversaries only. For each open premise we therefore state which guarantee survives it, and we make no unconditional claim.

A sharper disclosure is owed for the Griffin round count itself, since the hash makes up roughly 91% of the verification’s R1CS constraints (the PLUM-128 figure of Section 4). Neither PLUM nor Loquat prints a Griffin round count. The 14 rounds are our own instantiation, derived by the original Griffin Gröbner-resistance methodology at our parameters as $\lceil 1.2 \times 11 \rceil$, a 20% margin over the floor of 11 at which a constrained-input/constrained-output (CICO) solver of the FGLM type would cost 2^{64} for our ($d=3$, $t=4$, $\lambda=128$) instance over the 199-bit prime. That methodology is no longer the state of the art. The FreeLunch attack [55] solves the CICO problem below the FGLM model from which the count is derived, and the Improved Resultant Attack [56] reports that most variants of Griffin fail to reach their claimed security level, with CICO solutions found in practice for 8 of 10 rounds of a Griffin instance (7 for FreeLunch; the line continues in [57], which does not by itself threaten a full-round instance). The instances broken in practice use an $\alpha = 3$ S-box, the same degree as our instantiation, and were solved at 7–8 of their 10 rounds on reduced or smaller-field parameters. Our 14-round, width-4, 199-bit

instance is not among them, separated from the broken ones by its round count and field rather than by its S-box degree. We therefore do not claim the 14-round count establishes a 128-bit floor against the current attack frontier. Establishing such a floor would require instantiating the FreeLunch/resultant complexity estimator at our parameters ($r=14$, $d=3$, $t=4$, $\log_2 p \approx 199$) and re-deriving the count from that model, which we record as required future work. This is not independent of Assumption 6.2: a reduced-round CICO solution is precisely the non-random structure that replacing Griffin by a random oracle assumes absent, so the algebraic-attack question and Assumption 6.2 are two readings of one open problem.

The BDEC credential-generation prove anomaly reported in Section 7 does not bear on Assumption 6.1: that run executed on RISC Zero 3.0.5 with field arithmetic carried by `sys_bigint`, a path on which the bespoke Griffin- $\mathbb{F}_p^{192}$ AIR is not invoked (Section 5). The anomaly localises to the segment prover’s own integrity self-check, and the composite re-run disfavors any content-deterministic cause including a `sys_bigint` trace malformation (§7.6.1); the chip of Assumption 6.1 was not exercised by that run in any case. The genuinely dangerous case for Assumption 6.1 is silent *acceptance* of an invalid Griffin trace by the AIR. The equivalence suite exercises only valid inputs, and a separate fault-injection suite tests the rejection direction for seven of the ten constraint families (the per-row families) via `debug_constraints`: single-cell corruptions of a satisfying trace are rejected in each case (Appendix B). The lookup-borne families (round count, memory binding, and the cross-row threading of row scheduling) need a full-prover harness and remain untested; parameter immutability is argued by construction. Resolving Assumption 6.1 fully, by extending the fault-injection suite to the lookup-borne families, by metamorphic testing, or by constraint-level verification of the chip against the reference permutation (tractable because SP1’s prover is built on Plonky3), is left as future work; the metamorphic and fault-injection approach is that of Arguzz [45], which targets zkVM soundness and completeness bugs by that technique.

One structural point about Assumption 6.1 governs how the measurements of Section 7 should be read. Only the Griffin-precompile arms exercise the chip at all: the precompile-free baseline, the SHA-3 control arm, the RISC Zero arm (which completed in 6 h 19 m without invoking the chip), and the Aurora legs never touch it, so their figures carry no dependence on Assumption 6.1. Where the chip is exercised, every run does so on the honest path: the guest supplies a valid Griffin trace and the prover pays the cost of the constraint system as built. If that system admitted a non-reference trace, the defect would not have made any measured run cheaper; for any repair that strengthens the constraint set, the measured cost is then a cost floor for the corrected chip. Assumption 6.1 is therefore a premise of the positive results alone, namely the 14.25-minute feasibility figure, the 13.0-second single-permutation smoke proof, and the unforgeability bound of Theorem 6.3. The negative findings either do not exercise the chip at all or hold a fortiori under an unsound AIR: the zero-knowledge wrap’s failure at the recursion-shape provisioning wall and the SHA-3 control’s faster wall-clock could only widen against a corrected and at-least-as-costly chip.

6.3 The Zero-Knowledge Barrier as a Finding

BDEC’s anonymity and unlinkability require the underlying argument to be zero-knowledge. The succinct STARK receipt produced by either zkVM is, in the configurations we run, an argument of knowledge but not a zero-knowledge one, and this holds on both substrates for separately documented reasons. SP1’s specification states plainly that its STARK proofs are not zero-knowledge [58].

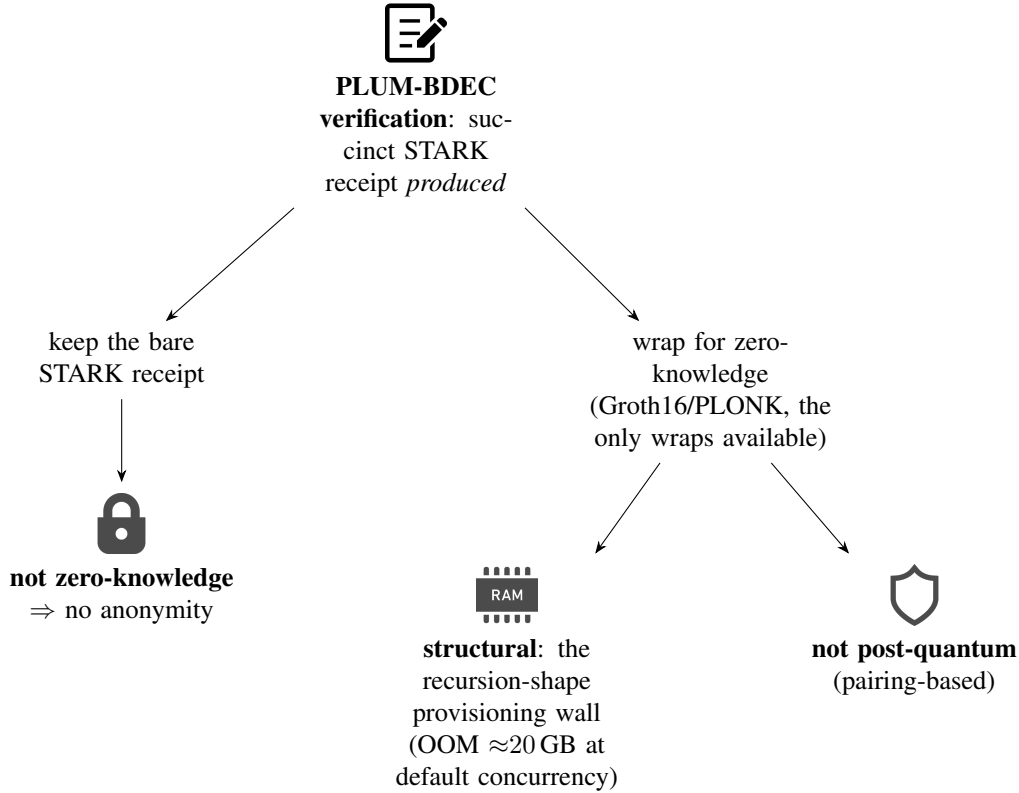


Figure 3: The dual obstruction. The proof feasible to generate (left, the bare STARK receipt) is not zero-knowledge, hence not anonymous; the proof that would be zero-knowledge (right, a Groth16/PLONK wrap) is blocked, at $\lambda = 80$ on the 24 GB target hardware, by two *independent* prongs: structural (the recursion-shape provisioning wall, with OOM at ≈ 20 GB the default-concurrency symptom) and primitive (pairing-based, not post-quantum, which more memory cannot fix).

RISC Zero documents perfect zero-knowledgeness for its STARK, but independent analysis by Haböck and Al Kindi, together with RISC Zero’s own security advisory, finds that several STARK implementations, RISC Zero among them, do not provably meet the zero-knowledge requirement [2, 3]. We therefore treat the bare receipt as not dependably zero-knowledge on either substrate, a concrete instance of the caution that a system may be called a zkVM without delivering zero-knowledge. Zero-knowledge is obtained only by wrapping the receipt in a succinct proof, and the wraps SP1 exposes, Groth16 and PLONK, are pairing-based and therefore only classically secure. RISC Zero’s wrap inventory is no wider: its Groth16 prover is likewise pairing-based and, per the vendor’s documentation, runs only on x86, so on the Apple-silicon target hardware it is unavailable altogether [16]. The barrier splits into two independent obstructions, set out below. Both block anonymity for separate reasons, and naming each one precisely is the security-level result of this analysis.

The first obstruction is the recursion-shape provisioning wall, structural rather than a memory limit. At SP1’s default prover concurrency the PLONK wrap is killed by system memory pressure (jetsam) at $\text{RSS} \approx 20$ GB on the 24 GB machine (`docs/measurements/zkwrap_worker_concurrency_20260627`) but lowering the recursion-worker count runs the same wrap under ≈ 17 GB, well inside the budget

(docs/measurements/zkwrap_default_shard_20260627), where it fails earlier instead, at the recursive-compression step, because SP1’s recursion shapes are provisioned for smaller relations than PLUM verification (Section 7). Bringing the wrap within reach is a one-time regeneration of those static artifacts, roughly two days of compute on this hardware (≈ 51 h; the measured per-setup rate, envelope, and run record are reported in Section 7), rather than more memory. No single stage of the wrap exceeds the 24 GB budget: PLUM’s core-and-recursion proving peaks at ≈ 17 GB and the program-independent gnark PLONK wrap stage at 15.4 GB (Section 7). The barrier is therefore the regeneration cost in compute time, which more memory does not remove, and the regenerated toolchain is not run end-to-end here. The anonymity-providing proof is thus not generated on the stack as shipped, even though the bare, non-anonymous receipt is. The second obstruction is one of primitives. Even with unlimited memory, wrapping a post-quantum STARK receipt in a pairing-based Groth16 or PLONK proof re-introduces a quantum-vulnerable assumption into the anonymity guarantee, the very class of assumption the post-quantum motivation rules out (Section 2). Such a wrap remains acceptable for a classically-secure deployment, and Groth16 and PLONK are not unsuitable in general. They are ruled out here only by the thesis’s post-quantum-anonymity goal. Recovering post-quantum anonymity requires a post-quantum-sound zero-knowledge wrap, for instance a STARK-based or lattice-based recursive argument, which the deployed toolchains do not yet provide.

The two obstructions differ in kind. The first is a one-time provisioning cost: each stage of the wrap fits within 24 GB (Section 7), and regenerating the static recursion artifacts, on the order of two days of compute, would in principle clear it. The second is a primitive limit within the post-quantum goal, since a pairing-based wrap is not post-quantum at any memory or time budget. A deployer can pay down the first; the second bounds what this stack can deliver.

We therefore use the classically secure Groth16 or PLONK wrap to locate the recursion-shape provisioning wall. It is not a deployable post-quantum-anonymous configuration. The two obstructions together leave no good option on the zkVM stack on commodity hardware: the proof that is feasible to generate is not anonymous, and the proof that would be anonymous is neither feasible to generate nor, with the available wraps, post-quantum, whereas a zero-knowledge-enabled Aurora run on the same machine proves the Loquat single-verification building block in about 22 minutes (mean of three runs, 21.4–22.9) (docs/measurements/audit5_campaign_20260612). Within the scope of the literature we surveyed, we did not find this conjunction stated explicitly for post-quantum anonymous credentials, and surfacing it is a contribution of this thesis: we quantify the structural prong, the recursion-shape provisioning wall of Section 7, and identify the primitive prong as a toolchain gap.

The barrier is scoped to the substrate and hardware studied: an Apple M5 Pro with 24 GB of memory, the CPU-bound provers of RISC Zero and SP1, and the parameter set $\lambda = 80$. We do not claim the barrier is fundamental to the primitive or that it persists on server-class hardware. Characterising how it moves with the memory budget and the security level is left as future work.

What would move this barrier can be named as three levers. Reaching post-quantum zero-knowledge for this workload requires (i) a post-quantum-sound zero-knowledge wrap in place of the pairing-based ones the toolchains currently expose, for which two hash-based routes already exist as code (masked FRI and the VEIL compiler; Section 9); (ii) that this wrap prove within the memory budget, the resource question this thesis leaves open; and, for everlasting in place of merely computational anonymity, (iii) a non-interactive, standard-model, statistically-hiding commitment to the witness-bearing trace (Proposition 6.5). Levers (i) and (ii) are engineering questions, the techniques for (i) already implemented and (ii) reducing to whether the masked prover fits the budget; lever (iii) is an

open construction problem.

A separate question is the strength of the anonymity guarantee where the argument is zero-knowledge. Two points bound it. The first concerns quantum adversaries. The bound of Equation (4) is stated in the classical random-oracle model, so it constrains a classical distinguisher. Lifting it to the quantum random-oracle model is attainable in principle: the compilation of a round-by-round sound interactive oracle proof into a non-interactive argument is post-quantum secure in the quantum random-oracle model [59], and this has been established for the STIR-class proof systems PLUM uses [60]. The lift is not settled here. It remains conditional on a PLUM transcript simulator (Assumption 6.2) and on a post-quantum analysis of PLUM’s own Fiat–Shamir step, which the scheme’s published analysis does not provide. We therefore claim computational anonymity against a classical distinguisher and record the quantum lift as attainable but open.

Remark 6.4 (The anonymity guarantee is computational). The guarantee is computational, and it is not everlasting: a recorded transcript hides the witness only while the underlying commitment stays hiding. The credential relation proves a hash commitment $c = H(w \parallel r)$ to the witness inside the circuit, so the witness enters the execution trace, whose commitment is an algebraic-hash Merkle root. A salted hash leaf hides its input statistically only in the random-oracle model. Once H is a fixed function the hiding rests on a computational assumption on that function, and the random-oracle methodology gives no assurance that the hiding survives the instantiation [61]. An adversary who stores a transcript and later acquires quantum computing power attacks the instantiated function, against which the hiding is at best computational. Recovering everlasting anonymity would require the trace commitment to be statistically hiding in the standard model and to be non-interactive, since the receipt is a single message. A non-interactive statistically-hiding commitment is not available from general assumptions: any black-box construction from one-way permutations needs $\Omega(n/\log n)$ rounds of interaction [62, 63]. Statistical hiding and statistical binding also cannot hold together, so binding must remain computational. The everlasting variant is therefore an open construction problem.

Proposition 6.5 (Conditional everlasting anonymity). *Suppose there is a non-interactive, post-quantum, statistically-hiding commitment to the witness columns of the trace that composes with the low-degree test, so that the witness is committed before it enters the algebraic-hash root and the proximity test does not re-expose it. Then, in a masked variant of the proof and under the hypotheses of Theorem 6.3 and Assumption 6.2, the zkVM-executed BDEC system is everlastingly anonymous and unlinkable: against an unbounded distinguisher the advantage is at most $\epsilon_{\text{ZK}}^{\text{stat}} + \epsilon_{\text{sZK}}$, where $\epsilon_{\text{ZK}}^{\text{stat}}$ is statistical. Binding, and hence unforgeability, remains computational.*

Proof sketch. The anonymity argument of the preservation theorem is unchanged except at the step that hides the witness in the trace commitment. In the construction analysed that step is computational, because the witness columns sit under a collision-resistant Merkle root (Remark 6.4). Under the hypothesised commitment the witness columns are statistically hidden before they reach the root, so the root carries no statistical information about the witness, and the low-degree test operates on the committed columns without re-exposing it. The masked interactive oracle proof is statistically honest-verifier zero-knowledge by randomisation, independently of the hash [64], so the transition incurs only the statistical distance $\epsilon_{\text{ZK}}^{\text{stat}}$, and the transcript-simulation step is unchanged at cost ϵ_{sZK} . An unbounded distinguisher therefore gains at most $\epsilon_{\text{ZK}}^{\text{stat}} + \epsilon_{\text{sZK}}$. Binding stays computational, since a commitment cannot be both statistically hiding and statistically binding, so the unforgeability bound is unaffected. $\square$

A concrete candidate for the hypothesised commitment is the Module-SIS-based commitment of Baum, Damgård, Lyubashevsky, Oechsner and Peikert [65], which is statistically hiding in a parameter regime that trades off against binding. It is post-quantum, but its proof of opening is interactive, so it does not by itself meet the non-interactive requirement above. Closing that gap is the open problem behind Proposition 6.5.

7 Evaluation

This section reports what the precompile-enabled zkVM regime can and cannot do for PLUM-based BDEC on consumer hardware, and uses the measurements to revisit the cost framework of Section 4. Consistent with the rest of the thesis, the contribution is the *quantification*: within the scope of the literature we surveyed, these are among the first reported figures for executing and proving a PLUM-instantiated post-quantum anonymous-credential workload inside a general-purpose zkVM, and we report them honestly, including the configurations that do not complete.

7.1 Measurement Methodology

The evaluation addresses four questions: whether PLUM verification, and the BDEC credential relation built on it, run inside the zkVM on the target hardware, with and without the precompile suite; where the zkVM cost falls and how much of it the precompile suite removes; for the configurations that do not complete, what the binding resource is and where the boundary lies; and how the two regimes compare on the deployment-lifecycle dimension of Definition 4.6.

Two measurement modes, and when each is reportable. A zkVM runs in two modes relevant here. *Execute mode* runs the guest and counts RISC-V *cycles* without producing a proof; its cost is machine-independent and terminates within the guest’s memory space. *Prove mode* produces the cryptographic receipt and is the deployment-relevant cost, but is orders of magnitude heavier and, as reported below, does not always complete on the target hardware. Our rule is therefore: *report prove wall-clock where proving completes, and fall back to execute-mode cycle counts (machine-independent and reproducible) where it does not*. Cycle counts and wall-clock times are never compared across this boundary as if commensurable; each table states its mode. Throughout this section, *feasible* means that the prove completes on the target machine within its 24 GB memory envelope; whether a completed time is acceptable for a given deployment is a separate judgment, treated by the decision framework of Section 8.

What we deliberately do not claim. We do not report a single “ $N\times$ speedup” headline. As §7.3 makes explicit, the with/without-precompile contrast at the prove level is an *infeasible-to-feasible* transition rather than a ratio of two finite times, and the finite reductions we do report are cycle-level reductions at a stated security level and mode. Conflating these into one multiplier would produce the kind of setting-dependent figure we avoid.

7.2 Experimental Setup

7.2.1 Settings Justification

A measurement is only persuasive if its settings are ones a deployer would plausibly choose, rather than ones selected to flatter a number. We state and justify each fixed parameter, and disclose where a setting is tuned.

Component	Specification
Hardware	Apple M5 Pro, 18-core CPU, 20-core GPU, 24 GB unified memory, 1 TB SSD
OS	macOS 26.5 (Darwin 25.5.0), Apple Silicon (arm64)
Prover	CPU-bound; no CUDA/network prover (Apple Silicon has no NVIDIA GPU path)
zkVMs	SP1 v6.2.1 (forked), RISC Zero v3.0.x
Signature	PLUM over $\mathbb{F}_p$, $p = 2^{64}p_0 + 1$ (199-bit; see §5.2)
Security levels	$\lambda = 80$ (prove-mode runs), $\lambda = 128$ (execute-mode cycle attribution)

Table 1: Evaluation platform. All prove-mode measurements are local single-machine runs with no network or dev-mode acceleration unless stated.

Consumer hardware (M5 Pro, 24 GB). The research question concerns deployability on a personal machine, so this hardware is the thesis’s explicit scope. Every feasibility claim is scoped to this envelope and is not asserted in general.

Security level: $\lambda = 80$ for prove-mode, $\lambda = 128$ for cycle attribution. We disclose this seam rather than hide it. Prove-mode runs use $\lambda = 80$ because $\lambda = 128$ proving is untested and expected to exceed the memory budget, the precompile-free arm already failing at $\lambda = 80$ (§7.6); $\lambda = 80$ is the largest level at which prove-mode wall-clock is observable on this hardware, which is why we use it and not because it is favourable. The cost-attribution analysis (§7.5) is reported at the audience-relevant $\lambda = 128$ because it is machine-independent (execute mode) and so remains measurable where proving is not. No number mixes the two levels silently; each table states λ .

Prover memory tuning. The base prove-mode runs used a documented reduced-memory configuration. At SP1’s default settings the Cell 2 prove is OOM-killed after about 3 minutes on the 24 GB machine and yields no figure at all; the tuning (`SHARD_SIZE = 222`, default `224`; `ELEMENT_THRESHOLD = 226`; `HEIGHT_THRESHOLD = 220`; `TRACE_CHUNK_SLOTS = 2`; `RAYON_NUM_THREADS = 8`, default 18; §7.2.2) trades more, smaller shards and fewer concurrent threads for fitting the 24 GB envelope, holding peak memory near 81%. The headline timings are therefore tuned-configuration times: Cell 2 completes in 14.25 min and the SHA-3 control in 13.05 min under identical tuning. The *zero-knowledge wrap* attempts ran under the same class of reduced configuration and still exhausted memory (§7.6.2). A contributing factor to the Griffin arm’s memory pressure surfaced during cost-model calibration: the deployed fork’s shard-budgeting cost table prices the Griffin chip at 348 trace columns, whereas the chip’s compiled width is 3,819, so the executor budgets a Griffin-bearing shard at roughly one tenth of its real committed area and splits shards later than the trace-area threshold intends. Correcting the table entry is a one-line change whose effect on Cell 2’s peak memory and wall-clock is untested and left to future work.

Micro-benchmark isolation. Cost-attribution runs isolate single primitives (one Griffin permutation; one verification) so that per-operation cost can be attributed rather than inferred from an aggregate.

7.2.2 Reproducibility

Prove-mode and attribution runs use fixed RNG seeds and fixed messages; the SP1/RISC Zero submodule versions are pinned in the lockfile; the memory-tuning environment variables, host binaries, and the adversarial-probe harness are recorded in the project’s measurement notes, together with the per-run commit hashes and, where applicable, guest-image identifiers.

7.3 PLUM Verification With and Without the Precompile

Table 2 reports prove-mode results for a single PLUM signature verification at $\lambda = 80$. The central comparison is between the precompile-free baseline and the precompile-enabled arm on the same hardware.

zkVM / mechanism	λ	Prove time	Verify time	Peak mem	Outcome
SP1, rv32im Griffin (no precompile)	80	n/a	n/a	87.94%	DNF (OOM $\approx 1\text{m}45\text{s}$)
SP1, Griffin-$\mathbb{F}_p^{192}$ precompile (this work)	80	14.25 min[†]	3.06 s	59.2%	receipt verified
SP1, SHA-3 hasher (control)	80	13.05 min	2.80 s	59.8%	receipt verified
RISC Zero, <code>sys_bigint</code> , SHA-3 hasher	80	6 h 19 m	n/a [‡]	n/a	receipt verified

Table 2: PLUM single-signature verification, prove mode, $\lambda = 80$, M5 Pro 24 GB. We refer to the three SP1 arms as *Cell 1* (no precompile), *Cell 2* (Griffin- $\mathbb{F}_p^{192}$ precompile), and *Cell 3* (software SHA-3 control) throughout. The precompile-free baseline does not complete; the precompile arm completes in 14.25 min. The SHA-3 arm is a *control* (PLUM with a standard hash computed in software, no hash precompile, only the shared `UINT256_MUL` field-arithmetic precompile active), not a deployable PLUM variant. The Cell 2 and Cell 3 prove times are means of three fixed-config runs on a clean machine (2026-06-30, §7.3); a 2026-06-13 same-session pair gave Cell 2 17.81 min and Cell 3 16.15 min ($n = 2$), and a single 2026-05 run measured Cell 2 32.53 min, the high figure attributed to machine-state contamination and not reproduced under replication. [†]Cell 2 prove time is the mean of three runs, 14.25 min (range 14.05–14.61, ≈ 14.2 GB peak RSS). [‡]RISC Zero receipt-verify wall-clock not separately logged; the receipt verified.

A re-run of the precompile-free arm on 2026-06-12, under the documented reduced-memory tuning (§7.2.1), ran for 5 h 07 m without terminating and without exhausting memory before we stopped it by hand, yielding no receipt (run record in `docs/measurements/audit5_campaign_20260612`). Under that tuning the binding resource is time rather than memory, and the prove still does not complete within five hours on this hardware.

The honest reading. The with/without-precompile contrast is not a ratio of two finite times: without the precompile the workload *does not complete* on this hardware, and with it the workload completes in 14.25 minutes (mean of three runs, 14.05–14.61). The defensible claim is therefore qualitative at the prove level (*the precompile moves PLUM verification from infeasible to feasible on consumer hardware*) and quantitative only at the cycle level (§7.5). The two zkVMs are not directly comparable here: the RISC Zero leg is the SHA-3-hasher arm, carrying the field arithmetic through `sys_bigint` without a native Griffin chip (§5.3), so its like-for-like SP1 counterpart is the 13.05-min SHA-3 control,

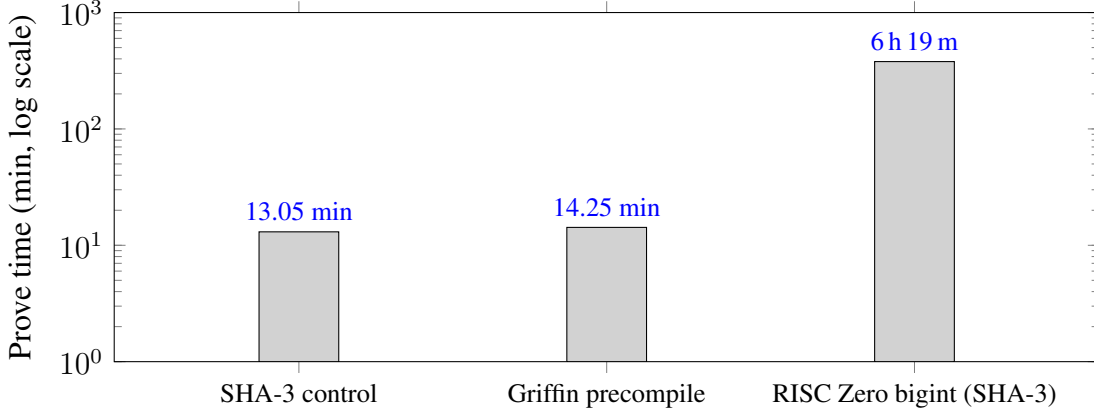


Figure 4: Prove time on the configurations that complete ($\lambda = 80$, M5 Pro 24 GB, log scale). The bespoke Griffin precompile (14.25 min) is slower than the SHA-3 control (13.05 min) at matched security, and RISC Zero’s `sys_bigint` route (also SHA-3-hashed) is slower again (6 h 19 m); these are different hash and substrate mechanisms, so the bars are not one artefact. The precompile-free baseline does not complete (DNF, OOM at $\approx 1m45s$) and so has no bar.

with the residual same-hash gap reflecting the substrate and receipt kind; no Griffin-hashed single-verification prove was run on RISC Zero (the Griffin AIR port is future work), and the gaps should not be read as a like-for-like substrate ranking. The SHA-3 control isolates the hash family: the verification harness is identical except that every Griffin- $\mathbb{F}_p^{192}$ sponge call is replaced by a SHA3-256 call computed in software, so that the SHA-3 arm fires *no* hash precompile and only the shared `UINT256_MUL` field-multiplication precompile remains active. It shows that a workload using a *standard* hash in software is markedly cheaper than one using the algebraic hash PLUM requires through a bespoke chip, which is the quantitative form of the arithmetic-mismatch argument of Section 4. The inversion is direct: the bespoke Griffin- $\mathbb{F}_p^{192}$ precompile proves *slower* at prove than the software SHA-3 control at matched $\lambda = 80$ in every run we took. The sign is robust. At a fixed tuned config on a clean machine, three Cell 2 runs and three Cell 3 runs (2026-06-30) give means of 14.25 min (range 14.05–14.61) and 13.05 min (range 12.93–13.16), a magnitude of $\approx 1.09\times$ in wall-clock and $1.13\times$ in cycles, with each arm’s prove-time spread under 5% (run records `docs/measurements/tier2_replication_20260629`). A same-session 2026-06-13 pair corroborates the sign and the magnitude at 17.81 versus 16.15 min ($1.10\times$). An earlier single 2026-05 run measured 32.53 versus 13.28 min; its high Cell 2 figure was not reproduced under fixed-config replication and is attributed to machine-state contamination rather than an intrinsic build difference, so it is retired in favour of the replicated 14.25 min. We read the inversion as a genuine property of the pipeline rather than a measurement artefact, and the trace-area accounting of Section 4 (Equation (1)) postdicts its sign from the chips’ committed dimensions. The Griffin chip emits 14 rows of 3,819 columns per permutation (each row carries 26 modular-arithmetic gadgets of 127 cells each, beside range checks and state limbs), so the 1,052 permutations of one verification commit $14 \times 3,819 \times 1,052 \approx 5.6 \times 10^7$ cells, plus one controller row per permutation, that the SHA-3 arm never pays. The shared `UINT256_MUL` chip is common-mode (69,396 versus 69,385 events) and cancels, and the Griffin arm additionally executes 13% more guest cycles (1.26×10^8 versus 1.11×10^8 , execute mode). Both terms of Equation (1) are therefore larger on the Griffin arm, so the model places it behind the control for every non-negative choice of the per-cell rates, granted a

common per-cycle core area across the two arms; the calibration of the model’s constants on these two arms is reported with the cost model (§7.7). The feasibility recovery and the inversion are the same inequality (Proposition 4.4, Equation (2)) evaluated at two operating points. The precompile bought a finite measurement, and the accounting prices why a speedup over the control was never available: against a control whose hash costs the substrate no chip area at all, the Griffin arm pays the chip area and recovers nothing.

Proof size, and why we do not re-measure it. Receipt size in SP1 is a property of the proof *type*, not of the workload. The runs above use SP1’s default *core* proof, whose size *scales with execution size* [66]; the compact, on-chain-deployable proof is instead the constant-size SNARK *wrap* of that receipt, about 260 bytes for Groth16 over BN254, or 868 bytes for PLONK [66]. Producing that wrap is exactly the step that does not complete on the target hardware as the toolchain ships. The failure is not a want of memory; it is that SP1’s recursion shapes are provisioned for smaller relations than PLUM verification, the recursion-shape provisioning wall of §7.6.2. On a 24 GB machine, then, the proof a deployer can actually generate is the large, shard-scaling core receipt; the 260-byte SNARK is a documented size the frontier of §7.6.2 prevents us from realising here. We therefore report the deployable figure as documented and do not serialise the workload-dependent core receipt, whose byte count is not the deployment-relevant number.

7.4 The Circuit-SNARK Reference Point

To situate the zkVM costs against the static-circuit regime they are meant to escape, we measured the Aurora prover end-to-end on the same hardware over a single Loquat-verification R1CS. This is the single-verification building block the BDEC construction composes [1]: the full credential relation merges two such verifications under a shared hidden public key, so it is roughly twice this figure, and we did not run the composed relation statically. We measure it directly rather than cite it, so that the comparison is on our hardware rather than across machines.

Quantity	Value
Constraints (padded)	524,288
Prover time (setup + prove)	225.4 s (3.76 min; 223.2–226.6)
Verifier time	41.3 s (41.0–41.6)
Proof size	704,416 B (688 KiB, identical across runs)
Achieved security	107.5 bits (target 128)
Zero-knowledge	disabled

Table 3: Aurora over a single Loquat-verification R1CS, M5 Pro 24 GB; mean of three solo runs with the per-run range in parentheses (run records in docs/measurements/pub_hardening_20260610). This is the *Loquat*-instantiated circuit over $\mathbb{F}_{p^2}$ (127-bit base) [1] at single-verification scope; the full BDEC relation is two such verifications ($\approx$ twice this), not run statically. Prover time includes Aurora parameter setup but is not a recompile-time measurement (§7.8). Achieved security 107.5 bits, below the 128 target. Verification ACCEPT in every run.

The zero-knowledge-enabled run. The runs of Table 3 have zero knowledge disabled. We additionally ran the same 2^{19} R1CS (524,288 constraints, RS extra dimensions 5) with zero knowledge *enabled*, three solo runs: the prover takes 1317.7 s at the mean (21.96 min; 1285.7–1376.1 s), the verifier 40.8–41.8 s, and the proof is 876,768 B (856 KiB), identical across the three runs, against 704,416 B with zk disabled; achieved security is 107.5 bits (target 128, as in the zk-disabled legs), peak RSS spans 9.2–11.0 GiB, comfortably inside the 24 GB envelope, and the verifier returns ACCEPT in every run (run records in docs/measurements/pub_hardening_20260612 and audit5_campaign_20260612). Against the zk-disabled prover mean of 225.4 s, the measured zero-knowledge premium on this circuit is about $5.8\times$ prover time at the means.

A same-scheme substrate comparison. To remove the scheme confound, we ran the *same* Loquat verification through the zkVM, at the same parameters. The result is a clean single-axis comparison (Table 4): for one Loquat verification at $\lambda = 80$, with neither side zero-knowledge, the static Aurora circuit proves in 3.76 min against 44–64 min for the SP1 zkVM (51.9 min at the mean of three runs; the spread on an otherwise idle machine is thermal, run 2 starting on a package heated by run 1). Same scheme, same $\mathbb{F}_{p^2}$ (127-bit) field, same unit, same security setting; only the substrate differs. The zkVM is the dearer per-proof substrate by $12\text{--}17\times$ ($13.8\times$ at the means), the gap being the cost of executing Loquat’s field arithmetic as emulated RISC-V over KoalaBear rather than as a native constraint system. The zkVM leg uses no Loquat-specific precompile (none exists for the 127-bit field), so this is the substrate’s emulated cost, which is precisely what adopting a general-purpose zkVM both buys and pays for. The two substrates are measured at different rates, the zkVM arm at $|U| = 4096$ and the Aurora arm at $|U| = 256$, so the ratio indicates direction rather than a calibrated factor. The instance uses the paper-faithful rate after the ρ^* fix but keeps L and B reduced below the paper’s $\lambda = 128$ set, so the timing is not fully security-calibrated.

Substrate	Prove time	Proof size	ZK
Aurora static circuit (native $\mathbb{F}_{p^2}$)	3.76 min	688 KiB	no
SP1 zkVM ($\mathbb{F}_{p^2}$ emulated on KoalaBear)	44–64 min	405 MiB	no

Table 4: Same-scheme substrate comparison: one Loquat verification, $\lambda = 80$, M5 Pro 24 GB, three solo runs per leg, neither zero-knowledge. The Aurora leg is the mean of Table 3; the SP1 leg is the range of three runs (44.4, 47.4, 64.0 min) under the documented reduced-memory configuration (§7.2.1), succinct core STARK, peak RSS 14.1–15.4 GiB, execute-mode cost 397,227,544 cycles (run records in docs/measurements/pub_hardening_20260610). Only the substrate differs (native R1CS versus zkVM emulation); the $12\text{--}17\times$ prover-time gap is the cost of the zkVM substrate for this scheme. This isolates the substrate for *Loquat*; it does not transfer to PLUM, which has no 127-bit Aurora harness.

What this reference does and does not license. Two readings of this point are legitimate and one is not. *Legitimate*: on this hardware the static-circuit regime proves a single Loquat verification in minutes, against the zkVM’s tens-of-minutes-to-infeasible (Tables 2, 6), so for a *fixed* predicate the static circuit is the faster prover, as one would expect from a specialised encoding. *Also legitimate*: the zk-disabled instance of Table 3, like the zkVM succinct receipt, does not by itself discharge BDEC’s

anonymity requirement; the zero-knowledge-enabled run reported above supplies that property for the same circuit, at the measured premium. *Not licensed:* reading Table 3 *against the PLUM zkVM numbers* (Tables 2, 6) as a same-scheme verdict, since that pairs Loquat over a 127-bit field with PLUM over a 199-bit field. The same-scheme verdict is instead the Loquat-versus-Loquat comparison of Table 4; either reading is timed without the predicate-change cost that is the separate subject of the update-churn comparison (§7.8).

7.5 Cost Attribution

To attribute the cost we use execute-mode cycle counts at $\lambda = 128$, the audience-relevant security level, where prove-mode is infeasible but cycle counts remain machine-independent. Table 5 reports the cumulative effect of routing PLUM’s field arithmetic through the host precompile, as an ablation over the SHA-3-hasher verification path.

Configuration (cumulative)	SP1 cycles	RISC Zero cycles	Δ vs. baseline
Baseline (BigUint emulation)	289,778,709	812,646,400	n/a
+ field mul via host precompile	276,643,994	n/a	SP1 −4.5%
+ power as square-and-multiply	240,857,140	n/a	SP1 −16.9%
+ skip redundant reduction	179,952,920	239,599,616	SP1 −37.9%, R0 −70.5%

Table 5: Cycle-level cost attribution, PLUM verification, SHA-3 hasher, $\lambda = 128$, *execute mode*. Routing field arithmetic through the host precompile (and removing redundant reductions) cuts 37.9% of SP1 cycles and 70.5% of RISC Zero cycles; RISC Zero’s larger reduction reflects its heavier per-multiplication baseline emulation (6,625 cycles/mul at baseline, falling to 1,826 with the precompile path, against SP1’s baseline of about 2,567 cycles/mul).

Where the cost lives, and the parameter-set caveat. At $\lambda = 128$ a single PLUM verification on the SHA-3-hasher arm issues about 112,902 $\mathbb{F}_p^{192}$ multiplication syscalls through the host precompile, as counted in our own SP1 run log; no Griffin permutation executes on that arm, so this figure measures the verifier’s *non-hash* field arithmetic (PLUM §4.2 reports $\approx 10^4$ non-hash algebraic R1CS constraints, 10,333 of 116,285). The Griffin-internal multiplication cost appears instead in the with/without-AIR contrast at $\lambda = 80$: the software-Griffin arm fires 4,870,724 `UINT256_MUL` syscalls against 69,396 with the Griffin AIR active, the hash-internal field operations either absorbed by the `GRIFFIN_FP192_PERMUTE` syscall or inflating the multiplication count $70\times$. The contrast is therefore consistent with the arithmetic-mismatch model: the cost is dominated by the algebraic hash, whose internal field operations leave the syscall stream only when the bespoke chip absorbs them. We caution explicitly against transferring the PLUM paper’s circuit-SNARK figure (that 91% of *R1CS constraints* are hash work) directly to the zkVM cost model: that figure is a constraint count in a different proving system, whereas Table 5 is a RISC-V cycle count. The qualitative conclusion (hash/field arithmetic dominates) agrees; the exact share differs by cost model and is reported here in the zkVM’s own units.

Execute-mode delta for the Griffin precompile (PLUM-80). For the bespoke Griffin AIR specifically, the precompile-free versus precompile-enabled execute-mode cycle counts differ by a factor

of $56.4\times$ at $\lambda = 80$, the ratio of precompile-free to Griffin-AIR-enabled cycles on the Griffin arm, consistent with the field-mismatch lower bound of Proposition 4.2 (which predicts a per-multiplication tax of up to ≈ 49 for PLUM’s 199-bit field on BabyBear); this is distinct from the 37.9%/70.5% of Table 5, a separate field-multiplication-reduction ablation reported at $\lambda = 128$. We report this as an *execute-mode cycle ratio* rather than a wall-clock speedup: it is the machine-independent counterpart of the infeasible-to-feasible prove transition of Table 2, and it is the figure that should be cited when a single number is wanted, in place of any prove-time “speedup,” which is undefined when the baseline does not complete.

7.6 PLUM-in-BDEC and the Consumer-Hardware Frontier

Moving from a single signature to the BDEC credential-generation relation $R_{\text{cre}}^{\text{PLUM}}$ (two signature verifications under a hidden user key; §5.6), Table 6 reports the application-level results at $\lambda = 80$: RISC Zero in execute mode and SP1 in succinct prove mode for both BDEC relations.

Workload	Mode	Cost	Outcome
BDEC CreGen, RISC Zero	execute	9.40×10^9 cyc, ≈ 65 s	both verifications accept
BDEC CreGen, RISC Zero	prove (succinct)	≥ 15.6 h, peak RSS 11.24 GB	internal-verifier rejection (anomaly; §7.6.1)
BDEC CreGen, SP1	prove (succinct)	26.98 and 31.11 min ($n = 2$)	receipt verified; not zero- knowledge
BDEC ShowCre $k=1$, SP1	prove (succinct)	40.15 and 44.24 min ($n = 2$)	receipt verified; not zero- knowledge
BDEC ShowCre $k=2$, SP1	prove (succinct)	58.14 min ($n=1$)	receipt verified; not zero- knowledge

Table 6: BDEC credential relations, $\lambda = 80$, M5 Pro 24 GB. On RISC Zero, execute mode validates the honest-input path of CreGen (9.4×10^9 cycles, $\approx 4.69 \times 10^9$ per signature verification); its succinct prove yielded no receipt, from an internal-verifier rejection rather than a resource limit (§7.6.1), not used as a feasibility result. On SP1 both relations complete a *succinct* prove: CreGen in 26.98 and 31.11 min, ShowCre in 40.15 and 44.24 min at $k=1$ and 58.14 min at $k=2$. These receipts are succinct STARKs and are *not* zero-knowledge; the $k=2$ figure agrees with the additive cost model within $\approx 3.3\%$ of the $(k+2)/3$ projection from the $k=1$ mean (run records docs/measurements/pub_hardening_20260611 and audit5_campaign_20260612).

The execute-mode result validates the honest path of the BDEC credential relation end-to-end inside the zkVM: both signature verifications accept, at 9.4×10^9 cycles; rejection behaviour was probed only at single-verification scope (Section 5). The prove-mode outcome requires care, because it is *not* the deployability frontier it might first appear to be, and we separate the two readings explicitly.

7.6.1 An internal-verifier rejection rather than a deployability frontier

The succinct-prove run (RISC Zero 3.0.5, the resolved version of the repository lockfile, verified against the compiled library; `ProverOpts::succinct()`, the succinct-STARK path; RISC Zero’s

API default is the composite receipt) terminated after 15.6 hours with `verify segment / verification` indicates proof is invalid. The error context localises the failure precisely: the string `verify segment` is attached at a single site in the prover (`prover_impl.rs:280`, identical in releases 3.0.3 through 3.0.5), the integrity self-check a freshly proved *base segment* must pass immediately after `prove_core`; the recursion layers carry distinct contexts (`verify lift`, `verify join`) that never appeared. The prover therefore rejected one ordinary segment of its own making, well mid-stream: at the measured ≈ 40 s per segment, 15.6 hours corresponds to roughly segment 1,400 of 9,936. This is categorically different from running out of memory: peak resident memory was only 11.24 GB on the 24 GB machine, with kernel free memory at 70–85% for nearly the whole run and dipping to 47% only during a brief memory-compression spike in the first minutes, so the machine was *not* the limiting resource. An internal-verifier rejection is a correctness anomaly; it tells us nothing about deployability, and we therefore do *not* report it as a frontier or fold it into any feasibility claim. The localisation also rules candidates out. A segment-count threshold is contradicted by the index arithmetic: the rejection occurred near segment 1,400, below the $\approx 4,500$ segments the single-verification workload clears (the RISC Zero row of Table 2, 6 h 19 m, receipt verified). A content-deterministic cause, including a malformation of the `sys_bigint` precompile trace (the bespoke Griffin AIR named by Assumption 6.1 of Section 6 is SP1-only and was not on this proving path), is disfavoured by the partial composite re-run (started 2026-05-31): with the same guest, the same deterministically seeded input, and the same machine, it proved 2,452 of the 9,936 segments ($\approx 24.6\%$, covering the region where the succinct run failed) through the same per-segment integrity check, in 34.7 h at a steady ≈ 40 s per segment, peak RSS ≈ 9.9 GB, without reproducing the rejection, before we terminated it (projection ≈ 4.6 days for all segments). The surviving candidates are therefore (i) a transient error over the ≈ 245 CPU-hour run on non-ECC memory, and (ii) a concurrency-sensitive defect in the succinct path’s overlapped prove/lift/join pipeline, the one structural element the composite path does not exercise. The failing-segment index was not captured in the succinct run; the discriminating next step is a succinct re-run with segment-index logging and the recursion stages serialised, which separates (i) from (ii) by whether the rejection recurs and where. Pending it, the anomaly is recorded as exactly that (an anomaly under diagnosis), and the thesis’s claims do not rest on it.

7.6.2 The genuine deployability frontier

Two failures bound the deployability frontier of this thesis, both unambiguous as to cause and independent of the anomaly above. The first is the PLONK zero-knowledge wrap of the standalone PLUM single-signature-verification receipt (Cell 2 of Table 2, the 14.25-min prove, not CreGen or ShowCre). At SP1’s default prover concurrency this wrap is killed under memory pressure at $\approx 83\%$ of the 24 GB budget (peak RSS 19.98 GB per the process accounting at the kill; a periodic RSS sampler observed ≈ 17.8 GB before the final spike; run records `docs/measurements/bench_suite_20260529_overnight` (sampler) and `docs/measurements/zkwrap_worker_concurrency_20260627`). This kill is not, however, a memory limit of the workload: lowering the recursion-worker count runs the same wrap under ≈ 17 GB, comfortably inside the budget (`docs/measurements/zkwrap_default_shard_20260627`) where it fails earlier instead, at the recursive-compression step, because SP1’s static recursion shapes are provisioned for smaller relations than PLUM verification; we call this **the recursion-shape provisioning wall**. Bringing the wrap within reach is a one-time regeneration of those static artifacts (the recursion-shape and verifying-key tables), roughly 51 hours of compute on this hardware (48–54 h, from a measured per-setup rate of ≈ 1 s in a single CPU process over the 185,862-entry rebuild; the

measured rate is in run record `docs/measurements/zkwrap_vkmap_slope_20260628` and the extrapolation to ≈ 51 h in `docs/measurements/zkwrap_failure_map_20260628`), rather than more memory. We do not run the regenerated PLUM toolchain end-to-end, but the wrap’s two memory-heaviest stages each fit the budget. PLUM’s core-and-recursion proving peaks at ≈ 17 GB, and the program-independent gnark PLONK wrap stage at 15.4 GB peak RSS (a trivial-guest probe of the fixed BN254 wrap circuit, $n=1$; `docs/measurements/gnark_plonk_wrap_ram_20260629`). No stage exceeds 24 GB, so the binding barrier is the one-time ≈ 51 h regeneration, a compute cost that more memory does not remove. The provisioning wall specific to the anonymity-providing wrap is thus structural, not a memory ceiling, and it is the measured form of the anonymity–deployability tension of Section 6.

The precompile is the source of the provisioning wall. The provisioning wall attributes entirely to the Griffin- $\mathbb{F}_p^{192}$ precompile. Removing only the Griffin chip, with every other fork edit intact, reproduces the pre-Griffin compose shape byte-for-byte (795,264 provisioned ExtAlu events, md5 `f97a8b2f`); with the chip the shape is 1,072,032, a marginal +276,768 ExtAlu (+34.8%) and +38,816 PrefixSumChecks, while the fork’s non-Griffin recursion edits move the shape by exactly zero. The required height (1,064,022, an overflow of 268,758 over the committed 795,264 budget) is what trips the arity-4 compose shape against that budget (run record `docs/measurements/zkwrap_griffin_shape_at`). The precompile that lowers the core-layer cost is therefore the same chip that enlarges the recursion shape the wrap must verify.

Second, and separately, a *non-zero-knowledge* baseline proof of a standardised-lattice signature verification (an ML-DSA proxy: the 17 NTT-domain polynomial multiplications of 256 coefficients modulo $q = 8,380,417$ that dominate ML-DSA-65 verification, proved as a baseline succinct STARK on SP1, not a zero-knowledge wrap) was jetsam-killed after 249 s with peak RSS 12.59 GB and a virtual-memory footprint of 82.7 GB (compressed memory plus swap, hence exceeding physical RAM); this is a substrate base-cost datum that *strengthens but does not instantiate* the tension, since it bears on the cost of proving any PQ verification on this substrate, not on the zero-knowledge wrap specifically. These are the data points the frontier claim uses; the positive feasibility anchor remains the PLUM single-verification prove of Table 2, which completes in 14.25 min. Table 7 summarises, for every proof mode attempted, whether a receipt is produced and whether it is zero-knowledge and post-quantum.

7.7 A Cost Model and the Substrate Crossover

The measurements above are point observations; to answer “what would this cost at a configuration we did not run, and when is the flexible substrate the rational choice,” we fit a cost model to them and validate it against the points we have. This is the analysis that converts the measurements from a benchmark table into a basis for a deployment decision. The model is fitted and validated in *execute-mode* cycle counts; carrying it to prove-mode wall-clock assumes the additive form survives recursion, memory pressure, segment aggregation, and the zero-knowledge wrap, any of which can introduce nonlinearity, so the prove-mode reading is an extrapolation, not a measured fit.

7.7.1 An additive cost model for the BDEC relations

Both BDEC relations are, at the cycle level, dominated by repeated signature verifications (Section 4.4): CreGen contains two, and ShowCre contains $k+2$ for k shown credentials. Writing $C_{\text{ver}}(\lambda, h)$ for the execute-mode cycle cost of one PLUM verification at security level λ under hasher h , and C_0 for the fixed per-run overhead (input parsing and witnessing; the ledger-membership, revocation, and disclosure checks are performed by the relying party outside the proof), the model is

$$C_{\text{CreGen}}(\lambda, h) \approx 2 C_{\text{ver}}(\lambda, h) + C_0, \quad (5)$$

$$C_{\text{ShowCre}}(k, \lambda, h) \approx (k + 2) C_{\text{ver}}(\lambda, h) + C_0, \quad (6)$$

The model is deliberately linear in the verification count, because the cost decomposition of Section 4.4 shows that everything else in the proof is dominated by the $\Theta(k)$ algebraic-hash work inside the verifications; the ledger-membership, revocation, and disclosure checks do not enter the proving cost, as the relying party performs them outside the proof.

7.7.2 Validation against the measured points

The model is falsifiable against the two BDEC points we measured. The strongest internal check is CreGen: Equation (5) predicts the two verifications should each cost $\approx C_{\text{ver}}$ and together dominate the trace. The measurement bears this out directly: the two verifications cost 4.69×10^9 cycles *each* (the two within 0.01% of *each other*), their sum 9.39×10^9 accounting for $\approx 99.8\%$ of the 9.40×10^9 total (Table 6); so for the Griffin hasher at $\lambda = 80$, $C_{\text{ver}}(80, \text{Griffin}) \approx 4.69 \times 10^9$ cycles and C_0 is small. We caution that this single relation validates the *linearity* and the *verification-dominance*, not the absolute constant at other (λ, h) ; the $k = 14$ projection below is labelled as such.

The hasher gap measures the tax. A single PLUM verification is markedly cheaper with the SHA-3 hasher control than with the Griffin algebraic hash, an order-of-magnitude separation that is the field-mismatch tax of Section 4 quantified at the verification level. Matched- λ figures exist on both substrates: on SP1 at $\lambda = 80$ in execute mode, the software-Griffin verification costs 7.08×10^9 cycles against 1.11×10^8 for the SHA-3 arm ($\approx 64\times$), and on RISC Zero at $\lambda = 80$ the Griffin per-verification cost of 4.69×10^9 cycles stands against $\approx 1.44 \times 10^8$ cycles for the SHA-3-hasher verify ($\approx 33\times$, field multiplications routed through `sys_bigint`). The qualitative conclusion (that the algebraic hash is the tax-bearing component while the field arithmetic is not, so C_{ver} must be reported per hasher) holds regardless of the exact multiplier. The precompile narrows this gap (Table 5); closing it entirely is what a native Griffin AIR on the proving path would buy.

ShowCre execute-mode cost (measured at $k = 1, 2$). The ShowCre relation is realised by the guest `bdec_showcre_plum_griffin`, the $k + 2$ generalisation of the CreGen guest, and measured in execute mode at the two running examples. At $k = 1$ (the ϕ_1 workload) ShowCre executes in 1.41×10^{10} cycles, and at $k = 2$ (the ϕ_2 workload) in 1.88×10^{10} cycles, each with all $k + 2$ verifications accepting. Both agree with Equation (6) at $C_{\text{ver}}(80, \text{Griffin}) \approx 4.69 \times 10^9$: the per-verification cost is constant at $\approx 4.69 \times 10^9$ cycles across CreGen and both ShowCre instances, so the additive $(k + 2) C_{\text{ver}}$ form is measured rather than assumed. Applying the same measured constant to a stress-test disclosure-set size, well beyond the running examples ($k = 1, 2$) and the $k \leq 4$ design target, projects

$\approx 7.5 \times 10^{10}$ cycles at $k = 14$. A direct $k = 14$ execute run does not reach this projection: the 16 accumulated verifications exhaust the guest’s default non-reclaiming bump allocator before the cycle count completes, an in-guest memory limit distinct from the prover-side provisioning wall and clearable only by the reclaiming allocator at additional cycle cost. The $k = 1$ and $k = 2$ executions are correct and their cost is measured; the zero-knowledge *proof*, however, rests on the same anonymity-providing wrap blocked by the recursion-shape provisioning wall of §7.6.2. A zero-knowledge proof of ShowCre at any $k \geq 1$ is therefore bounded by the provisioning wall rather than by the CreGen prove anomaly (§7.6.1). Clearing the provisioning wall is a structural fix, a one-time regeneration of SP1’s recursion-shape artifacts rather than added memory, and a post-quantum-anonymous proof additionally requires a post-quantum-sound wrap (Section 6).

Calibrating the trace-area constants. Calibrating the constants of the trace-area model (Equation (1)) on the two completing SP1 prove runs of Table 2 (the Cell 2 Griffin arm and the Cell 3 SHA-3 control; §7.3), the control arm prices core work at $\kappa_{\text{core}} \bar{a}_{\text{core}} \approx 7.2 \mu\text{s}$ per guest cycle, and attributing the Griffin arm’s residual 17.5 minutes to its 5.6×10^7 cells gives an effective $\kappa_{\text{Griffin}} \approx 19 \mu\text{s}$ per committed cell; in unit-free form, committing one Griffin-chip cell costs the prover about as much as removing 2.6 guest cycles buys. Two measured points fit two constants, so the magnitude is reproduced by construction; the out-of-fit content is the sign and the orderings the same constants imply. The calibration uses the May run pair, for which matched execute-mode guest-cycle counts exist; the June re-measurement (§7.3) confirms the postdicted sign, and we do not re-derive the constants for it, lacking matched June cycle counts. Those orderings are consistent with the rest of Table 2: the precompile-free baseline’s 7.08×10^9 cycles price at upwards of 14 hours of core work alone (before the chip area of its 4,870,724 UINT256_MUL events is charged), consistent with its DNF, and against that baseline the precompile still pays by orders of magnitude, removing about 6.6×10^6 cycles per permutation against 5.3×10^4 cells added.

7.7.3 The substrate crossover

The decision a deployer faces (Section 8) is not about which substrate is faster per proof, since the static circuit wins that in any case. It is about which substrate is cheaper over a deployment *lifetime* during which the predicate changes. Let the lifetime contain P proof generations and U localised predicate changes. Amortised total cost is, schematically,

$$\text{static: } U \cdot R_{\text{static}} + P \cdot t_{\text{static}}^P, \quad (7)$$

$$\text{zkVM: } U \cdot 0 + P \cdot t_{\text{zkVM}}^P, \quad (8)$$

where R_{static} is the per-change recompilation cost and t^P is per-proof time. We isolate R_{static} directly (§7.8): for the single-Loquat-verification R1CS at the 2^{19} BDEC building-block size it is R1CS construction (≈ 0.2 s) plus Aurora parameter setup (measured at $\approx 3 \mu\text{s}$, because Aurora is non-preprocessing), so $R_{\text{static}} \approx 0.3$ s. The multi-minute figure of Table 3 is the per-*proof* prover, not the per-change recompile, and does *not* enter R_{static} . The zkVM’s update term is zero (Table 8). The static regime is cheaper only while

$$U \cdot R_{\text{static}} < P (t_{\text{zkVM}}^P - t_{\text{static}}^P),$$

i.e. while the relation-change rate U/P stays below a crossover

$$\left(\frac{U}{P}\right)^* = \frac{t_{\text{zkVM}}^P - t_{\text{static}}^P}{R_{\text{static}}}.$$

The measured $R_{\text{static}} \approx 0.3$ s is decisive, and it cuts *against* a naive flexibility argument. Because R_{static} is sub-second while the zkVM’s per-proof penalty exceeds the static prover’s ($t_{\text{zkVM}}^P > t_{\text{static}}^P$ even granting the cross-scheme seam of §7.4: static Loquat- $\mathbb{F}_{p^{127}}$, zkVM PLUM- $\mathbb{F}_p$), the crossover $(U/P)^*$ of Eq. 7.7.3 is on the order of thousands of relation changes per proof, a regime no realistic deployment reaches. **Against a transparent, non-preprocessing argument such as Aurora, the zkVM thus has no wall-clock flexibility advantage:** both regimes rebuild a sub-second artefact per change, and the zkVM is then dearer per proof. Recompile wall-clock does not capture the genuine per-change cost of the static regime; that cost is the *re-audit and engineering* footprint of a new relation-specific circuit (Table 8, $N_{\text{circ}}/N_{\text{param}}$). A *wall-clock* flexibility advantage would appear only against a *preprocessing* argument (Groth16 [67], Marlin [40], or circuit-specific PLONK [68]), whose per-circuit key generation makes R_{static} large, and Aurora, the BDEC baseline, is not such a system. We therefore report the flexibility result as *qualitative* (the regeneration footprint of Table 8) together with this measured wall-clock caveat, rather than as a numeric crossover that favours the zkVM.

7.8 Update Churn

Table 8 enumerates the regeneration footprint of Definition 4.6 for the three localised verification-predicate changes (Definition 4.5) of §4.3, using $\phi_1 \rightarrow \phi_2$ (par. 1) as the presentation-change instance. Here $\phi_1 \rightarrow \phi_2$ is a presentation-*structure* change: it moves the credential count from $k=1$ to $k=2$, altering the relation $R_{\text{show}}^{\text{Sig}}$ and hence the static R1CS. A presentation *policy* change at fixed structure (e.g. retuning a threshold the verifier checks on the disclosed attributes) is handled verifier-side and triggers no regeneration on either substrate, so the rows below measure structural changes.

Structural separation, and the measured recompile cost. The structural separation in Table 8 is established by construction: a zkVM’s relation is fixed to the ISA, so a presentation-structure or signature change is absorbed as a source edit ($N_{\text{circ}} = N_{\text{param}} = 0$) with no recompilation. *Both* components of the static side are now measured, and the result overturns the earlier conjecture that recompilation is expensive. R1CS *construction* from the BDEC relation is sub-second (≈ 0.2 s of CPU time per emission at Loquat- $\lambda=80$ and $\lambda=128$, single timed runs; cached wall-clock 0.48 s). The Aurora *parameter setup* that a relation change forces to recompute is, measured directly, about $3 \mu\text{s}$ (the runner reports $t_{\text{setup}} = 0.003$ ms at the 2^{19} -padded single-Loquat-verification R1CS, the BDEC building-block size), because Aurora is *non-preprocessing*: it has no per-circuit indexer. The prover’s low-degree-encoding FFTs (its dominant step) run *per proof*, not per relation change, and so do not enter R_{static} . Hence $R_{\text{static}} \approx 0.3$ s at the BDEC building-block size. A $\phi_1 \rightarrow \phi_2$ change alters only the credential count and does not enlarge the microsecond parameter setup, so this bound also holds for the per-presentation delta; a k -parameterised emitter would refine the R1CS-construction term but cannot move the dominant conclusion, namely that for a transparent non-preprocessing argument the per-change recompile is sub-second. Consequently the zkVM’s wall-clock flexibility advantage over Aurora is negligible, and the static regime’s real per-change cost is the *re-audit* of a new circuit (Table 8) rather than recompile time.

The source-edit and audit components. The remaining tuple components, promised in prose by Section 4.3, are read off this project’s own repository record. For the security retune ($\lambda = 80 \rightarrow 128$) the zkVM side is the clean case: the guest binary is parameterised over λ , so $N_{\text{src}} = 0$ and $N_{\text{audit}} = 0$; the static side keeps its generator source ($N_{\text{src}} = 0$) but its regenerated matrices re-enter the review boundary ($N_{\text{audit}} \geq 1$). For the presentation change ($k:1 \rightarrow 2$) the zkVM edits one guest source (the presentation relation’s credential count), giving $N_{\text{src}} = 1$ and $N_{\text{audit}} = 1$ (the changed guest and its image identifier); the static side regenerates the merged R1CS, with the regenerated circuit as the re-audited component. For the signature swap (Loquat $\rightarrow$ PLUM) the static side replaces the constraint generator for the verification predicate and re-audits the full regenerated circuit. The zkVM side is the honest headline of this dimension: the guest edit itself is small, but because the swap moves the hash field, it crossed the precompile boundary, and in this project’s record the crossing cost was the bespoke Griffin- $\mathbb{F}_p^{192}$ chip, approximately 3,900 lines of chip and executor code (§5), with the chip and its soundness argument (Appendix B) entering the audit boundary as new components. The absorb-as-software property therefore held for every benchmark change except the one that touched a precompiled primitive, where the zkVM regime paid a circuit-engineering cost of the same kind it was adopted to avoid.

The advantage is conditional on the baseline’s preprocessing model. The negligible recompile cost above is a property of Aurora’s *non-preprocessing* structure; it does not generalise to static circuits at large. A *preprocessing* (holographic) transparent argument such as Fractal [39] must instead build a per-circuit index, and that index is exactly what a relation change forces to regenerate. Timing Fractal’s indexer (its `fractal_snark_indexer` stage) in isolation on the same 127-bit-field harness as §7.4 (≈ 107 -bit achieved, zero-knowledge disabled; a Loquat proxy for PLUM) gives the contrast of Table 9. The 2^{10} – 2^{14} rows are synthetic R1CS instances emitted on the same harness to expose the indexer’s scaling; only the 2^{19} row is the single-Loquat-verification relation at the BDEC building-block size. Aurora’s parameter setup is microseconds and flat in circuit size, whereas Fractal’s holographic indexer grows superlinearly and, at the 2^{19} -padded size, is killed under memory pressure mid-indexer (during a 2^{27} -element FFT) before returning.

The flexibility reading is therefore *conditional on the static baseline’s preprocessing model*: the zkVM’s wall-clock recompile advantage is negligible against a non-preprocessing argument (Aurora) and large against a preprocessing one (Fractal), where at the BDEC circuit size the baseline’s per-circuit indexing is itself infeasible on the target hardware. The same-scheme and field seam of §7.4 applies throughout (a Loquat proxy, not PLUM), and a smaller folding parameter than the Aurora-matched $RS = 5$ might let the Fractal indexer fit and yield a finite but still large R_{static} .

7.9 What the Measurements Establish

We close by reading the measurements back against the cost framework of Section 4.

Feasibility and the field-mismatch tax. Without the precompile a single PLUM verification does not complete on the target hardware (out-of-memory at $\approx 1\text{m}45\text{s}$, Table 2); with it the verification completes in 14.25 min. At the cycle level the bespoke Griffin precompile removes a $56.4\times$ factor at $\lambda = 80$ on SP1 (§7.5), and the modular-multiplication precompile (`UINT256_MUL / sys_bigint`) separately removes 37.9%/70.5% of SP1/RISC Zero cycles at $\lambda = 128$ on the SHA-3-hasher path

(Table 5). This is the field-mismatch tax of Definition 4.1 made concrete: the precompile turns an infeasible workload into a feasible one, and the residual cost is what the rest of the evaluation measures. The with/without contrast at the prove level is an infeasible-to-feasible transition rather than a finite ratio (§7.3); the factors above are cycle-level and single-verification, and we do not promote them to a CreGen-relation prove speedup we did not run.

Where the cost lives. The cost attribution of §7.5 bears out the decomposition of §4.4: the dominant term is the algebraic hash, whose internal field operations inflate the $\mathbb{F}_p^{192}$ multiplication count $70\times$ when not absorbed by the bespoke chip (4,870,724 against 69,396 `UINT256_MUL` syscalls at $\lambda = 80$), far beyond the explicit verifier field arithmetic. We report this in the zkVM’s own cycle units and caution against transferring PLUM’s circuit-SNARK “91% of constraints are hash” figure to the cycle model.

Per-proof cost against the static circuit. For a *fixed* predicate the specialised static circuit is the faster prover, as expected from a specialised encoding: Aurora proves the single Loquat-verification circuit in 3.76 min (Table 3) against 14.25 min for precompiled PLUM verification (Table 2). The prove times do *not* converge; but the two points are not the same scheme (Loquat over $\mathbb{F}_{p^2}$ versus PLUM over its 199-bit prime field, §7.4), so we state the gap as indicative, and the comparison the thesis turns on is the deployment-lifecycle one below rather than the per-proof one. For Loquat the same-scheme comparison is measured (Table 4); for PLUM, closing the per-proof gap requires building an $\mathbb{F}_p$ (199-bit) Aurora harness to run PLUM statically (the current Aurora harness is $\mathbb{F}_{p^{127}}$ -only), and we identify this as the required follow-up experiment.

Update churn. The structural separation of Definition 4.6 holds by construction (Table 8): every localised change regenerates at least one circuit and one parameter set on the static side and none on the zkVM side, provided the change does not alter a precompiled primitive or its field; a change to the hash family or field width regenerates the Griffin AIR and so incurs $N_{\text{circ}} \geq 1$ on the zkVM side too (Definition 4.6). The recompile wall-clock of the static side is now measured ($R_{\text{static}} \approx 0.3$ s; §7.8) and is sub-second because Aurora is non-preprocessing; the gap that remains is a *same-scheme* per-proof static prover time for PLUM (§7.4) rather than the recompile, the Loquat case being settled by Table 4.

The zero-knowledge frontier. The base proof completes; what binds on consumer hardware is the zero-knowledge wrap. The full account of the measured resource failures, including the separate lattice base-cost datum, is in §7.6.2. The wrap obstruction is substrate-specific: the static circuit completes a zero-knowledge proof of the same Loquat building block in about 22 minutes (mean of three runs, 21.4–22.9) on the same machine (§7.4). The frontier claim rests on this resource-failure evidence alone; the CreGen prove anomaly (§7.6.1) plays no part in it.

7.10 Threats to Validity

- **Single-run and repeated-run variance.** The standalone PLUM Cell 2 and Cell 3 prove-mode wall-clock figures (Table 2) are now means of three fixed-config runs each (`docs/measurements/tier2`); the remaining prove-mode figures (the BDEC CreGen and ShowCre relations) are single runs. The Loquat substrate legs are three runs each (`docs/measurements/pub_hardening_20260610`),

and they characterise the variance directly: the Aurora leg is tight (223.2–226.6 s, $\pm 0.8\%$), while the SP1 leg spans 44.4–64.0 minutes ($1.44\times$) on an otherwise idle machine, the slow run starting on a package heated by the preceding one. A separate datum measures contention: an Aurora run under concurrent prover load moved from 225.5 s to 370.0 s ($1.64\times$; docs/measurements/loquat_sub). Thermal state and co-running load on laptop-class hardware therefore move wall-clock by factors of 1.4–1.6. Conclusions resting on order-of-magnitude separations, the 12–17 $\times$ substrate gap and the memory-exhaustion walls, are robust to movement of this size. The Cell 2 versus Cell 3 inversion is now confirmed by $n = 3$ fixed-config runs per arm: the sign is robust (Griffin slower in every run), and the replicated magnitude is $\approx 1.09\times$ in wall-clock, with each arm’s prove-time spread under 5%. An earlier single run measured a $2.45\times$ cross-build figure; it was not reproduced under fixed-config replication, is attributed to machine-state contamination, and is retired (§7.3). The sign is model-postdicted; the magnitude we do not over-read.

- **Thermal throttling.** Runs are on a laptop-class M5 Pro; sustained multi-minute proving may throttle, which would inflate wall-clock relative to a thermally-unconstrained host.
- **Hardware/OS-string provenance.** The reported platform string is from `sysctl`; we have not cross-checked every sub-component against an independent inventory.
- **Incomplete statement binding.** The RISC Zero prototype does not yet bind the receipt journal to the public statement (c, h, ppk) (Section 5); until it does, an accepting receipt is a functional benchmark and not a statement-bound credential proof, which is a security gap and not a timing one.
- **Unresolved CreGen prove anomaly.** The 15.6-hour BDEC credential-generation prove run ended in an internal-verifier rejection (§7.6.1), not a resource limit; it is a single run with undetermined cause, no claim of this thesis rests on it, and a segment-diagnostic re-run is required before any statement about it. It is explicitly *not* part of the deployability frontier (§7.6.2).
- **Parameter-set seam.** Prove-mode wall-clock is at $\lambda = 80$; cycle attribution is at $\lambda = 128$. The two are never combined into one figure, but a reader must track which is which; the tables are labelled accordingly.
- **Non-uniform precompile.** SP1 uses the bespoke Griffin AIR; RISC Zero uses `sys_bigint`. Cross-zkVM numbers reflect this mechanism difference and are not a like-for-like substrate comparison.
- **Constraint-level soundness of the Griffin AIR is empirically supported, not proved** (the chip–constraint equality, Assumption 6.1, of Section 6); the positive feasibility figures, namely the 14.25-minute Cell 2 prove, the 13.0-second smoke proof, and any prove-level reading of the $56.4\times$ cycle ratio, assume the chip computes the reference permutation: proved at the per-row layer (Appendix B, conditional on the byte-range lookups under the lookup-binding hypothesis), tested on 256 random vectors and by single-cell fault injection on seven of the ten constraint families, and not verified at the constraint level for the lookup-borne families or the code-to-constraint transcription. The negative findings of this chapter do not carry this premise: an unsound AIR would mean the measured Griffin-arm cost understates that of a corrected

and at-least-as-costly chip, so the non-terminating precompile-free baseline, the SHA-3 inversion, the recursion-shape provisioning wall, and the Loquat substrate gap survive unchanged or strengthen, as argued in Section 6.

- **Control vs. deployable.** The SHA-3 arm is a diagnostic control; it is not a secure PLUM variant and its timing is not a deployment claim.

7.11 Summary

On consumer hardware, the precompile suite moves PLUM verification from infeasible to feasible (Table 2), with the cost dominated by large-field/algebraic-hash arithmetic and reduced by 37.9–70.5% at the cycle level (Table 5); the full BDEC credential relation’s honest path is validated in execute mode, while its succinct prove ended in an internal-verifier rejection categorically distinct from the deployability frontier (Table 6, §7.6.1), the binding consumer-hardware frontier being the zero-knowledge wrap (§7.6.2); and the zkVM regime is free of circuit and parameter regeneration for every localised change that leaves the precompiled primitive suite intact (hash-family or field-width changes regenerate the Griffin AIR; §7.8, Table 8). The one comparison the data does not yet settle is a *same-scheme* per-proof prover time for PLUM; for Loquat it is settled in the static circuit’s favour (Table 4), and the static-circuit recompile wall-clock is now measured ($R_{\text{static}} \approx 0.3$ s) and is not the bottleneck. The next chapter takes up what these results mean for the deployment decision and which questions they leave open.

8 Discussion

The measurements of Section 7 and the security analysis of Section 6 together answer the question this thesis set out to ask. That question concerns whether a general-purpose zkVM is a viable substrate for post-quantum anonymous credentials, and at what cost, rather than how fast the zkVM runs in isolation. We begin with the sharpest of these findings, the security–deployability tension, then turn the surrounding cost picture into guidance a deployer can act on.

8.1 The Security–Deployability Tension

The sharpest finding is the tension of Section 6, which outweighs any single timing figure. On the target hardware at $\lambda = 80$, the proof that is feasible to generate is not zero-knowledge, and the proof that is zero-knowledge, the one BDEC’s anonymity guarantee requires, is not feasible to generate within the memory budget. A deployer who needs anonymity on commodity hardware today cannot, on the evidence here and with the current toolchain versions, have it for this workload. That statement is scoped to the zkVM stack: the static-circuit regime produces a zero-knowledge post-quantum proof of the Loquat single-verification building block in about 22 minutes (mean of three runs, 21.4–22.9) on the same machine (`docs/measurements/audit5_campaign_20260612`), at the per-change regeneration cost this section weighs. Clearing the recursion-shape provisioning wall is a one-time toolchain regeneration, not added memory, and even cleared it would not restore the post-quantum anonymity that motivates the construction, for the substrate-specific reasons established in Section 6. Recovering it requires a post-quantum-sound zero-knowledge wrap, a STARK- or lattice-based recursive argument, which the toolchains do not yet provide. We locate this obstruction along two axes at once, structural and primitive, and it holds independently of how favourable our timing numbers are and of the open AIR-soundness Assumption 6.1, which conditions the feasibility figure rather than the obstruction (Section 6).

8.2 A Substrate-Selection Rule

The two regimes do not dominate each other; they trade different costs, and the right choice depends on a single property of the deployment: how often the verification predicate changes relative to how often proofs are generated.

- **When the predicate is fixed at deployment time** and proofs are generated frequently, the static-circuit regime is preferable: its specialised RICS amortises across many showings, and it pays its update cost only once. The zkVM’s per-proof overhead (Section 7) is not justified when there is nothing to update.
- **When the predicate is chosen at presentation time, or the signature or security parameters evolve over the deployment’s lifetime**, the static-circuit regime pays its update-churn cost (Table 8) *repeatedly*, and the zkVM regime, which absorbs each such change as a guest-program edit, becomes the structurally appropriate substrate, at the proving cost quantified in Section 7. For a non-preprocessing static argument such as Aurora this update-churn cost is a regeneration-and-re-audit (governance and assurance) burden rather than a large wall-clock penalty, which the next paragraph makes precise.

This rule is made precise by the crossover of §7.7.3: the predicate-change rate $(U/P)^*$ above which the zkVM is the cheaper substrate over a deployment lifetime is the ratio of the zkVM’s per-proof penalty to the static circuit’s per-change recompilation cost. The recompilation cost is now measured (Section 7): for a transparent non-preprocessing argument such as Aurora it is $R_{\text{static}} \approx 0.3$ s (R1CS construction plus a microsecond parameter setup), and it sits outside the multi-minute prover, which is paid per proof. At that magnitude, and with the zkVM dearer per proof, the wall-clock crossover favours the static circuit in any realistic churn regime: against Aurora the zkVM has no wall-clock flexibility advantage. The flexibility claim we can defend is the *qualitative* regeneration-and-re-audit footprint of Table 8 (zero on the zkVM side, ≥ 1 on the static side per change), which is an engineering-and-assurance cost paid by humans, not prover time. A wall-clock advantage appears only against a *preprocessing* baseline, as Section 7 measures for the transparent preprocessing argument Fractal (Table 9), whose per-circuit indexer takes 0.86–18.5 s at small synthetic sizes and is itself OOM-killed at the BDEC circuit size on 24 GB, and as would hold in principle for Groth16, Marlin, or circuit-specific PLONK; the non-preprocessing Aurora that BDEC uses does not offer one. This zero-regeneration property is partitioned rather than absolute. A change confined to the verification predicate is a guest-program edit that reuses the existing universal relation, but a change to the hash family or to the field width still regenerates and re-audits the bespoke Griffin-Fp192 precompile, so the decision rule applies to predicate churn and not to substrate-level changes of the signature itself. What the cost model licenses is limited: it fixes the *form* of the rule and the *sign* of every term, but not the numeric location of the crossover for PLUM, which requires a same-scheme static prover time we do not yet have at PLUM’s field width (§7.7.3); the Loquat counterpart is measured in the next paragraph. So the rule gives directional guidance, telling a deployer which side a high- or low-churn deployment lies on without yet handing them a numeric threshold to evaluate.

The contribution of this thesis to that rule is the *quantification of both sides*: the zkVM’s per-proof cost is now measured (Tables 2–6), and the structural form of the static circuit’s update cost is enumerated (Table 8). The static circuit’s recompile *time* is now measured ($R_{\text{static}} \approx 0.3$ s, §7.8) and is sub-second because Aurora is non-preprocessing; the remaining term is a *same-scheme* per-proof prover time on both substrates. For *Loquat* we now measure it directly: at single-verification scope, $\lambda = 80$, with neither side zero-knowledge, the static Aurora circuit proves in 3.76 min against 44–64 min for the zkVM across three runs each (Table 4), so on this clean single-axis comparison the zkVM is the dearer per-proof substrate by 12–17 $\times$; the sign of the crossover for Loquat is therefore fixed, with the static circuit the cheaper prover. For PLUM the sign remains a conjecture: PLUM over its 199-bit field has no Aurora harness (the harness is 127-bit only), so we cannot yet repeat the measurement on the thesis’s headline scheme. We expect the same direction, and the Loquat measurement is direct evidence for it, but we have not measured PLUM.

8.3 What Generalises and What Does Not

The arithmetic-mismatch mechanism (Section 4) and the update-churn dimension (Definition 4.6) are substrate-independent: they apply to any large-field algebraic-hash post-quantum signature of this form verified in any small-field zkVM, not only to PLUM in RISC Zero/SP1. The specific magnitudes (Section 7) do not generalise: they are tied to PLUM’s 199-bit field, the M5 Pro’s 24 GB envelope, and the CPU-bound provers of the two zkVMs studied. We drew the boundary deliberately, so that the shape of the cost carries to other pairings of a large-field signature with a small-field zkVM while the

numbers stay pinned to the configuration measured here.

8.4 Comparison to Prior Evaluations

As anticipated in Section 2, the prior-evaluation record omits two axes: the cost of a *change* to the verification predicate, and the consumer-hardware proving frontier for an algebraic-hash post-quantum signature. We do not restate that gap here; instead we report what our measurements add to the record. For predicate-change cost, Table 8 fixes its structural form (zero regenerations on the zkVM side against ≥ 1 per change on the static side) and §7.8 bounds its wall-clock magnitude; for the frontier, §7.6.2 locates the binding constraint as the zero-knowledge wrap rather than the base proof as built and measured (the base-proof cost is the honest-path cost of the chip and carries Assumption 6.1). Where our per-proof timings overlap the existing record, they agree in order of magnitude with the hitherto-qualitative observation that algebraic-hash workloads are expensive inside small-field zkVMs, now quantified at the verification level (Tables 2–5). The next section draws these threads together into the thesis’s overall conclusion.

9 Conclusion

This thesis asked whether a general-purpose zero-knowledge virtual machine (zkVM) is a viable substrate for post-quantum anonymous credentials on consumer hardware. The goal was to make BDEC credential verification, instantiated with the PLUM signature, generate proofs in a practically acceptable time inside a zkVM on a personal computer (Apple M5 Pro, 24 GB); the means was a custom precompile suite that absorbs the field-mismatch tax of PLUM’s large-prime-field algebraic hashing. We realised the construction on RISC Zero and SP1, measured it against an Aurora static-circuit baseline, and analysed what the substitution preserves and what it costs.

9.1 Main Findings

The central result is a two-pronged obstruction to deployable post-quantum anonymity. On the target hardware the proof one can afford to generate (the standalone PLUM-verification receipt) is not zero-knowledge, and the zero-knowledge wrap the toolchain exposes is not post-quantum (Section 6.3). The first prong is structural rather than a memory bound: at default prover concurrency the PLONK wrap of the standalone PLUM verification receipt is killed by memory pressure at ≈ 20 GB of the 24 GB budget, but throttling the concurrency runs it well within budget, where it fails instead at recursive compression, the recursion-shape provisioning wall: the toolchain’s static recursion shapes are provisioned for smaller relations than PLUM verification, a one-time regeneration away from completing; each stage of the wrap fits within 24 GB (Section 7), so the binding cost is the regeneration’s compute, with the regenerated toolchain not run end-to-end. The second is a fact about the deployed toolchains, and it takes a different form on each substrate: the only succinct zero-knowledge wraps exposed by the SP1 pipeline we measure (v6.2.1, forked) are Groth16 and PLONK, both pairing-based, so on SP1 even a completed wrap could be at best classically secure; on RISC Zero the blinded succinct receipt carries no formally established zero-knowledge guarantee, so BDEC’s anonymity premise cannot be formally discharged on it, and its pairing-based wrap is x86-only and does not run on the Apple-silicon target at all (Section 6). This obstruction on both sides holds independently of any timing number and of the open AIR-soundness Assumption 6.1 (the dependency direction is recorded in Section 6 and restated under Limitations below), and the rest of the thesis rests on it.

Around that finding, the measurements establish the surrounding cost picture. The custom Griffin- $\mathbb{F}_p^{192}$ precompile turns a non-terminating emulation into a finite measurement: without it, standalone PLUM verification exhausts memory at $\approx 1\text{m}45\text{s}$ on the target hardware; with it, the verification proves in 14.25 minutes (mean of three fixed-config runs, 14.05–14.61; Section 7) on SP1 at $\lambda = 80$, under the documented reduced-memory configuration, default settings exhausting memory (Section 7). Two of the measurements falsify natural predictions. First, the bespoke algebraic-hash precompile is *slower* than a software SHA-3 control at matched security in every run we took, on both the May and the June builds. The sign is robust; under fixed-config replication ($n = 3$ per arm) the magnitude is $\approx 1.09\times$ in prove wall-clock (14.25 versus 13.05 minutes), an earlier $2.45\times$ single run retired as machine-state contamination. Arithmetising a 199-bit-field algebraic hash through a custom chip costs more than a hash the substrate already supports. The trace-area accounting of Section 4 postdicts the sign of this inversion from the chips’ committed dimensions: the custom chip adds roughly 5.6×10^7 committed trace cells per verification (a count derived from the chip’s committed dimensions, not a logged trace readout) that the control never pays, a term the guest-cycle accounting alone does not carry. Second, against the transparent, non-preprocessing Aurora baseline the zkVM’s update-churn advantage never

shows up in wall-clock. The static-side recompile is sub-second ($R_{\text{static}} \approx 0.3$ s, Section 7.8), so what separates the two regimes is the regeneration-and-re-audit *footprint* (Table 8), a cost in engineering and assurance rather than in proving time. A wall-clock flexibility advantage would appear only against a *preprocessing* baseline, which is not what BDEC uses.

The BDEC relations themselves run correctly inside the zkVM: credential generation executes both of its signature verifications to acceptance in 9.4×10^9 guest cycles, and the presentation relation in 1.41×10^{10} (one credential shown) and 1.88×10^{10} (two) cycles, consistent with the additive cost model. On SP1 both relations complete a succinct, non-zero-knowledge prove (credential generation in 26.98 and 31.11 min, presentation in 40.15 and 44.24 min at $k=1$ and 58.14 min at $k=2$; Section 7). No zero-knowledge *proof* of either relation was produced on consumer hardware: on RISC Zero the credential-generation succinct prove terminated in an internal-verifier rejection (RISC Zero 3.0.5; an anomaly under diagnosis, categorically distinct from a resource limit and on which no claim of this thesis rests, Section 7.6.1), and the presentation wrap is blocked by the same recursion-shape provisioning wall.

9.2 Answer to the Research Question

The answer is *not yet*, for a reason more specific than “too slow.” Standalone PLUM verification becomes feasible on consumer hardware, at $\lambda = 80$ on SP1 under tuned memory settings and conditional on Assumption 6.1, once the precompile absorbs the field-mismatch tax. Deployable post-quantum *anonymity* is not reached, because it requires a zero-knowledge wrap that is at once feasible to generate within 24 GB and post-quantum-sound, and neither is provided by the pipelines measured here, for the substrate-specific reasons of the previous subsection. The substrate choice is therefore governed by the deployment-lifecycle decision rule of Section 8: where the verification predicate is fixed and proofs are frequent, the specialised static circuit is preferable; where the predicate, signature, or security parameters evolve over the deployment’s lifetime, the zkVM’s fixed universal relation confines each change to a guest-program edit, provided the change leaves the precompiled primitive suite intact, since a hash-family or field-width change regenerates the Griffin AIR. Against a non-preprocessing baseline its advantage there is a smaller re-audit footprint, not a faster runtime.

9.3 Limitations

The results are bounded in ways we state explicitly. Prove-mode wall-clock is measured at $\lambda = 80$, the largest level at which it is observable within the memory budget and not a deployment security level; the field-multiplication ablation is reported at $\lambda = 128$, while the Griffin-precompile cycle ratio ($56.4\times$) and the BDEC relation cycle counts are execute-mode figures at $\lambda = 80$. The static baseline is Loquat over $\mathbb{F}_{p_{127}}$ rather than same-scheme PLUM, with the per-proof figure measured at zero knowledge disabled and the zero-knowledge-enabled counterpart proving in about 22 minutes (mean of three runs, 21.4–22.9) (Section 7), so the per-proof comparison is indicative and the lifecycle crossover is bounded in form but not located in wall-clock. The standalone PLUM Cell 2 and Cell 3 prove figures are means of three fixed-config runs (ranges reported); the BDEC-relation prove figures are single runs, and the Loquat substrate comparison is three runs per leg (Section 7). The security guarantees are conditional: the constraint-level soundness of the Griffin AIR (Assumption 6.1), the existence of a PLUM signature-transcript simulator (Assumption 6.2), the quantum-random-oracle lifts, and the canonical prime instantiation are carried as open assumptions (Section 6). Of these, Assumption 6.1 conditions

only the positive results, the 14.25-minute feasibility figure and the security-preservation theorem; the negative findings, including both prongs of the obstruction, the SHA-3 inversion, the precompile-free baseline’s memory exhaustion, and the absence of a wall-clock flexibility advantage, would survive or strengthen under an unsound AIR, since the measured Griffin-arm cost would then understate that of a corrected and at-least-as-costly chip (Section 6). Finally, the RISC Zero prototype does not yet bind the receipt journal to the public statement, so the artifact is at present a functional benchmark rather than a statement-bound credential verifier. The wrap’s 24 GB-sufficiency is established at the stage level, each stage fitting the budget (Section 7), not by a completed regenerated-toolchain run, and the gnark wrap figure is a program-independent probe at $n = 1$. The Griffin (Q)ROM instantiation (Assumption 6.3) joins the open assumptions above.

9.4 Future Work

With the Loquat same-scheme substrate comparison now measured (Table 4), the most informative next measurement is the *same-scheme* per-proof comparison for PLUM itself: building a 199-bit $\mathbb{F}_p$ Aurora harness to run PLUM statically, which would test whether the measured $12\text{--}17\times$ Loquat substrate gap transfers to PLUM and locate the crossover that the decision rule currently bounds only in form. The open assumptions are the substantive cryptographic work that remains: Assumption 6.1, whose per-row layer now carries a written proof (Appendix B) and rejection-direction fault-injection coverage for seven of the ten constraint families, by extending the suite through a full-prover harness to the lookup-borne families (round count, memory binding, cross-row threading) and by machine-checking the code-to-constraint transcription, for instance by the method of Arguzz [45]; Assumption 6.2 by constructing a PLUM signature-transcript simulator; and the post-quantum-anonymity goal by a post-quantum-sound zero-knowledge wrap (a STARK- or lattice-based recursive argument) in place of the pairing-based wraps the toolchains currently expose. The immediate engineering step is to commit the public statement to the receipt journal, after which the credential-generation prove anomaly should be re-run with segment-level diagnostics. Confirming the canonical prime with PLUM’s authors would discharge the prime-instantiation premise.

The post-quantum-anonymity obstruction of Section 6.3 is, by that analysis, an engineering gap rather than a fundamental one, and the path through it can be stated precisely. The zero-knowledge the produced receipt lacks can be supplied without a pairing-based wrap, by masking the prover’s own low-degree test rather than wrapping its output: for the FRI/STARK-based prover of RISC Zero by masked FRI [64], and for the multilinear Basfold-style commitment of the SP1 prover by the VEIL compiler [69], both plausibly post-quantum and neither reintroducing a quantum-vulnerable assumption. Both exist as code (VEIL ships in the SP1 fork, as yet unwired); what remains is to wire one into the prove path and to measure whether the masked prover stays within the 24 GB budget, the one quantity this thesis leaves unsettled. Closing the obstruction this way would deliver *computational* post-quantum anonymity in the quantum random-oracle model. It would not, on its own, deliver *everlasting* anonymity, which additionally requires the standard-model statistically-hiding trace commitment of Proposition 6.5 and remains a separate open problem.

9.5 Closing

Long-lived post-quantum identity systems require both cryptographic soundness and operational evolvability, and this thesis measured the price of pursuing both on the same substrate. What it contributes is a mapped frontier. The precompile makes PLUM signature verification at the 80-bit level finite to prove on a personal computer. The deployment-lifecycle separation between the static and zkVM regimes is real, though it surfaces as a re-audit footprint and stays out of the wall-clock numbers. The obstruction to deployable post-quantum anonymity has two prongs that survive together: the proof one can afford is feasible but not anonymous, and the anonymous wrap is not post-quantum, and no amount of additional memory, favourable timing, or resolution of Assumption 6.1 removes either. That obstruction, named precisely and surrounded by its cost picture, is what we hand to the design of the next such system.

Acknowledgement

References

- [1] Z. Z. Li, X. Zhang, H. Cui, J. Zhao, and X. Chen, “Bdec: Enhancing learning credibility via post-quantum digital credentials,” in *Provable and Practical Security: 18th International Conference, ProvSec 2024, Gold Coast, QLD, Australia, September 25–27, 2024, Proceedings, Part II*. Berlin, Heidelberg: Springer-Verlag, 2025, pp. 45–64. [Online]. Available: https://doi.org/10.1007/978-981-96-0957-4_3
- [2] RISC Zero, “RISC Zero zkVM: Notes on zero-knowledge,” Security advisory GHSA-5xgj-pmjj-gw49, 2024. [Online]. Available: <https://github.com/risc0/risc0/security/advisories/GHSA-5xgj-pmjj-gw49>
- [3] L2BEAT, “ZK catalog: RISC Zero,” ZK Catalog; zero-knowledge property assessment citing Habock and Al Kindi, 2025. [Online]. Available: <https://l2beat.com/zk-catalog/risc0>
- [4] South China Morning Post, “India’s fake degrees: hundreds in Singapore, Malaysia, US, Canada left questioning qualifications after Manav Bharti University scandal,” 2021, accessed 2026-06-13. [Online]. Available: <https://www.scmp.com/week-asia/people/article/3123929>
- [5] BBC News, “India students facing Canada removal over fake documents get reprieve,” 2023, accessed 2026-06-13. [Online]. Available: <https://www.bbc.com/news/world-us-canada-65899113>
- [6] Nixon Peabody LLP, “Reported LinkedIn data breach: 700 million users’ data exposed,” 2021, accessed 2026-06-13. [Online]. Available: <https://www.nixonpeabody.com/insights/articles/2021/06/30/reported-linkedin-data-breach-700-million-users-data-exposed>
- [7] European Parliament and Council of the European Union, “Regulation (EU) 2024/1183 amending regulation (EU) no 910/2014 as regards establishing the European Digital Identity Framework,” Official Journal of the European Union, 30 April 2024, 2024, eIDAS 2.0. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1183/oj>
- [8] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” U.S. Department of Commerce, Federal Information Processing Standards Publication (FIPS) 204, 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.204.pdf>
- [9] —, “Stateless Hash-Based Digital Signature Standard,” U.S. Department of Commerce, Federal Information Processing Standards Publication (FIPS) 205, 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.205.pdf>
- [10] C. Jeudy, A. Roux-Langlois, and O. Sanders, “Lattice signature with efficient protocols, application to anonymous credentials,” in *Advances in Cryptology – CRYPTO 2023*, ser. Lecture Notes in Computer Science, vol. 14082. Springer, 2023, pp. 351–383, cryptology ePrint Archive, Paper 2022/509. [Online]. Available: <https://eprint.iacr.org/2022/509>
- [11] X. Zhang, R. Steinfeld, M. F. Esgin, J. K. Liu, D. Liu, and S. Ruj, “Loquat: A SNARK-friendly post-quantum signature based on the legendre PRF with applications in ring and aggregate signatures,” Cryptology ePrint Archive, Paper 2024/868, 2024. [Online]. Available: <https://eprint.iacr.org/2024/868>

-
- [12] T. Feneuil, and M. Rivain, “CAPSS: A framework for SNARK-friendly post-quantum signatures,” Cryptology ePrint Archive, Paper 2025/061, 2025. [Online]. Available: <https://eprint.iacr.org/2025/061>
 - [13] S. Argo, T. Güneysu, C. Jeudy, G. Land, A. Roux-Langlois, and O. Sanders, “Practical post-quantum signatures for privacy,” Cryptology ePrint Archive, Paper 2024/131, 2024. [Online]. Available: <https://eprint.iacr.org/2024/131>
 - [14] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward, “Aurora: Transparent succinct arguments for R1CS,” Cryptology ePrint Archive, Paper 2018/828, 2018. [Online]. Available: <https://eprint.iacr.org/2018/828>
 - [15] T. Dokchitser, and A. Bulkin, “Zero knowledge virtual machine step by step,” Cryptology ePrint Archive, Paper 2023/1032, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1032>
 - [16] J. Bruestle, P. Gafni, and the RISC Zero Team, “RISC Zero zkVM: Scalable, Transparent Arguments of RISC-V Integrity,” RISC Zero, Draft, Aug. 2023, draft, August 11. [Online]. Available: <https://dev.risczero.com/proof-system-in-detail.pdf>
 - [17] Y. Yang, Y. Cheng, H. Tang, G. Yang, B. Zhang, and K. Ren, “SoK: Understanding zkVM: From research to practice,” Cryptology ePrint Archive, Paper 2026/525; to appear, ACM AsiaCCS 2026, 2026. [Online]. Available: <https://eprint.iacr.org/2026/525>
 - [18] X. Zhang, Q. Fu, R. Steinfeld, J. K. Liu, T. H. Yuen, and M. H. Au, “Plum: SNARK-friendly post-quantum signature based on power residue PRFs,” in *Provable and Practical Security: 19th International Conference, ProvSec 2025, Yokohama, Japan, October 10–12, 2025, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2025, pp. 111–129. [Online]. Available: https://doi.org/10.1007/978-981-95-2961-2_6
 - [19] R. Lavin, X. Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari, “A survey on the applications of zero-knowledge proofs,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.00243>
 - [20] Succinct Labs, “SP1 zkVM documentation,” <https://docs.succinct.xyz/docs/sp1/introduction>, 2024, accessed 2026-05-29.
 - [21] L. Grassi, Y. Hao, C. Rechberger, M. Schofnegger, R. Walch, and Q. Wang, “Horst meets fluid-SPN: Griffin for zero-knowledge applications,” in *Advances in Cryptology – CRYPTO 2023, Part III*, ser. Lecture Notes in Computer Science, vol. 14083. Springer, 2023, pp. 573–606, full version: Cryptology ePrint Archive, Paper 2022/403, <https://eprint.iacr.org/2022/403>.
 - [22] National Institute of Standards and Technology, “FIPS 206: FN-DSA, a FALCON-based digital signature standard (forthcoming),” U.S. Department of Commerce, NIST, Federal Information Processing Standards Publication (in preparation), 2025, forthcoming. As of the writing, no public draft had been released: the draft was submitted for internal/Department of Commerce clearance on 28 August 2025 and an Initial Public Draft (IPD) had not yet been published. Status presented at the Sixth NIST PQC Standardization Conference, 25 September 2025. [Online]. Available: <https://csrc.nist.gov/pubs/fips/206>

-
- [23] G.-V. Policharla, B. Westerbaan, A. Faz-Hernández, and C. A. Wood, “Post-quantum privacy pass via post-quantum anonymous credentials,” Cryptology ePrint Archive, Paper 2023/414, 2023, presented at Real World Crypto 2023 (non-archival). [Online]. Available: <https://eprint.iacr.org/2023/414>
- [24] M. Chase, D. Derler, S. Goldfeder, C. Orlandi, S. Ramacher, C. Rechberger, D. Slamanig, and G. Zaverucha, “Post-quantum zero-knowledge and signatures from symmetric-key primitives,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1825–1842, cryptology ePrint Archive, Report 2017/279. [Online]. Available: <https://eprint.iacr.org/2017/279>
- [25] C. Baum, C. D. de Saint Guilhem, D. Kales, E. Orsini, P. Scholl, and G. Zaverucha, “Banquet: Short and fast signatures from aes,” in *Public-Key Cryptography – PKC 2021*, ser. Lecture Notes in Computer Science, vol. 12710. Springer, 2021, pp. 266–297, cryptology ePrint Archive, Report 2021/068. [Online]. Available: <https://eprint.iacr.org/2021/068>
- [26] C. D. de Saint Guilhem, E. Orsini, and T. Tanguy, “Limbo: Efficient zero-knowledge mpcith-based arguments,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2021, pp. 3022–3036. [Online]. Available: <https://doi.org/10.1145/3460120.3484595>
- [27] X. Zhang, Z. Li, R. Steinfeld, R. K. Zhao, J. K. Liu, and T. H. Yuen, “Pegasus and PegaRing: Efficient (ring) signatures from sigma-protocols for power residue PRFs with (Q)ROM security,” Cryptology ePrint Archive, Paper 2025/1841, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1841>
- [28] T. Feneuil, and M. Rivain, “CAPSS: A framework for SNARK-friendly post-quantum signatures,” Cryptology ePrint Archive, Paper 2025/061, 2025. [Online]. Available: <https://eprint.iacr.org/2025/061>
- [29] D. Chaum, “Security without identification: Transaction systems to make big brother obsolete,” *Communications of the ACM*, vol. 28, no. 10, pp. 1030–1044, 1985. [Online]. Available: <https://doi.org/10.1145/4372.4373>
- [30] J. Camenisch, and A. Lysyanskaya, “Signature schemes and anonymous credentials from bilinear maps,” in *Advances in Cryptology – CRYPTO 2004*, ser. Lecture Notes in Computer Science, vol. 3152. Springer, 2004, pp. 56–72.
- [31] D. Pointcheval, and O. Sanders, “Short randomizable signatures,” in *Topics in Cryptology – CT-RSA 2016*, ser. Lecture Notes in Computer Science, vol. 9610. Springer, 2016, pp. 111–126, cryptology ePrint Archive, Paper 2015/525. [Online]. Available: <https://eprint.iacr.org/2015/525>
- [32] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, “Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers,” in *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2019. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/coconut-threshold-issuance-selective-disclosure-credentials-with-applications-to-distributed-ledgers/>

-
- [33] D. Boneh, X. Boyen, and H. Shacham, “Short group signatures,” in *Advances in Cryptology – CRYPTO 2004*, ser. Lecture Notes in Computer Science, vol. 3152. Springer, 2004, pp. 41–55, cryptology ePrint Archive, Paper 2004/174. [Online]. Available: <https://eprint.iacr.org/2004/174>
 - [34] World Wide Web Consortium, “Verifiable credentials data model v2.0,” W3C, W3C Recommendation, May 2025, w3C Recommendation, 15 May 2025. [Online]. Available: <https://www.w3.org/TR/vc-data-model-2.0/>
 - [35] E. Ben-Sasson, A. Chiesa, and N. Spooner, “Interactive oracle proofs,” in *Theory of Cryptography – TCC 2016-B, Part II*, ser. Lecture Notes in Computer Science, vol. 9986. Springer, 2016, pp. 31–60, cryptology ePrint Archive, Paper 2016/116. [Online]. Available: <https://eprint.iacr.org/2016/116>
 - [36] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast Reed–Solomon interactive oracle proofs of proximity,” in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, 2018.
 - [37] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, “STIR: Reed–solomon proximity testing with fewer queries,” Cryptology ePrint Archive, Paper 2024/390, 2024. [Online]. Available: <https://eprint.iacr.org/2024/390>
 - [38] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf, “Proximity gaps for reed–solomon codes,” Cryptology ePrint Archive, Paper 2020/654, 2020. [Online]. Available: <https://eprint.iacr.org/2020/654>
 - [39] A. Chiesa, D. Ojha, and N. Spooner, “Fractal: Post-quantum and transparent recursive proofs from holography,” Cryptology ePrint Archive, Paper 2019/1076, 2019. [Online]. Available: <https://eprint.iacr.org/2019/1076>
 - [40] A. Chiesa, Y. Hu, M. Maller, P. Mishra, P. Vesely, and N. P. Ward, “Marlin: Preprocessing zkSNARKs with universal and updatable SRS,” in *Advances in Cryptology – EUROCRYPT 2020*, ser. Lecture Notes in Computer Science, vol. 12105. Springer, 2020, pp. 738–768, cryptology ePrint Archive, Paper 2019/1047. [Online]. Available: <https://eprint.iacr.org/2019/1047>
 - [41] A. Arun, S. Setty, and J. Thaler, “Jolt: SNARKs for virtual machines via lookups,” Cryptology ePrint Archive, Paper 2023/1217, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1217>
 - [42] N. Kaskov, M. Komarov, and M. Aksenov, “zkLLVM Circuit Compiler: A compiler from high-level programming languages into provable-computations protocol input representations,” =nil; Foundation, Technical Report, Apr. 2024, accessed April 6, 2024. [Online]. Available: https://cms.nil.foundation/uploads/zkllvm_a27f4af786.pdf
 - [43] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, “Circom: A circuit description language for building zero-knowledge applications,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4733–4751, 2023.
 - [44] RISC Zero, “R0VM 2.0: A GPU-accelerated, formally-verified zkVM,” RISC Zero technical announcement, Apr. 2025, reports Ethereum block proving reduced from ≈ 35 min to ≈ 44 s; accessed 2026-05-31. [Online]. Available: <https://risczero.com/blog>

- [45] C. Hochrainer, V. Wüstholtz, and M. Christakis, “Arguzz: Testing zkVMs for soundness and completeness bugs,” arXiv preprint arXiv:2509.10819, 2025. [Online]. Available: <https://arxiv.org/abs/2509.10819>
- [46] J. Bootle, V. Lyubashevsky, N. K. Nguyen, and A. Sorniotti, “A framework for practical anonymous credentials from lattices,” in *Advances in Cryptology – CRYPTO 2023*, ser. LNCS, vol. 14082. Springer, 2023, pp. 384–417, iACR ePrint 2023/560.
- [47] S. Argo, T. Güneysu, C. Jeudy, G. Land, A. Roux-Langlois, and O. Sanders, “Practical post-quantum signatures for privacy,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS ’24)*. ACM, 2024, pp. 1523–1537, full version: IACR ePrint 2024/131.
- [48] M. Rosenberg, J. White, C. Garman, and I. Miers, “zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure,” in *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2023, iACR ePrint 2022/878.
- [49] StarkWare, “S2morrow: STARK-based signature aggregation for Falcon and SPHINCS+,” Non-peer-reviewed engineering artifact, <https://github.com/starkware-bitcoin/s2morrow>, 2025, falcon-512 and SPHINCS+ verified in the Cairo zkVM.
- [50] Saygan, Gündoğan, Arslan, and Gönen, “HAPPIER: Hash-based, aggregatable, practical post-quantum signatures implemented efficiently with Risc0,” in *Lightweight Cryptography for Security and Privacy (LightSec 2025)*, ser. LNCS, vol. 16216. Springer, 2026, pp. 3–22.
- [51] Kota, “Building a zero-knowledge verifier for Dilithium signatures: NTT gadget implementation in SP1 zkVM,” Non-peer-reviewed engineering write-up, Medium, 2025-12-08, <https://medium.com/@phillyj1026/building-a-zero-knowledge-verifier-for-dilithium-signatures-ntt-gadget-implementation-in-sp1-6c50ab26283> 2025, mL-DSA verification in SP1 via a guest-code NTT gadget; proofs via the SP1 prover network.
- [52] L. Grassi, D. Khovratovich, and M. Schofnegger, “Poseidon2: A faster version of the poseidon hash function,” in *Progress in Cryptology – AFRICACRYPT 2023*, ser. Lecture Notes in Computer Science, vol. 14064. Springer Nature Switzerland, 2023, pp. 177–203, cryptology ePrint Archive, Paper 2023/323. [Online]. Available: <https://eprint.iacr.org/2023/323>
- [53] U. Haböck, “Multivariate lookups based on logarithmic derivatives,” Cryptology ePrint Archive, Paper 2022/1530, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1530>
- [54] A. Fiat, and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in Cryptology – CRYPTO ’86*, ser. Lecture Notes in Computer Science, vol. 263. Springer, 1987, pp. 186–194.
- [55] A. Bariant, A. Boeuf, A. Lemoine, I. M. Ayala, M. Øyegarden, L. Perrin, and H. Raddum, “The algebraic FreeLunch: Efficient Gröbner basis attacks against arithmetization-oriented primitives,” Cryptology ePrint Archive, Paper 2024/347 (CRYPTO 2024), 2024. [Online]. Available: <https://eprint.iacr.org/2024/347>

-
- [56] A. Bariant, A. Boeuf, P. Briaud, M. Hostettler, M. Øyegarden, and H. Raddum, “Improved resultant attack against arithmetization-oriented primitives,” Cryptology ePrint Archive, Paper 2025/259 (CRYPTO 2025), 2025. [Online]. Available: <https://eprint.iacr.org/2025/259>
- [57] A. Bak, A. Bariant, A. Boeuf, P. Briaud, M. Øyegarden, and A. Phanse, “The algebraic CheapLunch: Extending FreeLunch attacks on arithmetization-oriented primitives beyond CICO-1,” Cryptology ePrint Archive, Paper 2025/2040, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2040>
- [58] Succinct Labs, “Security model — SP1 documentation,” SP1 Documentation, 2025, accessed 2026-06-01. [Online]. Available: <https://docs.succinct.xyz/docs/sp1/security/security-model>
- [59] A. Chiesa, P. Manohar, and N. Spooner, “Succinct arguments in the quantum random oracle model,” Cryptology ePrint Archive, Paper 2019/834 (TCC 2019), 2019. [Online]. Available: <https://eprint.iacr.org/2019/834>
- [60] A. Chiesa, Z. Di, Z. Hu, and Y. Zheng, “How to prove post-quantum security for succinct non-interactive reductions,” Cryptology ePrint Archive, Paper 2025/2166, 2026, cryptology ePrint Archive, Paper 2025/2166. [Online]. Available: <https://eprint.iacr.org/2025/2166>
- [61] R. Canetti, O. Goldreich, and S. Halevi, “The random oracle methodology, revisited,” Journal of the ACM 51(4):557–594 (Cryptology ePrint Archive, Paper 1998/011), 2004. [Online]. Available: <https://eprint.iacr.org/1998/011>
- [62] I. Haitner, J. J. Hoch, O. Reingold, and G. Segev, “Finding collisions in interactive protocols: A tight lower bound on the round complexity of statistically-hiding commitments,” FOCS 2007 (ECCC TR07-038), 2007. [Online]. Available: <https://eccc.weizmann.ac.il/report/2007/038/>
- [63] H. Wee, “One-way permutations, interactive hashing and statistically hiding commitments,” Theory of Cryptography (TCC) 2007, LNCS 4392, pp. 419–433, 2007. [Online]. Available: <https://www.iacr.org/archive/tcc2007/43920418/43920418.pdf>
- [64] U. Haböck, and A. Kindi, “A note on adding zero-knowledge to STARK,” Cryptology ePrint Archive, Paper 2024/1037, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1037>
- [65] C. Baum, I. Damgård, V. Lyubashevsky, S. Oechsner, and C. Peikert, “More efficient commitments from structured lattice assumptions,” Security and Cryptography for Networks (SCN) 2018 (Cryptology ePrint Archive, Paper 2016/997), 2018. [Online]. Available: <https://eprint.iacr.org/2016/997>
- [66] Succinct Labs, “Proof types — SP1 documentation,” SP1 Documentation, 2025, accessed 2026-06-01. [Online]. Available: <https://docs.succinct.xyz/docs/sp1/generating-proofs/proof-types>
- [67] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology – EUROCRYPT 2016*, ser. Lecture Notes in Computer Science, vol. 9666. Springer, 2016, pp. 305–326, cryptology ePrint Archive, Paper 2016/260. [Online]. Available: <https://eprint.iacr.org/2016/260>

- [68] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge,” Cryptology ePrint Archive, Paper 2019/953, 2019. [Online]. Available: <https://eprint.iacr.org/2019/953>
- [69] R. Dalal, T. Hemo, E. Rabinovich, and R. Rothblum, “VEIL: Lightweight zero-knowledge for hash-based multilinear proof systems,” Cryptology ePrint Archive, Paper 2026/683, 2026. [Online]. Available: <https://eprint.iacr.org/2026/683>
- [70] StarkWare Team, “ethSTARK documentation,” Cryptology ePrint Archive, Paper 2021/582, 2021. [Online]. Available: <https://eprint.iacr.org/2021/582>
- [71] J. Avigad, L. Goldberg, D. Levit, Y. Seginer, and A. Titelman, “A verified algebraic representation of Cairo program execution,” in *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 2022, <https://arxiv.org/abs/2109.14534>.
- [72] Kardas, Kiraz, Savonin, Wang, and Dziadziuk, “Performance improvements of ZK-prover for rWasm: A sound and efficient AIR for 32-bit division and remainder,” Cryptology ePrint Archive, Paper 2025/2220, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2220>

A Formal Development of the Security Preservation

The body (Section 6) states the security-preservation theorem (Theorem 6.3) and sketches its proof. This appendix gives the formal development for audit: the building-block properties the reduction treats as black boxes, each carrying an error term; the relation-preservation lemma; the step-by-step reductions; a corollary for the case in which the open premises are closed; and a remark relating the result to BDEC. Throughout, $\mathcal{A}$ denotes a classical probabilistic polynomial-time adversary and all probabilities are in the random-oracle model. The advantages $\text{Adv}_{\text{zkVM-BDEC}}^\bullet$ are those of Definition 6.1.

A.1 Building-block properties

Each property is stated as the existence of an algorithm together with an error term. The premises of Theorem 6.3 assert the relevant errors negligible; we mark which properties are assumed and which carry an open premise.

Definition A.1 (Knowledge soundness). The zkVM argument Π is *knowledge-sound with knowledge error* ϵ_{KS} if there is a probabilistic polynomial-time extractor $\mathcal{E}$ with oracle access to the prover such that, for every prover P^* that makes $\Pi.\text{Verify}(x, \cdot)$ accept with probability ε , the extractor $\mathcal{E}^{P^*}(x)$ outputs a witness w with $R(x; w) = 1$ with probability at least $\varepsilon - \epsilon_{\text{KS}}$. This is the standard knowledge-soundness-with-error formulation; ϵ_{KS} is the additive extraction-failure term that enters the unforgeability bound (3), and the concrete soundness error of the STARK and BCS arguments we rely on is treated in §B.5. (Assumed; classical random-oracle model.)

Definition A.2 (Arithmetisation soundness error).

$$\epsilon_{\text{AIR}} = \Pr[\text{the Griffin AIR accepts a trace whose committed transition differs from } \pi],$$

the probability taken over the prover-chosen trace and the argument verifier’s randomness, where π is the reference permutation of Definition B.1. Assumption 6.1 asserts ϵ_{AIR} negligible; the supporting argument is Appendix B.

Definition A.3 (Signature-transcript simulatability). PLUM *admits a signature-transcript simulator with error* ϵ_{sZK} if there is a probabilistic polynomial-time algorithm $\mathcal{S}_\sigma$ such that, for every message m and public key, the output of $\mathcal{S}_\sigma$ run without the secret key is ϵ_{sZK} -indistinguishable from a credential or pseudonym transcript produced with the secret key. Assumption 6.2 asserts ϵ_{sZK} negligible. This assumption is *open*: PLUM’s analysis establishes EU-CMA security, which does not by itself furnish such a simulator.

Definition A.4 (Zero knowledge). The zkVM argument Π is *zero-knowledge with error* ϵ_{ZK} if there is a probabilistic polynomial-time simulator $\mathcal{S}_\Pi$ such that, for every (x, w) with $R(x; w) = 1$, the simulated proof $\mathcal{S}_\Pi(x)$ is ϵ_{ZK} -indistinguishable from an honest proof $\Pi.\text{Prove}(x, w)$. (Assumed for the analysis. In the measured deployment this premise is not met: the produced receipt mode (the default `prove` path) has no established ZK simulator, and the ZK-wrapped modes (the `groth16()` and `plonk()` proof modes, which compress the STARK internally before wrapping) are not produced on the target hardware as the toolchain ships, blocked by the recursion-shape provisioning wall; see Section 6.3.)

Definition A.5 (Existential unforgeability). PLUM is *EU-CMA secure with error* ϵ_{EUF} if every probabilistic polynomial-time forger, given the public key and a signing oracle, produces a valid signature on a message it never queried with probability at most ϵ_{EUF} . (Established in PLUM’s analysis, [18, Theorem 1], in the classical random-oracle model.)

A.2 Relation preservation

Proof of Lemma 6.2. We account for the operations a guest evaluation invokes. The credential-generation relation $R_{\text{cre}}^{\text{Sig}}$, two PLUM verifications under pk_U , decomposes into large-field multiplications, returned identically by the modular-multiplication precompile, and Griffin permutations inside PLUM’s hashing and inside every Merkle-path and low-degree-test recomputation, returned identically by the Griffin precompile under the lookup binding, together with native control flow, whose faithful execution follows from the *soundness* of the zkVM argument for the RISC-V execution relation (an accepting proof certifies a valid execution); the stronger *knowledge-soundness* (Definition A.1) that extracts a witness is invoked separately, by the unforgeability reduction. For $R_{\text{cre}}^{\text{Sig}}$ this enumeration is complete at the predicate level, conditional on the open statement-binding refinement of Section 5 and on Assumption 6.1. The presentation relation $R_{\text{show}}^{\text{Sig}}$ adds only $k + 2$ further signature-verification predicates built from the same multiplications and Griffin permutations; its ledger-membership, non-revocation, and disclosure checks are performed by the relying party outside the arithmetised relation. Composing the per-operation equalities along the finite, input-fixed execution yields equality of the accept-or-reject decision at the program level. This program-level equality lifts to credential-verifier acceptance only when the zkVM journal commits the public statement x ; the specification assumes this binding, whereas the current RISC Zero prototype commits the verification outcome but not x_{cre} itself (Section 5), so the code-level reading is conditional on that journal fix. The Griffin step’s constraint-level soundness rests on Appendix B, which is argued rather than machine-verified, so the equality holds modulo Assumption 6.1. $\square$

A.3 Proof of the preservation theorem

We re-instantiate BDEC’s three reductions [1] with PLUM as the signature scheme Sig and the zkVM as the zkSNARK Π ; by Lemma 6.2 the two provers accept the same relation.

Proof of Theorem 6.3, unforgeability. Let $\mathcal{A}$ win the zkVM-BDEC unforgeability experiment. We build a PLUM EU-CMA forger $\mathcal{B}$. $\mathcal{B}$ runs $\mathcal{A}$, answering its issuance and showing queries with its own signing oracle and the zkVM prover, so $\mathcal{A}$ ’s view matches the real experiment. When $\mathcal{A}$ outputs an accepting (x^*, π^*) for a statement outside the issued set, $\mathcal{B}$ invokes the extractor $\mathcal{E}$ of Definition A.1 to obtain $w^* = (\text{pk}_U^*, \text{psk}^*)$, which fails with probability at most ϵ_{KS} . Conditioned on extraction, w^* satisfies $R_{\text{cre}}^{\text{Sig}}$ except when the committed Griffin transition differs from the reference permutation, an event of probability at most ϵ_{AIR} by Definition A.2; otherwise the extracted psk^* (the pseudonym secret, structurally a PLUM signature under pk_U , Section 3) is valid on a message $\mathcal{A}$ never had signed, and by the black-box-transfer hypothesis the precompile substitution leaves PLUM’s EU-CMA reduction unchanged, so psk^* is a forgery against the unmodified scheme, which $\mathcal{B}$ outputs. Writing ϵ_{EUF} for PLUM’s EU-CMA advantage bound (Definition A.5),

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{unf}}(\mathcal{A}) \leq \epsilon_{\text{EUF}} + \epsilon_{\text{KS}} + \epsilon_{\text{AIR}},$$

which is (3). $\square$

Proof of Theorem 6.3, anonymity. Let $\mathcal{A}$ be the anonymity distinguisher, with advantage $|\Pr[b' = b] - \frac{1}{2}|$. We proceed through a sequence of games on the challenge showing for the credential cred_b . Game G_0 is the real experiment. In game G_1 the pseudonym key and credential are produced by $\mathcal{S}_\sigma$ rather than with the secret key, so by Definition A.3 $|\Pr[G_0=1] - \Pr[G_1=1]| \leq \epsilon_{\text{sZK}}$. The role of $\mathcal{S}_\sigma$ is to let the reduction produce the challenge credential c_b and pseudonym signature psk_b without the challenge user's long-term secret key sk_U , matching BDEC's anonymity simulator ([1, Appendix A.2]); the hop is load-bearing whenever PLUM's transcripts cannot be produced without sk_U . In game G_2 the proof is produced by $\mathcal{S}_\Pi$ rather than from the witness, so by Definition A.4 $|\Pr[G_1=1] - \Pr[G_2=1]| \leq \epsilon_{\text{ZK}}$. In G_2 we account for every b -dependent component of the challenge statement $x_{\text{show}} = (A^\downarrow, (\text{ppk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V})$. The disclosed attribute set $A^\downarrow$ is equalised across the two credentials by the well-formedness condition $\text{Leak}(\text{stmt}_0) = \text{Leak}(\text{stmt}_1)$. The verifier-facing pseudonym $\text{ppk}_{U,V}$ is sampled freshly at showing time, $\text{ppk}_{U,V} \leftarrow \{0, 1\}^\lambda$, independently of which credential is shown, so its distribution is uniform regardless of b . The teaching-authority pseudonyms $(\text{ppk}_{U,TA}^{(j)})_j$ are issuance-time, credential-bound values that the well-formedness condition equalises across the two challenge credentials, so Leak includes the multiset $\{\text{ppk}_{U,TA}^{(j)}\}$ (Section 3). The secret keys, the long-term key pk_U , and the credential $c_{U,V}$ are witness-only: they were replaced by $\mathcal{S}_\sigma$ in G_1 and removed from the proof by $\mathcal{S}_\Pi$ in G_2 . Every b -dependent component being equalised, freshly sampled, or simulated, the challenge is distributed identically under $b = 0$ and $b = 1$, so $\Pr[b' = b] = \frac{1}{2}$. Summing the two hops,

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{anon}}(\mathcal{A}) \leq \epsilon_{\text{ZK}} + \epsilon_{\text{sZK}},$$

which is (4). No witness is extracted, so ϵ_{KS} does not enter; the challenge showing is honestly generated, so ϵ_{AIR} does not enter. $\square$

Proof of Theorem 6.3, unlinkability. BDEC's unlinkability experiment ([1, Appendix A.3], Theorem 3) forwards a single challenge credential $c^{(b)}$ issued under one of two pseudonym keys $\text{ppk}^{(b)}$, with the verifier-facing pseudonym sampled freshly as in the anonymity step, and the distinguisher $\mathcal{A}$ guesses b with advantage $|\Pr[b' = b] - \frac{1}{2}|$. The reduction mirrors the anonymity proof, with the challenge bit selecting the pseudonym key rather than the credential. Game G_0 is the real experiment. In G_1 the challenge pseudonym signature under $\text{psk}^{(b)}$ is produced by $\mathcal{S}_\sigma$ in place of the secret key, so by Definition A.3 $|\Pr[G_0=1] - \Pr[G_1=1]| \leq \epsilon_{\text{sZK}}$, applied once to the single challenge transcript. In G_2 the proof is produced by $\mathcal{S}_\Pi$ in place of the witness, so by Definition A.4 $|\Pr[G_1=1] - \Pr[G_2=1]| \leq \epsilon_{\text{ZK}}$. In G_2 the only b -dependent component is the pseudonym signature, now simulated without $\text{psk}^{(b)}$, and the verifier-facing pseudonym is independent of b , so $\Pr[b' = b] = \frac{1}{2}$. Summing the two hops,

$$\text{Adv}_{\text{zkVM-BDEC}}^{\text{unlink}}(\mathcal{A}) \leq \epsilon_{\text{ZK}} + \epsilon_{\text{sZK}},$$

the same bound as (4); the single-transcript game incurs each simulation once, with no factor of two. One difference of presentation from BDEC's A.3: there $c^{(b)}$ is a public-statement component, so its $\mathcal{S}_\sigma$ hop is forced, whereas in this thesis's relation $R_{\text{show}}^{\text{Sig}}$ the credential $c_{U,V}$ is a witness component (Section 3), so the $\mathcal{S}_\Pi$ hop already hides it and $\mathcal{S}_\sigma$ is needed only to produce $c^{(b)}$ without sk_U inside the reduction. Either way ϵ_{sZK} enters once. $\square$

Corollary A.6 (Preservation under closed assumptions). *Suppose Assumptions 6.1 and 6.2 hold, so that ϵ_{AIR} and ϵ_{sZK} are negligible; the zkVM is knowledge-sound and zero-knowledge with negligible $\epsilon_{\text{KS}}, \epsilon_{\text{ZK}}$; and PLUM is EU-CMA secure. Then zkVM-BDEC is unforgeable, anonymous, and unlinkable. Against quantum adversaries the same conclusion holds once the quantum-random-oracle lifts of PLUM’s EU-CMA reduction and of the nested Fiat–Shamir compilation are established.*

Proof. Each bound of Theorem 6.3 is a sum of the named error terms; under the stated hypotheses every term is negligible, so each advantage is negligible. The quantum clause adds the two open lifts as further hypotheses. $\square$

Remark A.7 (Relation to BDEC). Theorem 6.3 re-instantiates BDEC’s Theorems 1–3, proved for a generic (signature, zkSNARK) pair, with the concrete pair (PLUM, zkVM). The contribution is not a new reduction but the explicit accounting of the substrate-specific errors ϵ_{AIR} (the precompile’s arithmetisation soundness) and $\epsilon_{\text{KS}}, \epsilon_{\text{ZK}}$ (the zkVM’s argument), together with the identification of ϵ_{sZK} as the open Assumption 6.2 that BDEC’s generic anonymity argument silently requires of the signature. We further note that the relation $R_{\text{show}}^{\text{Sig}}$ places the credential $c_{U,V}$ in the witness rather than, as in BDEC’s literal layout, the public statement; the result is therefore a re-derivation of BDEC’s Theorems 1–3 for this concrete pair, not a verbatim re-instantiation.

B Evidence for Assumption 6.1: Specification-Level Soundness of the Griffin- $\mathbb{F}_p^{192}$ AIR

This appendix develops the soundness argument for the Griffin- $\mathbb{F}_p^{192}$ permutation AIR of Section 5, on which the conditional precompile-soundness theorem (Theorem 5.1) and the relation-preservation lemma of Section 6 rest. We follow the standard practice for arithmetisation correctness. As in ethSTARK [70] (Definitions 3 and 4), the verified Cairo representation [71], and per-gadget AIR-soundness arguments such as [72], the formal object is a constraint-soundness statement, a universally quantified implication over satisfying traces, together with a separate completeness statement, while the knowledge-soundness of the surrounding argument is treated apart in Section B.5. The malicious prover is captured by the quantifier “for every trace that satisfies the constraints”, so the statement uses no challenger, oracle, or advantage function. The one component that is supported empirically rather than proved, namely that the implemented chip realises the constraint set analysed here, appears as the explicit premise of the soundness theorem and is discussed in Section B.6.

B.1 Setup

Definition B.1 (Reference permutation). Let $\pi : \mathbb{F}_p^4 \rightarrow \mathbb{F}_p^4$ denote one application of the Griffin permutation at PLUM’s parameters, a degree-3 S-box over a width-4 state with 14 rounds and the SHAKE256-derived round constants and base quadratic coefficients (α, β) , scaled per lane by Griffin’s index law, fixed in Section 3, as computed by the reference implementation. The reference permutation follows the implementation’s in-place nonlinear layer, which deviates from the published Griffin definition in the lane-3 combination (y_2 in place of x_2 ; Section 3); the lemmas below establish chip–reference equality, and conformance of the reference to the published design is the separate obligation recorded there.

Definition B.2 (Algebraic satisfiability). The Griffin AIR is a finite constraint set $\mathcal{C}$ over a trace w comprising 14 per-round rows and one controller row. $\mathcal{C}$ comprises (i) the constraint polynomials Q_i , each with an enforcement domain H_i ; (ii) the range conditions on limb columns (operand, result, and carry limbs in $[0, 2^8)$, offset witness limbs in $[0, 2^{16})$); and (iii) the multiset-balance conditions of the lookup interactions (the byte-table sends, the round-to-round threading, and the memory binding). In the chip, (ii) and (iii) are realised by the byte-table and cross-table lookup arguments, whose soundness is the lookup-binding hypothesis; a prover for whom the lookup argument fails to bind can violate them while leaving every Q_i vanishing. A trace w satisfies $\mathcal{C}$ on input $s \in \mathbb{F}_p^4$ with claimed output $s' \in \mathbb{F}_p^4$ if its first-row input lanes decode to s , its last-row output lanes decode to s' , and all of (i)–(iii) hold under w .

B.2 Soundness

Theorem B.3 (Specification-level AIR soundness). *Suppose the implemented chip, comprising the per-round chip together with its sibling controller chip, enforces the constraint set $\mathcal{C}$ analysed below. Then for every input $s \in \mathbb{F}_p^4$ and every trace w satisfying $\mathcal{C}$ (Definition B.2) with claimed output s' , we have $s' = \pi(s)$. Equivalently, $\mathcal{C}$ admits no satisfying trace whose committed output differs from the reference permutation.*

Proof. By Lemmas B.4, B.5, and B.6. Lemma B.4 bridges the limb identities the chip actually constrains over its small base field to congruences modulo p ; Lemma B.5 composes those congruences, for each row, into the reference round on the row’s input lanes; and Lemma B.6 forces the 14 rows to compose as 14 sequential rounds on a single state, bound on the controller row to the syscall’s memory input and output. Composing the per-round equalities along that schedule yields $s' = \pi(s)$. The per-row layer (Lemmas B.4 and B.5) is unconditional given the range predicate, which the byte-table lookups discharge under the lookup-binding hypothesis; the cross-row layer (Lemma B.6) is lookup-borne in its scheduling, threading, and memory-binding steps (round count, memory binding, row scheduling), while output canonicity is per-row algebra and parameter immutability holds by preprocessing. The status of the chip–constraint equality (Assumption 6.1) is discussed in Section B.6. $\square$

B.3 Per-family lemmas

The ten constraint families group into the round function (the round-function families) and the schedule, binding, and canonicity apparatus (the schedule-and-binding families). The round-function families admit a proof at the row level, given below. It is useful to read it adversarially: the prover chooses every cell of a row, all 3,819 columns, subject only to the constraint polynomials and the range checks, and the lemmas state that no choice decodes to an input–output pair outside the graph of the reference round. The proof requires no computational assumption. Of the assumption-bearing steps in the chip’s soundness, the round-to-round threading and the memory binding are collected in Lemma B.6; the byte-table range lookups enter as the range-predicate hypothesis of Lemmas B.4 and B.5; all are carried by the lookup-binding hypothesis.

The constraints are not enforced over $\mathbb{F}_p$ directly: the chip constrains 32-limb byte representations over the prover’s base field, and the bridge from limb identities over the small field to congruences modulo the 199-bit p is itself a proof obligation. We discharge it first. Throughout, q denotes the prover’s base-field modulus (KoalaBear, $q = 2^{31} - 2^{24} + 1 = 2,130,706,433$, in the deployed SP1

fork), $\text{int}(\cdot)$ decodes a limb vector as an integer in base 2^8 , and the *range predicate* of a field-operation cell asserts that its operand, result, and carry limbs lie in $[0, 2^8)$ and its offset witness limbs in $[0, 2^{16})$. In the chip these bounds are enforced by byte-table lookups, so the range predicate is exactly the part of a cell's soundness carried by the lookup-binding hypothesis.

Lemma B.4 (Limb-gadget soundness). *Consider one field-operation cell for $\circ \in \{+, \times\}$ with operand limb vectors a, b , result limbs c , carry limbs h , and witness limbs w stored with offset 2^{14} , all satisfying the range predicate. If the cell's vanishing identity*

$$p_{\text{op}}(x) - p_c(x) - p_h(x) p_{\bar{p}}(x) = (p_w(x) - 2^{14} \mathbb{K}(x)) (x - 2^8)$$

holds coefficient-wise over $\mathbb{F}_q$, where $p_{\text{op}} = p_a p_b$ for $\times$ and $p_a + p_b$ for $+$, $p_{\bar{p}}$ is the limb polynomial of the Griffin modulus p , and $\mathbb{K}$ is the all-ones polynomial, then

$$\text{int}(a) \circ \text{int}(b) = \text{int}(c) + \text{int}(h) \cdot p \quad \text{over } \mathbb{Z}, \quad \text{hence} \quad \text{int}(c) \equiv \text{int}(a) \circ \text{int}(b) \pmod{p}.$$

The same conclusion holds for the multi-operand linear cells used by the linear layer, whose p_{op} is a fixed integer combination of up to four operand polynomials with coefficient sum 5.

Proof. Under the range predicate, each coefficient of the difference of the two sides is an integer of absolute value at most

$$\underbrace{32 \cdot 255^2}_{p_a p_b} + \underbrace{255}_{p_c} + \underbrace{32 \cdot 255^2}_{p_h p_{\bar{p}}} + \underbrace{(2^{16} - 2^{14})(2^8 + 1)}_{\text{offset witness, two shifts}} = 16,793,919 < q,$$

where 32 bounds the overlap of two 32-limb polynomials; the additive and linear-combination cases are smaller. An integer that vanishes in $\mathbb{F}_q$ and has absolute value below q is zero, so the identity holds in $\mathbb{Z}[x]$. Evaluating at $x = 2^8$ annihilates the right-hand side and evaluates each limb polynomial to its integer, giving $\text{int}(a) \circ \text{int}(b) - \text{int}(c) - \text{int}(h) p = 0$. $\square$

The lemma concludes a congruence, not canonicity: $\text{int}(c)$ is bounded by 2^{256} through its byte limbs but not by p , and the chip indeed admits non-canonical representatives in intermediate cells. Canonicity is restored where output canonicity and the hardening checks on the nonlinear-output cells impose it, which the next lemma makes explicit.

Lemma B.5 (Per-row round soundness: the round-function and canonicity families). *Fix a real row with round index $r \in \{0, \dots, 13\}$, input-lane limbs $s = (s_0, s_1, s_2, s_3)$, and claimed output-lane limbs $s' = (s'_0, \dots, s'_3)$. Suppose the row satisfies, over $\mathbb{F}_q$, the per-row constraints of the round-function and canonicity families and the per-row row-scheduling selectors, and that every cell satisfies the range predicate. Then each $\text{int}(s'_\ell)$ is the canonical representative of lane ℓ of $\rho_r(s \bmod p)$, where ρ_r is the reference Griffin round with constants $\text{RC}[4r], \dots, \text{RC}[4r+3]$ (zero for $r = 13$), and the $\mathbb{F}_p$ -classes of all intermediate result cells are determined by s .*

Proof. Selectors. The per-row row-scheduling constraints make each is_round_r boolean with $\sum_r \text{is_round}_r = 1$ and $\sum_r r \cdot \text{is_round}_r = \text{round_idx}$ on a real row, so exactly one selector is set, at position r , and the round-constant mux evaluates to the byte-bounded limbs of $\text{RC}[4r+\ell]$.

Forward S-box (lane 1). Lemma B.4 applied to the squaring cell and then the cube cell gives $\text{int}(\text{cube}) \equiv s_1^3 \pmod{p}$; the bitwise binding of the lane-1 nonlinear output to *cube* transfers this congruence to the lane.

Inverse S-box (lane 0). The chip witnesses the nonlinear output s_0^{nl} and checks the backward cube: two applications of Lemma B.4 give $\text{int}(\text{cube}_0) \equiv (s_0^{\text{nl}})^3 \pmod{p}$, and the bitwise binding $\text{cube}_0 = s_0$ yields $(s_0^{\text{nl}})^3 \equiv s_0 \pmod{p}$. Since $\gcd(3, p-1) = 1$ (Remark 3.5), cubing is a bijection on $\mathbb{F}_p$, so the class of s_0^{nl} is the unique cube root $s_0^{1/3}$, the reference inverse S-box.

Quadratic layer (lanes 2, 3). The composite relation $s_i^{\text{nl}} = s_i(L_i^2 + \alpha_{i-2}L_i + \beta_{i-2})$ is enforced as a chain of binary cells (sums building L_i , its square, the α -multiple, the shifted sum, and the final product). Each link satisfies the range predicate, since every cell’s result limbs are byte-checked, so Lemma B.4 applies link by link and the congruences compose to $s_i^{\text{nl}} \equiv s_i(L_i^2 + \alpha_{i-2}L_i + \beta_{i-2}) \pmod{p}$ with $L_2 = s_0^{\text{nl}} + s_1^{\text{nl}}$ and $L_3 = 2s_0^{\text{nl}} + s_1^{\text{nl}} + s_2^{\text{nl}}$, the updated lanes, and with the index law $\alpha_0 = \alpha$, $\beta_0 = \beta$, $\alpha_1 = 2\alpha$, $\beta_1 = 4\beta$ baked into the constant operands; the base pair (α, β) and the 52 round constants are read from SHAKE256 seeded by the domain string PlumGriffin($p, 4, 2, 128$) and enter the constraints as preprocessing-bound constants (parameter immutability), not prover columns.

Linear layer. The four linear cells, by the multi-operand case of Lemma B.4, give $\text{mds}_j \equiv \langle \text{circ}(2, 1, 1, 1)_j, s^{\text{nl}} \rangle \pmod{p}$, where $\text{circ}(2, 1, 1, 1)_j$ is the symmetric circulant row $\text{first}[(j+\text{col}) \bmod 4]$ matching `build_matrix`, not the standard $(\text{col} - j) \bmod 4$ form (the two differ on rows 1, 3).

Round constants. The four addition cells give $\text{int}(s'_\ell) \equiv \text{mds}_\ell + \text{RC}[4r+\ell] \pmod{p}$, the mux having been fixed by the selector step; the preprocessing table holds 56 entries of which the final round’s 4 are zero, so the uniform constraint covers all 14 rounds and degenerates to the bare linear layer on the last.

Canonicity. The bitwise less-than cells enforce $\text{int}(s'_\ell) < p$, so each output lane is the canonical representative of its class.

Composing the five layers, the class of every intermediate result cell is determined by s (lane 0 by bijectivity of cubing, the remaining cells as images of deterministic operations; the carry and witness columns vary with the representatives the prover chooses), and $\text{int}(s'_\ell)$ is the canonical representative of lane ℓ of $\rho_r(s \bmod p)$. $\square$

Lemma B.6 (Composition, binding, canonicity: the schedule-and-binding families). *Under the schedule-and-binding families, every trace satisfying $\mathcal{C}$ realises its 14 rows as the 14 sequential reference rounds on a single state bound to the syscall’s memory input and output, with canonical output lanes.*

Proof. Round count. The schedule is fixed to 14 rounds by the trace-generation row count and the controller’s lookup anchors at directions 0 and NB_ROUNDS, rather than by a dedicated algebraic constraint; under the lookup binding (the lookup-binding hypothesis) a trace with a different round count produces an unbalanced lookup and has no satisfying assignment. *Memory binding*. The controller chip constrains the input and output state lanes to the values at the syscall’s memory addresses and clock, a memory-consistency lookup constraint resolved under the lookup-binding hypothesis, so a trace reading or writing a different address or clock produces an unbalanced lookup. *Output canonicity*. Each output lane s'_j carries the range constraint $0 \leq s'_j < p$ enforced by a byte-wise less-than comparison (`FieldLtCols`); this is the per-row step already proved within Lemma B.5 and is restated here only for the family inventory. *Row scheduling*. The real/padding and first/last-round selector flags enforce one ordered run with cross-row state threading $\text{state_out}_r = \text{state_in}_{r+1}$, realised by the per-row lookup interactions (a RECEIVE at direction r and a SEND at direction $r+1$) under the lookup binding (the lookup-binding hypothesis); a ghost or mis-ordered row creates an unbalanced lookup. *Parameter immutability*. The modulus, round count, linear-layer entries, round constants, and quadratic coefficients are preprocessing-bound rather than prover-supplied, so a per-row choice of any of them has no satisfying assignment.

By row scheduling, and given the lookup binding (the lookup-binding hypothesis), the per-round equalities of Lemma B.5 chain into 14 sequential rounds on one state; with the round count fixed by the lookup schedule (round count, the lookup-binding hypothesis) and with output canonicity and parameter immutability, this is the reference permutation, and with memory binding it is bound to the syscall’s memory input and output. $\square$

B.4 Completeness

Lemma B.7 (Completeness). *For every input $s \in \mathbb{F}_p^4$, the honest Griffin execution on s yields a trace satisfying $\mathcal{C}$ with output $\pi(s)$.*

Proof. Following the practice of [71], where soundness is the load-bearing direction and completeness is discharged operationally, we support the claim by the tests that evaluate the constraints on an honest trace: the single-permutation smoke proof that completes execute, prove, and verify (Section 5), and the fault-injection suite’s positive control, in which the uncorrupted satisfying trace yields zero failing rows under `debug_constraints` (Section B.6). The 256-vector randomised suite and the hand-picked and zero/high-entropy vectors pin the trace generator’s underlying executor against the reference computation and do not themselves evaluate the constraints. $\square$

B.5 The argument layer

Theorem B.3 concerns the *arithmetisation*. The surrounding claim, that an accepting receipt implies a satisfying trace and that a witness is extractable, is the *knowledge-soundness* of the zkVM’s STARK, a separate object we do not re-establish. We take it in the IOP and extractor sense of ethSTARK [70] (Definition 4): for every prover producing an accepting proof with non-negligible probability there exists an extractor outputting a satisfying trace, up to a negligible knowledge error. This is the knowledge-soundness hypothesis of the preservation theorem of Section 6, itself obtained through the Fiat–Shamir and BCS compilation of an interactive oracle proof. The arithmetisation soundness of this appendix and the argument knowledge-soundness are independent objects. End-to-end soundness needs both, and we keep them separate.

B.6 The open step

The compositional argument of Theorem B.3 is unconditional given Lemmas B.4, B.5, and B.6 and the premise that the implemented chip enforces at least the analysed constraint set $\mathcal{C}$ (the chip carries further hardening checks beyond $\mathcal{C}$, which only shrink the satisfying set). That premise, the fidelity of the code-to-constraint transcription, is the step we do not machine-verify. The evidence for it is the cross-codebase agreement of the chip’s native compute with the reference permutation on 256 random vectors, with per-vector false-pass at most $1/p \approx 2^{-199}$ and aggregate at most 2^{-191} across the suite, together with the hand-picked and zero/high-entropy vectors. This per-vector bound assumes a wrong executor produces outputs uniform over $\mathbb{F}_p$ and independent of the reference; a deterministic structural bug agreeing on every sampled input would not be caught, so 2^{-191} bounds only random disagreement on the sampled set and is *not* a bound on the open constraint-level premise. The suite pins the *executor* against the reference, but the *constraint system* checked against that executor is what the vector suite does not reach. A separate fault-injection suite reaches the constraint system in the

rejection direction for seven of the ten families (the per-row families): a single-cell corruption of a satisfying trace is rejected by the per-row constraints in each case, including the linear-layer lanes 1 and 3 that distinguish the implemented symmetric circulant from the standard one. This evidence is itself bounded, and we state the bounds. It is evaluated through `debug_constraints`, which checks the per-row algebraic constraints but not the cross-table lookups, so the lookup-borne families (round count, memory binding, and the cross-row threading of row scheduling) lie outside its reach and would require a full-prover harness; parameter immutability is enforced by construction, with no prover-chosen column to reject. The suite thus shows that specific local deviations are caught, not that no wrong witness satisfies $\mathcal{C}$, which remains the open premise. Closing it fully requires deriving the AIR witness directly from the reference code and checking, ideally by machine, that $\mathcal{C}$ admits only that witness. We therefore report Theorem B.3 as *proved* at the specification level for the per-row layer (Lemmas B.4 and B.5, conditional on the range predicate that the byte-table lookups discharge under the lookup-binding hypothesis), argued for the lookup-borne layer (Lemma B.6), and supported empirically at the implementation level; what Assumption 6.1 still carries is the code-to-constraint transcription fidelity together with the lookup-borne families, for both this theorem and the conditional precompile-soundness theorem Theorem 5.1.

Proof mode (substrate / configuration)	Produced?	Zero-knowledge?	Post-quantum?	Resource outcome
SP1, PLUM verify, Griffin precompile (core)	yes	no	yes	completes, 14.25 min
SP1, PLUM verify, SHA-3 control (core)	yes	no	yes	completes, 13.05 min
SP1, PLUM verify, Groth16/PLONK wrap	no	yes	no	structural: recursion-shape provisioning wall (OOM ≈ 20 GB, default conc.)
RISC Zero, PLUM verify, sys_bigint, SHA-3 hasher (succinct)	yes	blinded; ZK not formally established [‡]	yes	completes, 6 h 19 m
RISC Zero, BDEC CreGen (succinct)	no	n/a	n/a	≥ 15.6 h, internal-verifier rejection
SP1, BDEC CreGen (succinct)	yes	no	yes	completes, 27–31 min
SP1, BDEC ShowCre $k=1, 2$ (succinct)	yes	no	yes	completes, 40–58 min
Aurora, Loquat single-verify	yes	no [†]	yes	completes, 3.76 min
Aurora, Loquat single-verify, ZK enabled	yes	yes	yes	completes, ≈ 22 min ($n=3$)

Table 7: Proof-mode feasibility matrix. Of the modes *produced* on the target hardware by the zkVMs, none is dependably zero-knowledge: the SP1 rows are not zero-knowledge, the RISC Zero receipt carries no formally established zero-knowledge guarantee, and the only zero-knowledge mode the zkVM toolchain exposes (row 3, a pairing wrap) is neither post-quantum nor produced as the toolchain ships, which is the dual obstruction. The zero-knowledge mode that is produced on this hardware is the static-circuit Aurora run (last row; §7.4). [†]The zero-knowledge-enabled Aurora run is now measured (last row); the zk-disabled row is retained for comparison. Both Aurora rows achieve 107.5 bits, below the $\lambda = 128$ target. [‡]RISC Zero’s succinct receipt carries zero-knowledge blinding, but the vendor disclaims a formal zero-knowledge guarantee, so we treat the receipt as not dependably zero-knowledge [2, 3] (Section 6). The SHA-3 row is a control, not a deployable PLUM variant.

Localised change	Static circuit (Aurora)		zkVM	
	N_{circ}	N_{param}	N_{circ}	N_{param}
Presentation change ($\phi_1 \rightarrow \phi_2, k:1 \rightarrow 2$)	≥ 1	≥ 1	0	0
Signature swap (Loquat $\rightarrow$ PLUM)	≥ 1	≥ 1	$\geq 1^*$	$\geq 1^*$
Security retune ($\lambda=80 \rightarrow 128$)	≥ 1	≥ 1	0	0

Table 8: Circuit/parameter regeneration footprint ($N_{\text{circ}}, N_{\text{param}}$ of Definition 4.6). The static-circuit regime regenerates and re-audits its R1CS and constraint-dependent parameters on every localised change; the zkVM absorbs a change as a guest-program edit, leaving the universal relation unchanged ($N_{\text{circ}} = N_{\text{param}} = 0$), except where the change crosses the precompile boundary. ^{*}The Loquat $\rightarrow$ PLUM swap moves the hash field from $\mathbb{F}_{p^2}$ (127-bit) to the 199-bit $\mathbb{F}_p$, so the field-specific Griffin AIR is regenerated (§4.3 carve-out). The components $N_{\text{src}}, N_{\text{audit}}$ are reported in prose below.

R1CS size (constraints)	Aurora param. setup (one component of R_{static})	Fractal indexer (one component of R_{static})
2^{10} (1,024)	0.0021 ms	0.86 s
2^{12} (4,096)	0.0023 ms	3.74 s
2^{14} (16,384)	0.0025 ms	18.5 s
2^{19} (524,288; Loquat single-verify)	0.0029 ms	OOM-killed at ≈ 33 min

Table 9: Per-change recompile components for a non-preprocessing (Aurora) versus a preprocessing (Fractal) transparent argument on the same 127-bit-field harness, M5 Pro 24 GB. The 2^{10} – 2^{14} rows are synthetic R1CS instances; the 2^{19} row is the single-Loquat-verification relation at the BDEC building-block size. Aurora has no per-circuit indexer (parameter setup flat at $\sim \mu\text{s}$, so R_{static} stays sub-second); Fractal’s indexer is superlinear and exceeds 24 GB at the 2^{19} size (OOM mid-indexer). The zkVM regenerates no circuit.